<TAR UMT Supervisor Selection and Allocation System (SAAS)> : <Availability & Status Module, Discovery & Search Module, Request & Proposal Module, Timeline & Milestone Module, Student Profile Module, Student Review Module>

By

Ting Rong You

FACULTY OF COMPUTING AND
INFORMATION TECHNOLOGY

TUNKU ABDUL RAHMAN UNIVERSITY OF
MANAGEMENT AND TECHNOLOGY
KUALA LUMPUR

ACADEMIC YEAR
2025/26

TAR UMT Supervisor Selection and Allocation System (SSAS)

By

Ting Rong You

Supervisor: Mr. Zahrul Azwan Absl Kamrul Adzhar

A project report submitted to the
Faculty of Computing and Information Technology
in partial fulfillment of the requirement for the
Choose an item.

Faculty of Computing and Information Technology
Tunku Abdul Rahman University of Management and Technology
Kuala Lumpur

Declaration

The project submitted herewith is a result of my own efforts in totality and in every aspect of the project works. All information that has been obtained from other sources had been fully acknowledged. I understand that any plagiarism, cheating or collusion or any sorts constitutes a breach of TAR University rules and regulations and would be subjected to disciplinary actions.

_____

Ting Rong You

Choose an item.

ID: 24WMR08039

Abstract

This is a **one-page** summary of the project (without any subheading) of usually not more than **300 word**s. It is often used to help the reader to quickly ascertain the documentation's purpose. Like all summaries, abstract covers the main points of a piece of writing that includes the field of study, problem statement, methodology adopted, research process, conclusion and planning of the project work, etc. It helps readers understand the project by acting as a pre-reading outline of key points.

The contents that you may include in an abstract are as follows:

***Purpose**:* Describes the main purpose of carrying the project, i.e. to justify the existence of the project by stating the existing problem to be solved. Explain why it is necessary to solve it and how the project contributes to the solution.

***Scope**:* This covers areas or size of the project in terms of function and features of the system, modules or sub-modules involved, and functional areas covered in an organization.

***Methodology**:* To describe the methodology, tools, techniques and models that are used throughout the project.

***Testing criteria used**:* To describe the various assessment areas this project has undergone.

***Results and Conclusion**:* To brief the outcome, strengths and weakness of the system or the entire project.

Note: Your abstract SHOULD NOT have bullet points or headings according to the contents listed above. Refer to abstracts of research papers for examples of abstracts.

Acknowledgement

This contains acknowledgement to those who have contributed directly or indirectly to the completion of the project. Usually the people to be acknowledged include the project supervisor(s), moderators, family, and those who have given assistance and supports to ensure the success of the project.

Table of Contents

1 Introduction **2**
1.1 Background Study 3
1.2 Problem Statement 7
1.2.1 Real-World Problem Context 8
1.3 Objectives 12
1.3.1 General Objectives 12
1.3.2 Specific Objectives 12
1.4 Scope 13
1.4.1 User Scope 13
1.4.2 System Scope 15
1.4.3 Out of Scope 17
1.4.4 Distribution of Scope 18
1.5 Hardware and Software Requirements 20
1.5.1 Hardware Requirements 20
1.5.2 Software Requirements 22
1.6 Significance of Project 23
1.6.1 Alignment with UN Sustainable Development Goals (SDGs) 24
1.6.2 Commercialisation Potential and Innovative Features 27
1.7 Constraints and Critical Assumptions 28
1.7.1 Project Constraints 28
1.7.2 Critical Assumptions 29
1.8 Project Management 30
1.8.1 Hardware Costs 30
1.8.2 Software Costs 31
1.8.3 Project Timeline 32
1.8.4 Feasibility Analysis (TELOS) 39
1.9 Chapter Summary and Evaluation 40
2 Literature Review **42**
2.1 Domain Analysis 42
2.1.1 The Significance of the Student-Supervisor Match 43
2.1.2 The Manual Approach (Spreadsheets and Email) 44
2.1.3 The Automated Approach 45
2.1.4 Comparison Summary of Allocation Models 46
2.1.5 Summary of Findings 49
2.2 Review of Similar Existing Systems 50
2.2.1 Institutional Staff Directories (Static Web Portals) 50
2.2.2 Learning Management Systems (LMS) 51
2.2.3 Social Media and Informal Information Networks 52
2.2.4 Comparative Analysis of Existing Frameworks 55
2.3 Technology Review 57
2.3.1 Platform Architectural Analysis 58
2.3.2 Database Technology Analysis 66

2.4 Chapter Summary and Evaluation 70
2.5 Citation and References 72
3 Methodology and Requirements Analysis 76
3.1 Methodology 76
3.1.1 Overview of the Chosen Methodology 77
3.1.2 Justification of the Waterfall Methodology 78
3.1.3 Application of the SDLC to the SSAS 80
3.2 Requirement Gathering Techniques 82
3.2.1 Asynchronous Unstructured Interview (Email) 83
3.2.2 Observation and Document Analysis of the “As-Is” System 84
3.2.3 Role-Based Structured Questionnaires 86
3.3 Requirement Analysis 88
3.3.1 Analysis of Asynchronous Unstructured Interview Findings 89
3.3.2 Analysis of Observation and Document Analysis Findings 91
3.3.3 Analysis of Role-Based Structured Questionnaires Findings 94
3.3.4 Summary of Requirement Analysis Findings 97
3.3.5 Use Case Diagram and Use Case Description 100
3.4 Functional and Non-Functional Requirements 124
3.4.1 Functional Requirements (FR) 125
3.4.2 Non-Functional Requirements (NFR) 133
3.5 Chapter Summary and Evaluation 136
4 System Design 139
4.1 Process Design (Activity Diagram) 140
4.1.1 Supervisor Availability & Status Module 141
4.1.2 Discovery & Search Module 142
4.1.3 Request & Proposal Module 144
4.1.4 Student Profile Module 146
4.1.5 Student Review Module 147
4.1.6 Timeline & Milestone Module 149
4.2 Data Design 150
4.2.1 Context Diagram (DFD Level 0) 150
4.2.2 Data Flow Diagram Level 1 (Student Subsystem) 152
4.2.3 Data Flow Diagram Level 2 (Student Subsystem) 155
4.2.4 Entity-Relationship (ER) Diagram (Without Attributes) 162
4.2.5 Entity-Relationship (ER) Diagram (With Attributes) 164
4.2.6 Data Dictionary 166
4.3 UI Design 176
4.4 Security Design 182
4.5 Structural Design 188
4.5.1 Supervisor Availability & Status Module 188
4.5.2 Discovery & Search Module 192
4.5.3 Request & Proposal Module 196
4.5.4 Student Profile Module 200
4.5.5 Student Review Module 202

4.5.6 Timeline & Milestone Module 205
4.6 System Architecture Design 209
4.7 Algorithm Design 213
4.7.1 Multi-Criteria Supervisor Discovery and Capacity Filtering Algorithm 213
4.7.2 Temporal Phase Evaluation & System Lock Algorithm 218
4.7.3 Supervisor Recommendation & Matching Engine 221
4.8 Chapter Summary and Evaluation 227
5 Implementation and Testing 229
5.1 Implementation 230
5.1.1 Sub-subsection Heading 230
5.2 Sub-section 2 Heading 230
5.3 Sub-section 1 Heading 230
5.3.1 Sub-subsection Heading 230
5.4 Sub-section 1 Heading 230
5.4.1 Sub-subsection Heading 230
5.5 Chapter Summary and Evaluation 230
6 System Deployment 232
System Backup and Risk Management 232
On-site Setup 232
Training Procedure 232
Follow-up 232
Chapter Summary and Evaluation 232
7 Discussions and Conclusion 234
Summary 234
Achievements 234
Contributions 234
Limitations and Future Improvements 234
Issues and Solutions 234
References 235

Chapter 1

Introduction

1 Introduction

This chapter serves as the foundational framework for the proposed TAR UMT Supervisor Selection and Allocation System (SSAS). It begins by establishing the context of the study, analysing the operational inefficiencies within the current manual supervisor selection process at Tunku Abdul Rahman University of Management and Technology (TAR UMT). Following the background study, the chapter defines the specific problem statement, the project objectives, and the significance of the proposed solution.

Critically, this chapter also establishes the boundaries of the development. While the SSAS is a collaborative group project, this individual report specifically focuses on the design and development of the student-facing and front-end interaction modules, namely, the Supervisor Availability & Status Module, Discovery & Search Module, Request & Proposal Module (Student View), Timeline & Milestone Module, Student Profile Module, and Student Review Module.

The backend administrative modules (such as Student Management, Allocation & Quota, and Administrator Report) are developed collaboratively with the project teammate and are detailed in the “Distribution of Scope” section to provide context.

1.1 Background Study

1. **The Role of Final Year Projects in Higher Education**
   The Final Year Project (FYP) serves as a terminal educational undertaking that provides a capstone experience for undergraduate students, challenging them to synthesise the knowledge, skills, and experience they have acquired throughout their programme. A critical determinant of FYP success is the supervision process, which matches students with appropriate academic staff. Research indicates that the efficiency of this matching process, pairing the right student with the right supervisor based on expertise, is fundamental to both the student's academic performance and the fair distribution of workload among faculty members (Hussin & Azlan, 2017).

2. **Current Operational Context at TAR UMT**
   In its pursuit of a global digital revolution, the administration of Tunku Abdul Rahman University of Management and Technology (TAR UMT) has successfully implemented the TAR UMT Intranet and the TARCApp ecosystem. However, the administration of FYP supervision currently suffers from a significant operational lag. At present, the supervisor selection process remains decentralised. The primary medium for administration is static Excel spreadsheets, with email serving as the secondary communication channel. This reliance on manual tools has led to issues regarding document version control and the failure of email systems to provide the necessary quota validation to prevent over-subscription.

3. **Challenges and Inefficiencies**
   The reliance on manual workflows creates distinct challenges for all stakeholders:
   - **For Students:** The process is chaotic and opaque. Lacking a centralised platform to view real-time supervisor availability, students are forced into a "blind" selection process. This triggers a "first-come, first-served" panic, where students select supervisors based solely on availability rather than academic alignment.
   - **For Supervisors:** The absence of a Request Management Module results in an overwhelming administrative burden. Supervisors must manage a high volume of unstructured email applications. Furthermore, the lack of system-level quota enforcement leads to "over-booking" where supervisors unintentionally receive applications beyond their authorised limits.
   - **For Administrators:** Manual allocation processes carry considerable governance risks. Research by Ugboaja and Eziechina (2021) highlights that replacing manual tracking with centralised web portals successfully automates routine administrative activities, thereby eliminating data duplication and procedural delays. Furthermore, the current practice of maintaining supervisor quotas in isolated spreadsheets is highly susceptible to critical human error and unauthorised data manipulation, a foundational vulnerability extensively documented in spreadsheet risk research (Panko, 2005). Transitioning to a web-based architecture resolves this by enforcing Role-Based Access Control (RBAC), which guarantees the safe handling of private student data and ensures that only authorised personnel can alter system quotas (Sahare, 2025).

4. **Alignment with Institutional Strategy**
   The persistence of these labour-intensive, error-prone processes stands in direct contradiction to the university's strategic vision. As outlined in the TAR UMT 10-Year Roadmap (2021-2030), the institution has established a strict mandate to accelerate the digitalisation of work processes.

   As illustrated in Figure 1.1, the university's short-to-mid-term plan explicitly calls for the identification of manual processes in order to "automate and digitise manual administration" and "create self-service customer experience" (Tunku Abdul Rahman University of Management and Technology - 10-year Roadmap (2021-2030), n.d.).

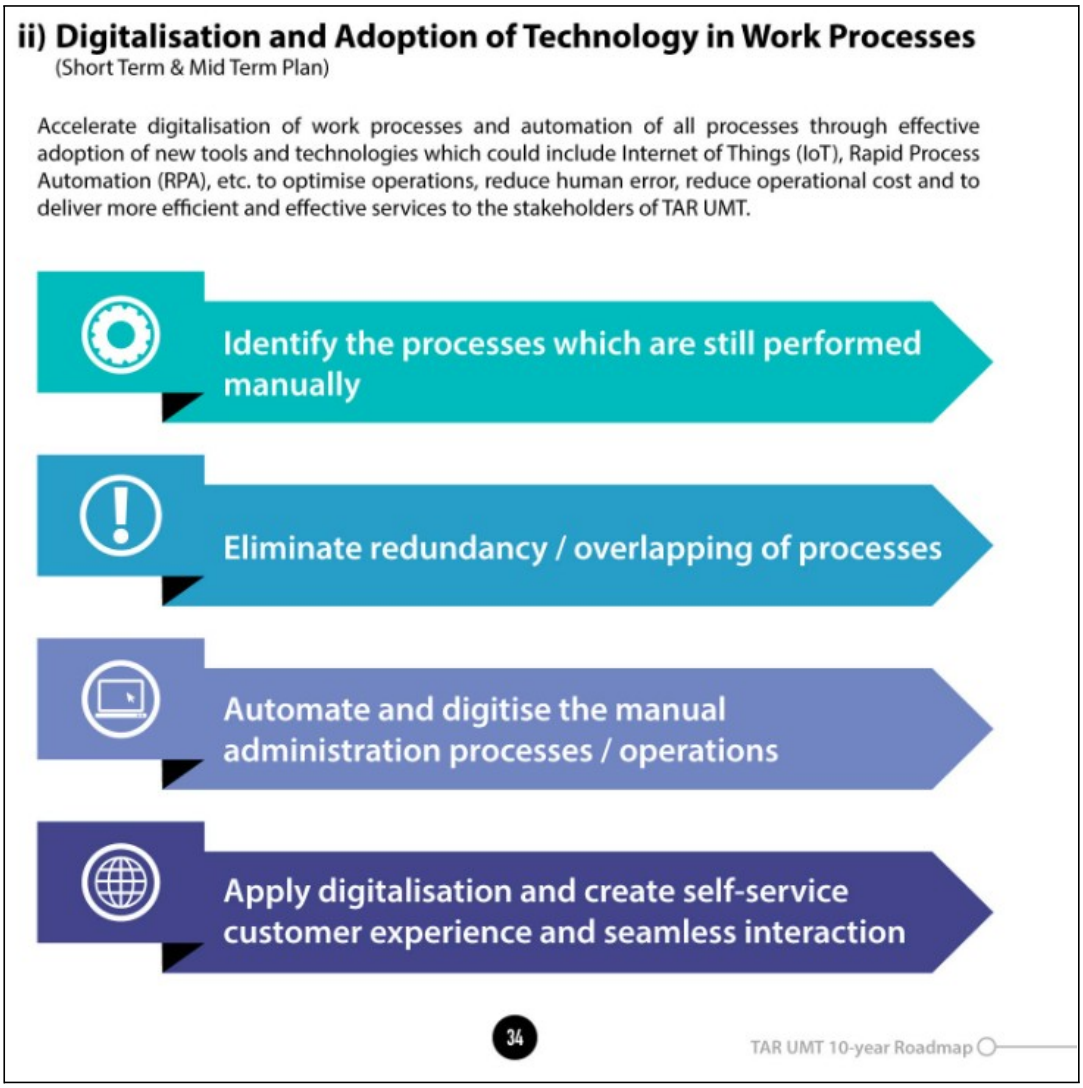

Figure 1.1: Excerpt from the TAR UMT 10-Year Roadmap detailing the strategic mandate for digitalising manual workflows.

Continuing to utilise unstructured data files, such as isolated Excel spreadsheets, to manage vital academic processes creates a widening “digital gap” between the university’s forward-looking mission and operational reality. By replacing the manual supervisor allocation process with a centralised, automated web portal, the proposed SSAS directly acts upon the university’s directive to eliminate process redundancy and deliver more efficient services to TAR UMT stakeholders.

5. The Need for a Technology Solution

Considering the outlined inefficiencies, there is an urgent requirement to transition from the current manual process to a Supervisor Selection and Allocation System (SSAS). This new system must not only digitise the workflow but also incorporate logic-based governance to resolve the “blind selection process”.

This involves developing a Real-Time Quota Engine to strictly enforce data integrity and prevent “over-booking”, alongside a student-centric Discovery Module to ensure transparency and informed decision-making. Such a comprehensive solution is crucial to align TAR UMT’s operational capabilities with its strategic goals, eliminating the administrative chaos that currently affects the academic community.

1.2 Problem Statement

The current Final Year Project (FYP) supervisor selection process at Tunku Abdul Rahman University of Management and Technology (TAR UMT) relies on a decentralised, manual workflow conducted primarily through unstructured email communication and static Excel spreadsheets. This traditional approach has resulted in significant operational inefficiencies that negatively affect both students and academic staff.

1. **Lack of Transparency and Student Uncertainty**

   From the student's perspective, the absence of a centralised platform creates a "blind" selection environment. Research indicates that students in manual allocation systems often suffer from high anxiety due to a lack of visibility and poor matching process (Kushwah & Navrouzoglou, 2022). At TAR UMT, students are forced to rely on a "first-come, first-served" panic approach, sending mass emails without knowing if a supervisor is already fully booked. This opacity often leads to poor decision-making, where students prioritise availability over academic compatibility (Tam & Abdullah, 2025).

2. **Unstructured Request Handling and Communication Chaos**

   The supervision request procedure is entirely dependent on unstructured email communication. During peak selection windows, supervisors are inundated with a high volume of student enquiries, which they must track manually. Manual tracking of such high-volume requests is inherently prone to errors and bottlenecks (Chikwendu, 2021). The risk of missed emails is high, and students often face long waiting periods without any formal status update, leading to repeated follow-up enquiries that further clog the communication channel.

3. **Inefficient Matching and Difficulty in Discovery**

   Under the current system, there is no efficient mechanism for students to discover supervisors based on specific research domains (e.g., IoT, Machine Learning). Students are often limited to viewing a static name list without detailed insights into a lecturer's expertise. Studies show that without intelligent discovery tools, students frequently apply to supervisors whose expertise does not align with their proposed topics, resulting in mismatched expectations, lower project quality, and poor relationship between the student and the supervisor (Hussain et al., 2019).

4. **Operational Risks and Data Inconsistency**

   The manual consolidation of allocation data involves high risks of human error. Issues such as multiple assignments for a single student or "over-booking" frequently occur due to the lack of real-time validation. As foundational research by Panko (2016) notes, manual spreadsheet-based systems lack strict automation controls, making them highly vulnerable to

data integrity failures during manual consolidation. This guarantees a high statistical probability of human error, requiring significant administrative intervention to resolve.

1.2.1 Real-World Problem Context

In the actual operational environment at TAR UMT, supervisors frequently experience a "flood" of email requests within a very short timeframe. Because there is no system to physically block incoming requests once a quota is reached, supervisors often continue to receive applications even after they are full. This forces them to send manual rejection emails to dozens of students, a time-consuming process that could be entirely eliminated by a digital system with real-time status enforcement.

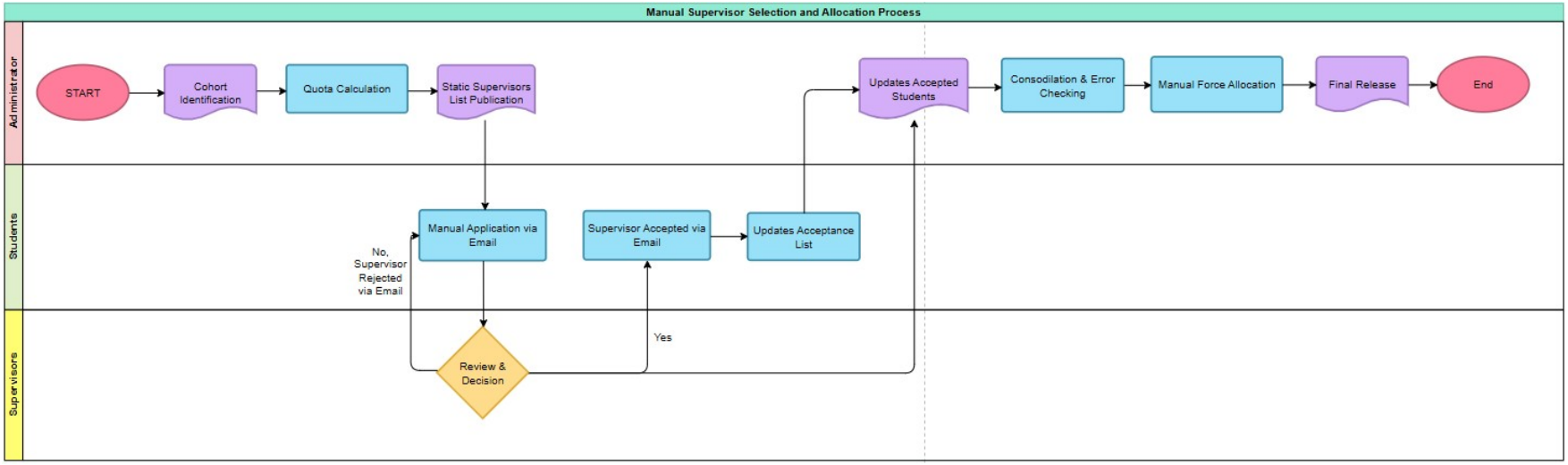

Figure 1.2: Analysis of the Legacy Manual System Workflow

As illustrated in the cross-functional flowchart (Figure 1.2), the current Supervisor Selection and Allocation process at TAR UMT relies heavily on decentralised, manual handoffs between Administrators, Students, and Supervisors. A critical analysis of this "As-Is" workflow reveals several severe operational bottlenecks and points of failure that justify the development of the automated SSAS.

Process Overview:

The manual workflow, as depicted in Figure 1.2, begins with the Administrator identifying the eligible student cohort and calculating the maximum supervisory quotas. A static Excel supervisor list is then published. Students must manually approach and upload their project proposals via email. Upon review, if a supervisor rejects a student's proposal, the student needs to manually approach other supervisors and repeat the process. Conversely, if a supervisor accepts a proposal, a decentralised recording process begins, the student must manually update a shared acceptance list, while the supervisor is required to update their accepted students to the administrators via unstructured reporting channels. The administrator is then required to manually consolidate these disparate files and information, cross-reference them against the student's self-reported list, and force-allocation of any remaining unassigned students before publishing the finalised faculty roster.

Identification of Critical Pain Points:

1. **The "Static Data" Trap (List Publication)**
    a. **The Flaw:** During the supervisors list publication phase, the Administrator publishes a "Static Supervisors List." Since this document is static (Excel file), it does not update in real-time when a lecturer's quota fills up.
    b. **The Impact:** Students unknowingly continue to waste time applying to supervisors who are already at maximum capacity, creating unnecessary administrative spam.
2. **The "Rejection Loop" and Unmonitored Channels**
    a. **The Flaw:** The core application process relies entirely on manual emails. If a supervisor rejects a proposal, the student is thrown into a cyclical loop, forced to restart the "Manual Application via Email" process from scratch.
    b. **The Impact:** Since this communication happens in private email inboxes, the administrators have no visibility into pending applications, wait times, or student progress until the final deadline.
3. **The Double-Data Entry Conflict**
    a. **The Flaw:** The diagram clearly shows two separate data streams flowing back to the Administrator, one from the student (Updates Acceptance List) and another from the supervisor's decision.
    b. **The Impact:** This creates a dangerous "Double Data Entry" scenario. The administrator is forced to act as a human reconciliation engine, hoping that the student's self-reported list perfectly matches the supervisor's private records. If the supervisors did not update their accepted students to the administrator, the administrator might consider ignoring the student's self-reported list and reallocate them to other supervisors.
4. **The Ultimate Bottleneck: Consolidation & Error Checking**
    a. **The Flaw:** Since the data is collected via unstructured emails and isolated lists, the "Consolidation & Error Checking" step falls entirely on manual effort.
    b. **The Impact:** As foundational software engineering research notes, manually merging dozens of isolated spreadsheets guarantees a high statistical probability of human error, such as accidental data duplication, overwritten cells, or undetected quota violations.

1.3 Objectives

1.3.1 General Objectives

The primary goal of this project is to architect and develop a centralised Supervisor Selection and Allocation System (SSAS) for TAR UMT. This system aims to replace the existing decentralised manual processes with a fully digital platform that automates student-supervisor matching, enforces quota management, and streamlines the project allocation workflow.

1.3.2 Specific Objectives

The specific objectives of this project are as follows:

1. **To engineer a multi-criteria discovery engine with a content-based recommendation algorithm**

   To develop an intelligent search module that mathematically quantifies the overlap between student research tags and supervisor expertise, ranking the top three academically compatible with sub-millisecond *O(n)* latency to optimise the discovery process.

2. **To digitise student-supervisor engagement by replacing manual email-based communication with a structured Request Management Module**

   To replace decentralised email interactions with digital submission workflow that enforces a 72-hour Time-To-Live (TTL) for responses and provides real-time status tracking to eliminate administrative delays and “ghosting”.

3. **To implement a secure supervisor evaluation module with data-masking privacy controls**

   To design a review system to strictly segregate administrative traceability from frontend visibility, allowing students to submit quantitative star ratings and qualitative feedback while maintaining optional anonymity through server-side identity masking.

4. **To design a centralised temporal synchronisation engine for global system integrity**

   To create a milestone tracker that enforces academic deadlines via server-side UTC synchronisation, automatically invoking global system locks to revoke submission privileges and ensure 100% adherence to the university’s administrative timeline..

1.4 Scope

The scope of this project encompasses the design, development, and testing of a web-based Supervisor Selection and Allocation System (SSAS) for Tunku Abdul Rahman University of Management and Technology (TAR UMT).

The primary aim of this system is to digitise the current manual supervisor selection process, enhancing efficiency, transparency, and data integrity in FYP supervision management. The project boundaries are defined in terms of User Scope, System Scope, and Out of Scope functionalities.

1.4.1 User Scope

This project involves three primary user groups, each with clearly defined roles and access privileges within the system.

A) Final Year Project (FYP) Students

The system empowers students to proactively manage their supervisor selection process:

- **Real-Time Availability Viewing:** Access to a dynamic dashboard that displays supervisor availability and quota status in real-time.
- **Request Management:** The ability to submit supervision proposals and track the lifecycle of their requests (Pending, Accepted, or Rejected).
- **Automated Placement Participation:** For students who remain unassigned after the deadline, the system provides an automated allocation mechanism based on programme compatibility.
- **Deadline Monitoring:** Visual alerts and countdown for critical selection phases to ensure timely submissions.

B) Supervisors (Academic Staff)

The system provides academic staff with comprehensive tools to manage their professional presence and workload:

- **Professional Profile Management:** A "Digital Business Card" interface allowing supervisors to showcase their research interests, define expertise tags, and upload introductory videos to attract suitable students.
- **Request Arbitration:** A centralised inbox to review student proposals and execute "Accept" or "Reject" decisions with operational feedback.
- **Workload Monitoring:** Access to a dedicated Supervisor Report Module, featuring an "Applicant Demographic Chart" and "Slot Utilisation Tracker" to monitor personal capacity against faculty averages.
- **History Tracking:** A digital archive of previously supervised student cohorts organised by academic session.

C) Administrators / Programme Coordinators

The system equips administrators with high-level governance tools to manage cohorts and enforce rules:

- **Eligibility & Classification:** Automated logic to verify student eligibility and classify supervisors (Full-Time vs Part-Time) for accurate quota calculation.
- **Quota Enforcement:** Tools to define and enforce maximum supervision limits per supervisor type, preventing the risk of "over-booking".
- **Auto-Allocation Execution:** The ability to trigger the Auto-Allocation Engine to assign unattached students to available slots based on pre-defined logic.
- **Cohort Reporting:** Access to the Administrator Report Module for real-time insights into "Cohort Overview" (assigned vs unassigned stats) and allocation fill rates.

1.4.2 System Scope

The system scope describes the specific functional modules that will be developed to support the student-facing operations and core matching logic of the system. The following modules comprise this scope of development:

1. **Supervisor Availability & Status Module**
    - **Real-Time Dashboard**
      Visualises granular quota data (e.g., '3/8 slots taken') to provide immediate transparency on vacancy levels.
    - **Dynamic Status Badges**
      Utilises visual indicators (Green for 'Available', Red for 'Full') to allow students to instantly identify open supervisors.
    - **Submission Blocking**
      A logic-based constraint that physically disables the "Apply" button for supervisors who have reached their maximum quota, preventing "over-booking".
2. **Discovery & Search Module**
    - **Multi-Criteria Filtering Engine**
      A dynamic interface allows students to narrow down the supervisor list based on Academic Programme, Research Interest (e.g., IoT, AI) and Availability Status.
    - **Recommendation Algorithm**
      An intelligent matching feature that compares a student's self-selected interest tags against supervisor expertise to automatically suggest the "Top 3 Matches".
3. **Request & Proposal Module**
    - **Digital Submission**
      Enables students to upload PDF project proposals and submit formal supervision requests directly to specific lecturers.
    - **Status Tracking**
      Provides a centralised view of application status (Pending, Accepted, Rejected, or Auto-Allocation), eliminating the need for follow-up emails.

4. **Student Profile Module**
    - **Personal Details**

      Enables authenticated students to securely view and update their mutable academic data (e.g., Email Address, CGPA) while locking primary identification attributes (e.g., Student ID) to maintain data integrity.
    - **Interest Tagging**

      Allows students to select their own research interests (e.g., "Machine Learning") to feed the recommendation algorithm.
5. **Student Review Module**
    - **Structured Review System**

      Enables past students to leave quantitative ratings (1 to 5 start) and specific qualitative feedback on their officially assigned supervisors, assisting juniors in making informed decisions.
    - **Anonymous Review Toggle**

      Allows students to post public reviews anonymously while retaining backend identity for verification, encouraging honest feedback.
6. **Timeline & Milestone Module**
    - **Phase Tracker**

      A distinctive visual display of current supervisor selection phases (e.g., "Application Window Open", "Final Lists Published").
    - **Deadline Countdown**

      Visual countdown timers for critical dates (e.g., "3 days left to select supervisor") to create urgency and ensure adherence to schedules.

1.4.3 Out of Scope

The following features are specifically excluded from the current development phase in order to guarantee that the project remains feasible within the allocated timeframe and resources:

- **Native Mobile Application**
  The system is designed to be a mobile browser-accessible, responsive web application. A dedicated native mobile application is not included.

- **Complex Machine Learning**
  The "Auto-Allocation Engine" assigns students using rule-based logic. Predictive AI and machine learning models are not used.

- **Payment Gateway Integration**
  Financial transactions and payment processing functionalities are neither necessary nor included because this is an internal academic management system.

- **Calendar Scheduling**
  The system does not include an integrated calendar for scheduling in-person meetings, this is still an external procedure.

- **Real-Time Chat System**
  External email clients are used for student-supervisor communication. The scope does not include an integrated live chat engine.

- **Alternative Faculty Allocation Workflows**
  The proposed SSAS is specifically engineered to digitise the "Student-Initiated Application" model currently utilised by the target faculty, wherein students proactively approach supervisors with proposals. Alternative administrative workflows utilised by other TAR UMT faculties, such as the "Supervisor-Initiated Selection" model (where supervisors independently select students from a pool) are strictly out of scope for this system iteration. The system architecture assumes the student is the primary initiator of the allocation request.

- **Post-Implementation Maintenance and Hosting**

The project scope concludes at the Testing phase of the Waterfall model. Permanent server deployment, cloud hosting costs, and long-term system maintenance are outside the current project's purview.

1.4.4 Distribution of Scope

To ensure the feasibility of developing the Supervisor Selection and Allocation System (SSAS) within the standard academic timeframe, the system architecture has been divided into two distinct, independent development tracks. This modular approach allows for parallel development, equitable workload distribution, and specialised focus areas for both team members.

The division of labour is structured so that the student-facing journey and the administrator or supervisor backend are handled concurrently. The specific module allocations and primary development roles for the project team are visually mapped in Figure 1.3 and further detailed in Table 1.1.

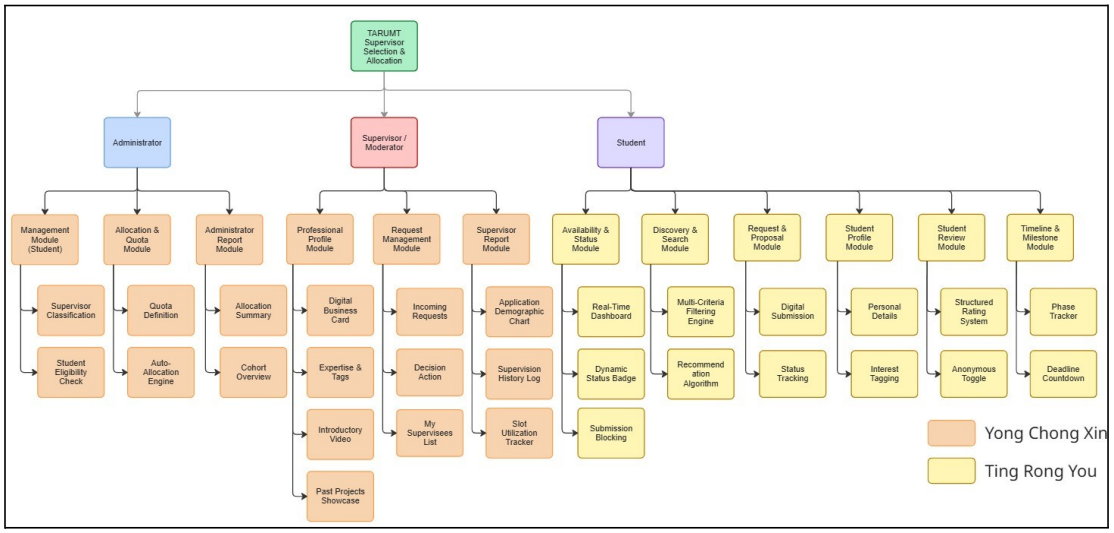

Figure 1.3: Work Breakdown Structure (WBS) illustrating system modules and team member allocation

Table 1.1: Team Workload Breakdown and Module Allocation

Team Member	**Primary Project Role**	**Assigned System Modules & Core Responsibilities**
Ting Rong You	Student Journey Developer & QA Testing Lead	**Student-Based Modules:** ● **Availability & Status Module:** Real-Time Dashboard, Dynamic Status Badges, and Submission Blocking constraints. ● **Discovery & Search Module:** Multi-Criteria Filtering Engine and, Recommendation Algorithm ● **Request & Proposal Module:** Digital Submission and, Status Tracking ● **Student Profile Module:** Personal Details and, Interest Tagging ● **Student Review Module:** Structured Rating System and, Anonymous Toggle ● **Timeline & Milestone Module:** Phase Tracker and, Deadline Countdown
Yong Chong Xin	Supervisor Journey Developer & Backend Architect	**Administrator-Based Modules:** ● **Management Module (Student):** Supervisor Classification and, Student Eligibility Check ● **Allocation & Quota Module:** Quota Definition and, Auto-Allocation Engine ● **Administrator Report Module:** Allocation Summary and, Cohort Overview **Supervisor-Based Modules:** ● **Professional Profile Module:** Digital Business Card, Expertise & Tags, Introductory Video and, Past Project Showcase ● **Request Management Module:** Incoming Requests, Decision Action and, My Supervisees List ● **Supervisor Report Module:** Application Demographic Chart, Supervision History Log and, Slot Utilisation Checker

By dividing the system along these functional boundaries, the team mitigates the risk of code conflicts and ensures that the complex matching algorithms and centralised dashboards can be developed, tested, and integrated systematically by both developers.

1.5 Hardware and Software Requirements

Certain hardware and software resources are needed to guarantee that the Supervisor Selection and Allocation System (SSAS) is developed and implemented successfully.

1.5.1 Hardware Requirements

To ensure optimal performance and accessibility, specific hardware environments are required for both the system creators (development) and the end-users (client). These specifications are detailed in Table 1.2.

Table 1.2: Hardware Requirements for Development and Client Environments

Component	**Minimum Specification**	**Technical Justification & Project Alignment**
Development Environment (System Creators)		
Processor (CPU)	Intel Core i5 / AMD Ryzen 5 (or equivalent)	Ensures smooth, concurrent execution of the local XAMPP web server, database operations, and automated testing frameworks without CPU bottlenecking.
Memory (RAM)	8 GB Minimum (16 GB Recommended)	Required to simultaneously host the local MySQL database, run the IDE (Visual Studio Code), and execute memory-intensive End-to-End (E2E) testing scripts via Katalon Studio.
Storage	256 GB Solid State Drive (SSD)	Provides high-speed I/O operations necessary for rapid database querying, fast local repository management, and swift application loading times.
Display	1920 x 1080 (Full HD) Resolution	Provides adequate screen real estate for split-screen code authoring, database monitoring, and high-fidelity UI/UX prototyping via Figma.
Network / Connectivity	Stable Broadband (Minimum 10 Mbps)	Essential for real-time cloud collaboration on UI wireframes (Figma), continuous integration and repository synchronisation (GitHub), and downloading necessary IDE extensions or testing dependencies.
Operating	Windows 10 / 11 (64-bit)	Required to ensure full compatibility and

System (OS)	or macOS 11+	stable execution of the local XAMPP stack, Visual Studio Code, and the Katalon Studio automated testing framework.
Client Environment (End-Users)		
Desktop / Laptop	Any standard workstation with an active Network Interface Card (NIC)	The web-based portal is hardware-agnostic for clients, requiring only standard internet connectivity to access the system via modern web browsers (e.g., Chrome, Edge).
Mobile Devices	Standard iOS or Android Smartphone	Utilised specifically to validate the system's responsive CSS framework, ensuring proper touch-target sizing and mobile viewport adaptability for student-facing modules.
Network / Connectivity	Standard 4G / Wi-Fi (Minimum 5 Mbps)	Required to maintain stable HTTP / HTTPS sessions, execute asynchronous (AJAX) data requests without timeouts, and ensure real-time quota updates during peak application periods.
Operating System (OS)	Platform Agnostic (Windows, macOS, iOS, Android)	The web-based architecture ensures the SSAS is fully accessible across any modern desktop or mobile operating system, provided a compliant web browser is utilised.

1.5.2 Software Requirements

To guarantee viability, maintainability, and ease of deployment, the project utilises a stack of widely supported, open-source technologies. The software required for the development, architecture modelling, and testing of the SSAS is detailed in Table 1.3.

Table 1.3: Software Requirements for System Development and Deployment

Software Category & Tool	**Technical Justification & Project Alignment**
Development Tools & Version Control	
Visual Studio Code (VS Code)	Primary Integrated Development Environment (IDE) due to its extensive extension ecosystem, syntax highlighting, and debugging support for HTML, CSS, JavaScript, and PHP.
Git & GitHub	Git is utilised as the local version control software to track source code modifications, while GitHub serves as the remote cloud repository to oversee teamwork, backup iterations, and manage deployment branches.
Backend & Database Infrastructure	
XAMPP (Apache Web Server)	Provides the local server environment. Specifically, the Apache component is used to host the application locally and process incoming HTTP / HTTPS requests during development.
PHP 8.x	The primary server-side scripting language used to program the Auto-Allocation Engine, enforce real-time supervisor quotas, and handle secure session management.
MySQL (via phpMyAdmin)	The Relational Database Management System (RDBMS) used to store, query, and manage crucial system data, including supervisor profiles, student records, and allocation statuses.
Frontend Technologies	
HTML5, CSS3, JavaScript	Forms the foundational presentation layer. Used to construct responsive user interfaces and execute asynchronous (AJAX) data updates for real-time quota reflections on the student dashboard.
System Modelling & UI Design	
Figma	Utilised for high-fidelity UI / UX prototyping and wireframing, ensuring the web interface meets usability

	standards before development begins.
Visual Paradigm Online	The primary cloud-based modelling tool used to author standardised Unified Modelling Language (UML) diagrams (e.g., Use Case, Activity diagrams) to map out system architecture and data flows collaboratively.
draw.io (diagrams.net)	Utilised as a lightweight, cloud-based diagramming tool for creating high-level system architecture visualisations, Work Breakdown Structures (WBS), and flowcharts.
Testing, QA & Documentation	
Katalon Studio	Deployed for End-to-End (E2E) automated testing. It simulates user interactions (e.g., simultaneous student logins) to validate system stability, and ensure the quota enforcement logic holds under heavy load.
Google Chrome (with DevTools)	Serves as the primary web browser for evaluating responsive layout rendering across different viewport sizes and debugging JavaScript console errors.
Google Workspace (Docs / Drive)	Utilised for the collaborative drafting, formatting, and cloud storage of all official project documentation, thesis chapters, and proposal reports.

1.6 Significance of Project

By using automation and logic-driven governance to address operational pain points, the development of these core modules offers significant value to the TAR UMT academic ecosystem.

Significance to the Institution:

- **Digital Transformation:** By closing the "digital gap" in academic administration, the project directly supports the TAR UMT 10-Year Roadmap (*Tunku Abdul Rahman University of Management and Technology - 10-year Roadmap (2021-2030)*, n.d.). The shift from an outdated manual workflow to a centralised web-based solution aids the university's objective of becoming a completely digitally connected institution.

Significance to Students:

- **Empowered Matching & Transparency:** The Discovery & Search Module allows students to easily find supervisors aligned with their specific research interests through intelligent recommendation algorithms. Coupled with real-time status badges, students gain immediate visibility into supervisor quotas, eliminating the frustration of applying to fully booked lecturers.
- **Streamlined Workflow & Reduced Anxiety:** The Request & Proposal Module and Timeline & Milestone Module replace scattered email communications with a centralised portal. Students benefit from real-time status tracking, clear deadline visualisers, and structured proposal submissions, significantly reducing the anxiety associated with the selection process.
- **Informed Decision-Making:** The Student Profile and Student Review Module provides access to structured, anonymised feedback from previous cohorts. This peer-driven insight helps juniors make highly informed decisions, ultimately leading to more compatible and successful student-supervisor pairings.

Significance to Supervisors (Operational Impact)

- **Workload Optimisation:** Even from the student-facing side, the Submission Blocking logic lessens the administrative load on lecturers. By physically blocking new submissions once a supervisor reaches capacity, it removes the need for supervisors to manually reject excessive email requests.

1.6.1 Alignment with UN Sustainable Development Goals (SDGs)

The development of the Supervisor Selection and Allocation System (SSAS) actively contributes to the broader global agenda for sustainable, equitable, and modernised institutional practices. Specifically, the system aligns with the core targets of two primary United Nations Sustainable Development Goals (SDGs):

Figure 1.4: UN Sustainable Goals (SDGs)

1. **SDG 4: Quality Education**

   A core tenet of SDG 4 is ensuring equal and transparent access to quality educational resources. The capstone Final Year Project is a critical component of higher education. By reducing opaque, "first-come, first-served" manual allocation methods with a centralised web portal, the SSAS ensures all students have equitable, real-time visibility into supervisor availability. Furthermore, the system's "Discovery & Search Module" pairs students with supervisors based on actual academic expertise rather than chance, directly elevating the quality of the mentorship and the resulting research output.

2. **SDG 9: Industry, Innovation, and Infrastructure**

   SDG 9 emphasises the need to upgrade existing infrastructure to become sustainable, resilient, and technologically advanced. The current operational context at TAR UMT relies on outdated, decentralised digital infrastructure (isolated Excel spreadsheets and unstructured email). The SSAS directly fulfils this goal by digitally transforming a labour-intensive administrative bottleneck into a streamlined, automated, and secure digital workflow. This innovation not only reduces the carbon footprint associated with manual administrative processing but also establishes a scalable technological framework for the university.

1.6.2 Commercialisation Potential and Innovative Features

The Supervisor Selection and Allocation System (SSAS) is designed not only to resolve immediate administrative bottlenecks at TAR UMT but also to serve as a scalable, market-ready software solution. Its architecture incorporates modern industry practices that provide significant potential for commercialisation and technology showcases.

1. **Commercialisation and Scalability (White-Label Potential)**

   While initially scoped specifically for TAR UMT, the core matching logic and quota enforcement algorithms are universally applicable to any academic institution managing capstone projects, practicums, or mentorship programmes. The system is architected as a modular platform, meaning it has high potential to be a "white-labelled" and licensed as a Software-as-a-Service (SaaS) product to other universities facing similar manual allocation challenges. Recent systematic reviews of higher education infrastructure confirm that the adoption of SaaS models by universities is a rapidly accelerating global trend, driven by the need to standardise administrative workflows and reduce on-premise IT overhead (Akande & Van Belle, 2016).

2. **Innovative Features and Industry Trends**

   The SSAS distinguishes itself from standard academic management portals by integrating several advanced, enterprise-grade features:

   - **Intelligent Discovery Engine:** Moving beyond traditional static directories, the system utilises a dynamic recommendation algorithm. By cross-referencing student-selected "Interest Tags" with faculty "Expertise Tags", the system proactively suggests optimal academic pairings, elevating the quality of research outcomes.
   - **Enterprise-Grade Quality Assurance:** Unlike typical academic prototype projects, the SSAS development lifecycle incorporates automated End-to-End (E2E) testing pipelines. This ensures high system reliability under load and demonstrates strict alignment with modern software testing trends.
   - **Dynamic Concurrency Control:** The system replaces the inherent vulnerabilities of static spreadsheets with real-time database locks and strict Role-Based Access Control (RBAC) (Sahare, 2025). This systematically

prevents quota "over-booking" during high-traffic selection windows, a critical innovation over the legacy "first-come, first-served" email method.

1.7 Constraints and Critical Assumptions

A number of limitations and important assumptions have been identified to guarantee the effective delivery of the Supervisor Selection and Allocation System (SSAS) within the project deadline. These elements establish the parameters of the development process and the prerequisites for the efficient operation of the system.

1.7.1 Project Constraints

The development is to subject the following constraints, which govern the technical and operational scope of the student-facing modules:

- **Time Constraints:** The project design, development, and testing phases must be completed within two academic semesters.
- **Technical Environment:** The system is limited to XAMPP-based local development environments. Since the primary goal is to demonstrate the functionality of the matching logic, real-time dashboards, and database integrity, deployment on a live public cloud server is excluded from this phase.
- **Zero-Cost Budget:** There is no financial budget allocated for the project. Consequently, the development relies entirely on open-source technologies (PHP, MySQL) and free tools, prohibiting the use of premium APIs or commercial enterprise software.
- **Policy Compliance (Quota Limits):** The official TAR UMT policies strictly dictate supervision limits. Therefore, the Submission Blocking logic in the system is constrained by these rules and cannot permit any student to bypass a "Full" status, enforcing strict adherence to university governance.

1.7.2 Critical Assumptions

The following assumptions, which are deemed true for the purpose of development, guide the project's matching logic and timeline functionalities:

- **Active Supervisor Participation:** It is assumed that supervisors will actively update their profiles and establish "Expertise Tags". This is a critical prerequisite, without populated profiles, the Recommendation Algorithm and Multi-Criteria Filtering Engine cannot successfully match students to ideal supervisors.
- **Fixed Timeline & Phasing:** It is anticipated that the university's timeline for FYP selection will remain structured. Any sudden, mid-semester changes to how selection phases operate would require significant backend re-engineering of the Timeline & Milestone Module (Phase Tracker and Deadline Countdown).
- **Constructive Student Feedback:** For the Student Profile and Student Review Module to be effective, it is assumed that past students will utilise the rating system to provide honest, constructive feedback rather than malicious reviews, ensuring the data remains valuable for junior cohorts.
- **Stable Internet Connectivity:** Since the platform features real-time dynamic status updates, it is presumed that all users will access it using modern web browsers (e.g., Chrome, Edge, or Safari) with reliable internet connectivity during peak selection windows.
- **Standardised Student-Initiated Workflow:** The core system architecture fundamentally assumes that the target faculty will maintain its current "Student-Initiated" allocation policy, it is assumed that the primary operational flow will continue to require students to proactively submit proposals to supervisors, rather than transitioning to a top-down, supervisor-initiated assignment model.

1.8 Project Management

For the Supervisor Selection and Allocation System (SSAS) to be implemented on schedule and within scope, effective project management is crucial. The project budget estimates the and the development schedule are described in this section.

1.8.1 Hardware Costs

The available personal computer resources of the development team are utilised to construct the project. Since this is a student project operating under a zero-cost paradigm, no new hardware needs to be purchased. Table 1.4 outlines the estimated worth of the existing hardware utilised:

Table 1.4: Hardware Costs

Item Description	**Specification**	**Estimated Cost (RM)**	**Remarks**
Development and Testing Laptop (Model: Acer Nitro AN515-58)	12th Gen Intel Core i7-12650H (2.70 GHz) / 16 GB RAM / 512 GB SSD	0.00	Existing Personal Asset. Utilised for backend PHP coding, local XAMPP database hosting, and running Katalon Studio tests.
Mobile Testing Device (Model: Samsung Galaxy Note 10 Plus)	Android Version 12, One UI version 4.1 Standard Android OS / Mobile Viewport	0.00	Existing Personal Asset. Utilised specifically for validating the responsive CSS framework and mobile interface usability.
Backup Storage (Model: Kingston USB Flash Drive)	16 GB Capacity	0.00	Existing Personal Asset. Utilised for daily offline source code backups and local versioning redundancy.
Total Hardware Cost		0.00	

1.8.2 Software Costs

To keep the project affordable without sacrificing functionality or performance, the development environment strictly utilises free-tier tools and open-source technologies. Table 1.5 details the software stack utilised under this zero-cost paradigm:

Table 1.5: Software Costs

Software Item	**Licence Type**	**Estimated Cost (RM)**	**Purpose**
Windows 11 Home	Proprietary (Pre-installed)	0.00	Base Operating System for the development environment.
XAMPP Server	Open Source (GPL)	0.00	Local Web Server (Apache) and Database Management (MySQL).
Visual Studio Code	Open Source (MIT)	0.00	Primary IDE for Code Editing and Debugging.
Git & GitHub	Open Source / Free Tier	0.00	Local Version Control and Cloud Team Collaboration.
Figma	Starter Plan (Free)	0.00	UI / UX Prototyping and Wireframing.
draw.io (diagrams.net)	Open Source / Freeware	0.00	High-level Flowcharts and WBS Diagramming.
Visual Paradigm Online	Free Workspace	0.00	Cloud-based UML Architecture Modelling.
Katalon Studio	Free Edition	0.00	End-to-End (E2E) Automated System Testing.
Google Chrome	Freeware	0.00	Responsive Web Browser and DevTools Testing.

Google Workspace (Docs / Drive)	Cloud Freeware	0.00	Documentation Cloud Storage and Report Generation.
Total Software Cost		0.00	

1.8.3 Project Timeline

This phase focuses on understanding the problem domain, analysing the existing manual Excel-based processes, and outlining the initial system scope.

Table 1.6 Overall Project Timeline

Task	**Assigned To**	**Duration (days)**	**Start Date**	**End Date**
Phase 1: Requirement Analysis				
Chapter 1: Introduction	Yong Chong Xin, Ting Rong You	**12**	**26.01. 2026**	**06.02. 2026**
Background Study & Identify Problem	Yong Chong Xin, Ting Rong You	2	26.01. 2026	27.01. 2026
Initial System Understanding & Scope Definition	Yong Chong Xin, Ting Rong You	3	28.01. 2026	30.01. 2026
Defining the Hardware and Software Requirements	Yong Chong Xin, Ting Rong You	3	31.01. 2026	02.02. 2026
Defining the Project Management Cost	Yong Chong Xin, Ting Rong You	1	03.02. 2026	03.02. 2026
Preparing for the Timeline	Yong Chong Xin, Ting Rong You	3	04.02. 2026	06.02. 2026
Chapter 2: Literature Review	Yong Chong Xin, Ting Rong You	**14**	**07.02. 2026**	**20.02. 2026**
Domain Analysis	Yong Chong Xin, Ting Rong You	5	07.02. 2026	11.02. 2026
Review of Similar System	Yong Chong Xin, Ting Rong You	5	12.02. 2026	16.02. 2026

Technology Review	Yong Chong Xin, Ting Rong You	4	17.02. 2026	20.02. 2026
Chapter 3: Methodology & Requirements	Yong Chong Xin, Ting Rong You	**17**	**21.02. 2026**	**09.03. 2026**
Select Appropriate Software Development Methodology	Yong Chong Xin, Ting Rong You	3	21.02. 2026	23.02. 2026
Collect Requirement from Stakeholders using different Techniques	Yong Chong Xin, Ting Rong You	4	24.02. 2026	27.02. 2026
Requirement Analysis (Including Use Case Diagram)	Yong Chong Xin, Ting Rong You	7	28.02. 2026	06.03. 2026
Defining Functional and Non-Functional Requirements	Yong Chong Xin, Ting Rong You	3	07.03. 2026	09.03. 2026
Milestone: Finalize Requirements	Yong Chong Xin, Ting Rong You	0	09.03. 2026	09.03. 2026
Phase 2: System Design				
Chapter 4: System Design	Yong Chong Xin, Ting Rong You	**35**	**10.03. 2026**	**13.04. 2026**
Activity Diagram	Yong Chong Xin & Ting Rong You	6	10.03. 2026	15.03. 2026
Data Design (Class Diagram)	Yong Chong Xin & Ting Rong You	3	16.03. 2026	18.03. 2026
UI / UX Prototyping	Yong Chong Xin & Ting Rong You	6	19.03. 2026	24.03. 2026
Security Design	Yong Chong Xin & Ting Rong You	3	28.03. 2026	30.03. 2026
Process Design	Yong Chong Xin	3	31.03. 2026	02.04. 2026

	& Ting Rong You			
Software Architecture Design	Yong Chong Xin & Ting Rong You	3	03.04. 2026	05.04. 2026
Algorithm Design	Yong Chong Xin & Ting Rong You	3	06.04. 2026	08.04. 2026
Final Report Checking & Compilation	Yong Chong Xin & Ting Rong You	4	09.04. 2026	12.04. 2026
Milestone: Finalize System Design & FYP 1 Final Report Submission	Yong Chong Xin & Ting Rong You	0	13.04. 2026	13.04. 2026
Phase 3: Development				
Chapter 5: Implementation & Testing (Implementation)	Yong Chong Xin, Ting Rong You	**60**	**20.04. 2026**	**18.06. 2026**
Environment Setup (XAMPP / GitHub)	Yong Chong Xin, Ting Rong You	1	20.04. 2026	20.04. 2026
Core Development: Professional Profile Module	Yong Chong Xin	4	21.04. 2026	24.04. 2026
Core Development: Student Profile Module	Ting Rong You	4	25.04. 2026	28.04. 2026
Core Development: Management Module	Yong Chong Xin	5	29.04. 2026	03.05. 2026
Core Development: Allocation & Quota Module Module	Yong Chong Xin	5	04.05. 2026	08.05. 2026
Core Development: Availability & Status Module	Ting Rong You	4	09.05. 2026	12.05. 2026
Core Development: Discovery & Search Module	Ting Rong You	6	13.05. 2026	18.05. 2026
Core Development: Request & Proposal Module	Ting Rong You	4	19.05. 2026	22.05. 2026

Core Development: Request Management Module	Yong Chong Xin	6	23.05. 2026	28.05. 2026
Core Development: Timeline & MileStone Module	Ting Rong You	5	29.05. 2026	02.06. 2026
Core Development: Student Review Module	Ting Rong You	3	03.06. 2026	05.06. 2026
Core Development: Administrator Report Module	Yong Chong Xin	4	06.06. 2026	09.06. 2026
Core Development: Supervisor Report Module	Yong Chong Xin	6	10.06. 2026	15.06. 2026
Documentation: Drafting Chapter 5: Implementation	Yong Chong Xin, Ting Rong You	3	16.06. 2026	18.06. 2026
Milestone: Finalize Module Development	Yong Chong Xin, Ting Rong You	0	18.06. 2026	18.06. 2026
Phase 4: Testing				
Chapter 5: Implementation & Testing (Testing)	Yong Chong Xin, Ting Rong You	**29**	**19.06. 2026**	**17.07. 2026**
Requirement Analysis (Testing Context)	Yong Chong Xin, Ting Rong You	1	19.06. 2026	19.06. 2026
Test Planning: Defining Scope, Risk Register, Strategy	Yong Chong Xin, Ting Rong You	3	20.06. 2026	22.06. 2026
Test Case Development: Drafting & Review	Yong Chong Xin, Ting Rong You	7	23.06. 2026	29.06. 2026
Test Environment Setup: Katalon Studio Configuration	Yong Chong Xin, Ting Rong You	1	30.06. 2026	30.06. 2026
Test Execution: Automated Functional & Unit Testing (Katalon)	Yong Chong Xin, Ting Rong You	6	01.07. 2026	06.07. 2026
Test Execution: System Integration Testing (SIT)	Yong Chong Xin, Ting Rong You	2	07.07. 2026	08.07. 2026

Test Execution: Manual User Acceptance Testing (UAT)	Yong Chong Xin, Ting Rong You	1	09.07. 2026	09.07. 2026
System Testing	Yong Chong Xin, Ting Rong You	1	10.07. 2026	10.07. 2026
Test Cycle Closure	Yong Chong Xin, Ting Rong You	1	11.07. 2026	11.07. 2026
Drafting Chapter 5: Testing Results	Yong Chong Xin, Ting Rong You	5	12.07. 2026	16.07. 2026
Milestone: Finalize Testing	Yong Chong Xin, Ting Rong You	0	17.07. 2026	17.07. 2026
Phase 5: Discussion & Conclusion				
Chapter 7: Discussion & Conclusion	Yong Chong Xin & Ting Rong You	**36**	**17.07. 2026**	**21.08. 2026**
Poster & Demo Video Preparation	Yong Chong Xin & Ting Rong You	11	17.07. 2026	27.07. 2026
Drafting Chapter 7: Conclusion	Yong Chong Xin & Ting Rong You	9	28.07. 2026	05.08. 2026
Submission for FYP Report	Yong Chong Xin & Ting Rong You	1	07.08. 2026	07.08. 2026
Final Compilation (System, Report, Poster)	Yong Chong Xin & Ting Rong You	1	16.08. 2026	16.08. 2026
Milestone: Finalize FYP Report, Code & FYP 2 Final Submission	Yong Chong Xin & Ting Rong You	0	21.08. 2026	21.08. 2026

Gantt Chart (Waterfall)

1 Requirement	**43 days**	**Jan 26 '26**	**Mar 9 '26**		
1.1 Chapter 1: Introduction	**12 days**	**Jan 26 '26**	**Feb 6 '26**		
1.1.1 Background Study & Identify Problem	2 days	Jan 26 '26	Jan 27 '26		Ting Rong You,Yong Chong Xin
1.1.2 Initial System Understanding & Scope Definitions	3 days	Jan 28 '26	Jan 30 '26	3	Ting Rong You,Yong Chong Xin
1.1.3 Defining Hardware & Software Requirements	3 days	Jan 31 '26	Feb 2 '26	4	Ting Rong You,Yong Chong Xin
1.1.4 Defining Project Management Cost	1 day	Feb 3 '26	Feb 3 '26	5	Ting Rong You,Yong Chong Xin
1.1.5 Preparing for the Timeline	3 days	Feb 4 '26	Feb 6 '26	6	Ting Rong You,Yong Chong Xin
1.2 Chapter 2: Literature Review	**14 days**	**Feb 7 '26**	**Feb 20 '26**		
1.2.1 Domain Analysis	5 days	Feb 7 '26	Feb 11 '26	7	Ting Rong You,Yong Chong Xin
1.2.2 Review of Similar Systems	5 days	Feb 12 '26	Feb 16 '26	9	Ting Rong You,Yong Chong Xin
1.2.3 Techonology Review	4 days	Feb 17 '26	Feb 20 '26	10	Ting Rong You,Yong Chong Xin
1.3 Chapter 3: Methodology	**17 days**	**Feb 21 '26**	**Mar 9 '26**		
1.3.1 Select Appropriate Software Development Methodology	3 days	Feb 21 '26	Feb 23 '26	11	Ting Rong You,Yong Chong Xin
1.3.2 Collect Requirements from Stakeholders using different techniques	4 days	Feb 24 '26	Feb 27 '26	13	Ting Rong You,Yong Chong Xin
1.3.3 Requirement Analysis (Including Use Case Diagram)	7 days	Feb 28 '26	Mar 6 '26	14	Ting Rong You,Yong Chong Xin
1.3.4 Defining Functional and Non-Functional Requirements	3 days	Mar 7 '26	Mar 9 '26	15	Ting Rong You,Yong Chong Xin
1.3.5 Finalizes Requirements	0 days	Mar 9 '26	Mar 9 '26		

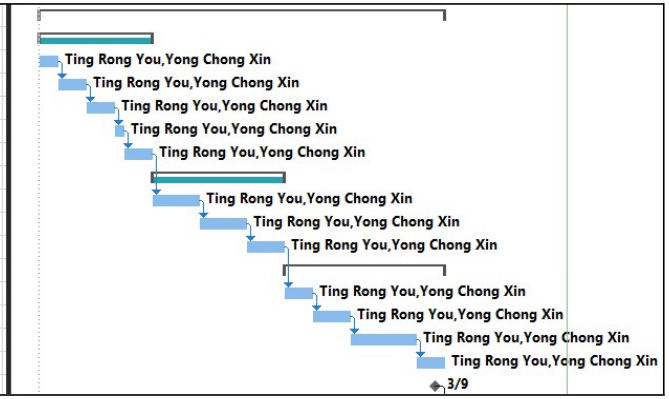

Figure 1.5: Phase 1 Requirement (26.01.2026 - 09.03.2026)

2 Design	**35 days**	**Mar 10 '26**	**Apr 13 '26**		
2.1 Chapter 4 : Design	35 days	Mar 10 '26	Apr 13 '26		
2.2 Activity Diagram	6 days	Mar 10 '26	Mar 15 '26	17	Ting Rong You,Yong Chong Xin
2.3 Data Design	3 days	Mar 16 '26	Mar 18 '26	20	Ting Rong You,Yong Chong Xin
2.4 UI / UX Prototyping	6 days	Mar 19 '26	Mar 24 '26	21	Ting Rong You,Yong Chong Xin
2.5 Security Design	3 days	Mar 28 '26	Mar 30 '26	22	Ting Rong You,Yong Chong Xin
2.6 Process Design	3 days	Mar 31 '26	Apr 2 '26	23	Ting Rong You,Yong Chong Xin
2.7 Software Architecture Design	3 days	Apr 3 '26	Apr 5 '26	24	Ting Rong You,Yong Chong Xin
2.8 Algorithm Design	3 days	Apr 6 '26	Apr 8 '26	25	Ting Rong You,Yong Chong Xin
2.9 Final Report Checking & Compilation	4 days	Apr 9 '26	Apr 12 '26	26	Ting Rong You,Yong Chong Xin
2.10 Finalize System Design & FYP 1 Final Report Submission	0 days	Apr 13 '26	Apr 13 '26		

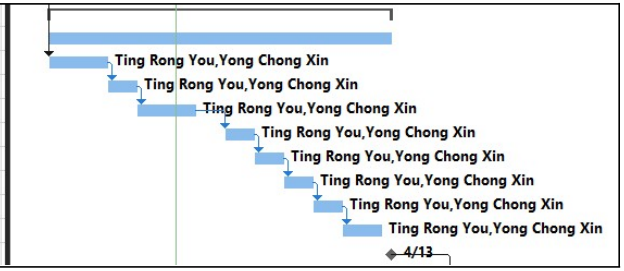

Figure 1.6: Phase 2 Design (10.03.2026 - 13.04.2026)

3 Development	**60 days**	**Apr 20 '26**	**Jun 18 '26**		
3.1 Chapter 5: Implementation & Testing (Implementation)	60 days	Apr 20 '26	Jun 18 '26		
3.2 Environment Setup (XAMPP / GitHub)	1 day	Apr 20 '26	Apr 20 '26	28	Ting Rong You,Yong Chong Xin
3.3 Core Development: Professional Profile Module	4 days	Apr 21 '26	Apr 24 '26	31	Yong Chong Xin
3.4 Core Development: Student Profile Module	4 days	Apr 25 '26	Apr 28 '26	32	Ting Rong You
3.5 Core Development: Management Module	5 days	Apr 29 '26	May 3 '26	33	Yong Chong Xin
3.6 Core Development: Allocation & Quota Module	5 days	May 4 '26	May 8 '26	34	Yong Chong Xin
3.7 Core Development: Availability & Status Module	4 days	May 9 '26	May 12 '26	35	Ting Rong You
3.8 Core Development: Discovery & Search Report Module	6 days	May 13 '26	May 18 '26	36	Ting Rong You
3.9 Core Development: Request & Proposal Module	4 days	May 19 '26	May 22 '26	37	Ting Rong You
3.10 Core Development: Request Management Module	6 days	May 23 '26	May 28 '26	38	Yong Chong Xin
3.11 Core Development: Timeline & Milestone Module	5 days	May 29 '26	Jun 2 '26	39	Ting Rong You
3.12 Core Development: Student Review Module	3 days	Jun 3 '26	Jun 5 '26	40	Ting Rong You
3.13 Core Development: Administrator Report Module	4 days	Jun 6 '26	Jun 9 '26	41	Yong Chong Xin
3.14 Core Development: Supervisor Report Module	6 days	Jun 10 '26	Jun 15 '26	42	Yong Chong Xin
3.15 Documentation: Drafting Chapter 5: Implementation	3 days	Jun 16 '26	Jun 18 '26	43	Ting Rong You,Yong Chong Xin
3.16 Finalize Module Development	0 days	Jun 18 '26	Jun 18 '26		

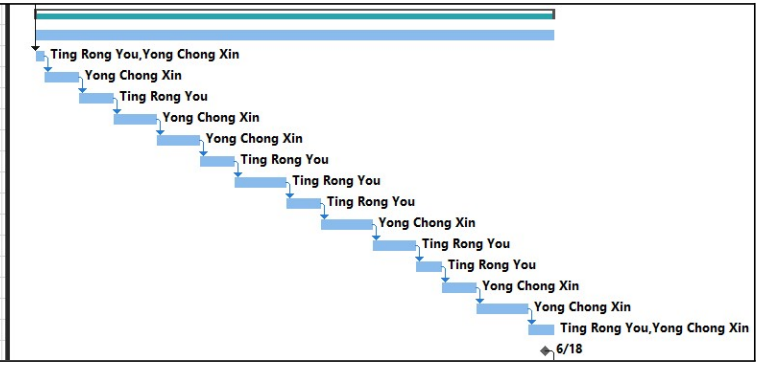

Figure 1.7: Phase 3 Development (20.04.2026 - 18.06.2026)

4 Testing	**29 days**	**Jun 19 '26**	**Jul 17 '26**		
4.1 Chapter 5: Implementation & Testing (Testing)	29 days	Jun 19 '26	Jul 17 '26		
4.2 Requirement Analysis (Testing Context)	1 day	Jun 19 '26	Jun 19 '26	45	Ting Rong You,Yong Chong Xin
4.3 Test Planning: Defining Scope, Risk Register, Strategy	3 days	Jun 20 '26	Jun 22 '26	48	Ting Rong You,Yong Chong Xin
4.4 Test Case Development: Drafting & Review	7 days	Jun 23 '26	Jun 29 '26	49	Ting Rong You,Yong Chong Xin
4.5 Test Environment Setup: Katalon Studio Configuration	1 day	Jun 30 '26	Jun 30 '26	50	Ting Rong You,Yong Chong Xin
4.6 Test Execution: Automated Functional & Unit Testing (Katalon)	6 days	Jul 1 '26	Jul 6 '26	51	Ting Rong You,Yong Chong Xin
4.7 Test Exececution: System Integration Testing (SIT)	2 days	Jul 7 '26	Jul 8 '26	52	Ting Rong You,Yong Chong Xin
4.8 Test Execution: Manual User Acceptance Testing (UAT)	1 day	Jul 9 '26	Jul 9 '26	53	Ting Rong You,Yong Chong Xin
4.9 System Testing	1 day	Jul 10 '26	Jul 10 '26	54	Ting Rong You,Yong Chong Xin
4.10 Test Cycle Closure	1 day	Jul 11 '26	Jul 11 '26	55	Ting Rong You,Yong Chong Xin
4.11 Defining Chapter 5: Testing Results	5 days	Jul 12 '26	Jul 16 '26	56	Ting Rong You,Yong Chong Xin
4.12 Finalize Testing	0 days	Jul 17 '26	Jul 17 '26		

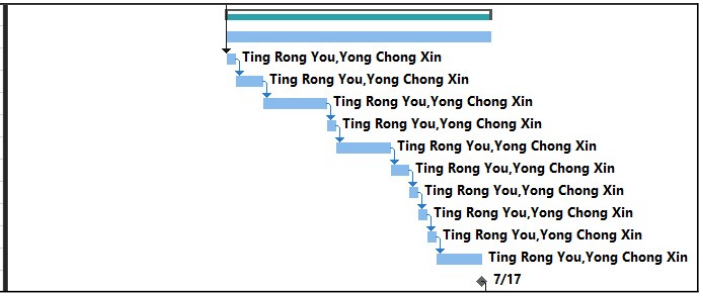

Figure 1.8: Phase 4 Testing (19.06.2026 - 17.07.2026)

5 Discussion & Conclusion	**36 days**	**Jul 17 '26**	**Aug 21 '26**		
5.1 Chapter 7: Discussion & Conclusion	36 days	Jul 17 '26	Aug 21 '26		
5.2 Poster & Demo Video Preparation	11 days	Jul 17 '26	Jul 27 '26	58	Ting Rong You,Yong Chong Xin
5.3 Drafting Chapter 7: Conclusion	9 days	Jul 28 '26	Aug 5 '26	61	Ting Rong You,Yong Chong Xin
5.4 Submission for FYP Report	1 day	Aug 7 '26	Aug 7 '26	62	Ting Rong You,Yong Chong Xin
5.5 Final Compilation (System, Report, Poster)	1 day	Aug 16 '26	Aug 16 '26	63	Ting Rong You,Yong Chong Xin
5.6 Finalize FYP Report, Code & FYP 2 Final Submission	0 days	Aug 21 '26	Aug 21 '26		Ting Rong You,Yong Chong Xin

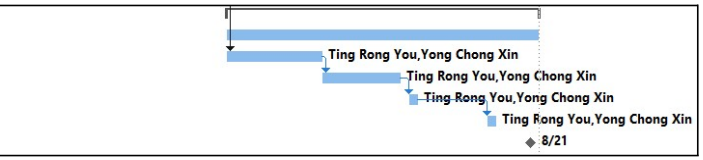

Figure 1.9: Phase 5 Discussion & Conclusion (17.07.2026 - 21.04.2026)

1.8.4 Feasibility Analysis (TELOS)

To ensure the proposed Supervisor Selection and Allocation System (SSAS) can be successfully developed, deployed, and adopted within the academic constraints, a comprehensive feasibility study was conducted utilising the TELOS framework. The summarised assessment is detailed in Table 1.12.

Table 1.7: TELOS Feasibility Assessment for the SSAS Project

TELOS Criterion	**Feasibility Status**	**Justification & Project Alignment**
Technical	Highly Feasible	The architecture relies on proven, industry-standard web technologies (PHP, MySQL, via XAMPP) that are well-supported. The team possesses the requisite technical competencies in database management, backend logic, and automated E2E testing (Katalon Studio) to execute the system scope.
Economic	Highly Feasible	The development incurs negligible financial overhead. By exclusively leveraging open-source frameworks, free-tier testing environments, and personal hardware, the project is highly cost-effective and remains well within a student budget.
Legal / Ethical	Feasible	Because the system handles sensitive user credentials and academic profiles, data governance is prioritised. The strict enforcement of Role-Based Access Control (RBAC) ensures compliance with institutional data privacy standards, guaranteeing only authorised administrators can alter quotas.
Operational	Highly Feasible	The system directly resolves the severe administrative bottlenecks identified in the legacy workflow. By eliminating manual email processing and providing real-time availability tracking, the SSAS vastly improves the user experience, ensuring high user acceptance upon deployment.
Schedule	Feasible	The project is strictly bound by the standard FYP academic calendar. The modular Work Breakdown Structure (Section 1.4.4) enables parallel, concurrent development between the two team members, ensuring all phases from requirements to testing are completed on time.

1.9 Chapter Summary and Evaluation

This chapter has established the foundational framework for the development of the student-centric modules within the TAR UMT Supervisor Selection and Allocation System (SSAS).

The initial sections identified critical inefficiencies in the legacy manual selection process, notably the lack of transparency regarding supervisor availability and the communication chaos caused by unstructured emails. Consequently, Section 1.3 defined a centralised digital platform designed to empower students with real-time visibility, intelligent recommendation matching, and structured proposal submissions. Section 1.6 further elevated this mission by aligning the SSAS with the United Nations Sustainable Development Goals (SDG 4 and 9) and identifying its long-term commercialisation potential as a scalable, white-labelled SaaS solution.

The boundaries of this endeavour were precisely defined in Section 1.4, which focused strictly on the student experience through the Discovery & Search, Supervisor Availability & Status, Request & Proposal, Student Profile, Student Review, and Timeline & Milestone modules. To ensure successful delivery, Section 1.8 provided a realistic development timeline, zero-cost budget strategy, and a balanced workload distribution. The project's viability was rigorously validated through a TELOS feasibility assessment in Section 1.8.4, confirming that the proposed system is technically sound, economically efficient, and operationally necessary for the faculty.

Evaluating the contents of this chapter indicates that the proposed development is both highly necessary and technically grounded. Section 1.5, leveraging XAMPP, PHP, and JavaScript, are perfectly suited to handle the real-time dashboards and matching logic required within the institutional constraints. Section 1.7.2, particularly the reliance on active supervisor participation to populate the recommendation engine, provides realistic operational boundaries to protect the development scope. Ultimately, the transition from a manual, opaque process to this automated, student-driven platform is positioned to significantly enhance the FYP experience by promoting transparency, informed decision-making, and streamlined administrative workflows.

Chapter 2

Literature Review

2 Literature Review

This chapter provides a comprehensive review of the literature and existing technologies relevant to the development of the Supervisor Selection and Allocation System (SSAS). The objective is to establish a critical understanding of both the administrative problem and the current technological landscape. Section 2.1 explores the academic domain, detailing the significance of student-supervisor matching and the inherent flaws of traditional, manual allocation workflows. Following this, Section 2.2 critically evaluates the existing software solutions, ranging from generic Learning Management Systems to academic networking platforms, to identify functional gaps in the current market. Section 2.3 then examines the underlying web technologies, database architectures, and recommendation algorithms required to address these gaps. Finally, Section 2.4 summarises the findings, ultimately justifying the necessity and technical approach of the proposed SSAS.

2.1 Domain Analysis

The domain of academic resource allocation has undergone a significant paradigm shift, transitioning from decentralised, opaque workflows to centralised, transparent digital ecosystems. This section examines the operational landscape of supervisor allocation at TAR UMT, specifically from the perspective of the student experience and psychological well-being. It begins by establishing the pedagogical significance of the student-supervisor match before mapping the “as-is” manual workflow currently in use. By identifying the core inefficiencies of legacy tools, this analysis introduces the conceptual requirements for modern automated frameworks and concludes with a structural comparison between the conventional and automated modes.

2.1.1 The Significance of the Student-Supervisor Match

The Final Year Project (FYP) represents a critical milestone in undergraduate education, serving as the capstone of a student's academic journey. The success of this endeavour relies heavily on the compatibility between a student's research interests and a supervisor's specific domain expertise and mentoring approach. Recent empirical research highlights that thesis supervision is not merely an administrative formality, but a core pedagogical function. According to Mårtensson and Söderström (2025), the quality and style of supervision a student receives directly impacts four key academic outcomes: thesis quality, timely completion, deep learning, and scientific curiosity. Their findings demonstrate that a poorly managed allocation process, where students are mismatched or lack proper supervisory support, can lead to sub-optimal research outputs and increase frustration for both the student and the faculty member.

Beyond academic performance, the allocation process significantly impacts the student's psychological well-being during the capstone semester. The FYP is widely recognised as one of the most high-pressure components of an undergraduate degree, often acting as a primary catalyst for academic stress (Prasetyaningrum et al., 2026). When the supervisor matching process is opaque, highly competitive, or reliant on unstructured communication, it increases student anxiety and cognitive load before the research has begun. A structured, transparent matching process not only pairs students with the correct domain experts but also provides a crucial sense of institutional support. By alleviating the administrative friction associated with securing a supervisor, a transparent system enables students to focus their cognitive resources entirely on their research objectives, ultimately elevating the quality of the institution's resulting research output.

2.1.2 The Manual Approach (Spreadsheets and Email)

Historically, faculties have relied on decentralised tools to manage the matching of students to supervisors. This workflow typically utilises static Microsoft Excel lists for supervisor directories and external email clients for negotiation. While functional for basic communication, this disjointed approach introduces severe usability and transparency bottlenecks for students.

- **Information Asymmetry and Opaque Availability**

  In the current workflow, students are provided with a static list of supervisors. Because this spreadsheet is not dynamically linked to the supervisors' actual acceptance rates, students suffer from information asymmetry. They frequently waste time emailing supervisors who have already reached their maximum quota but have not yet manually updated a centralised board. This lack of real-time status visibility leads to a highly inefficient trial-and-error application process (Panko, 2016).

- **High Communication Latency and Status Anxiety**

  The manual reliance on email for proposal submissions creates a fragmented communication loop. Students have no centralised way to track the lifecycle of their application. They submit a proposal and enter a "waiting state", unsure if the email was read, pending, or ignored. This lack of structured status tracking results in high administrative latency and unnecessary follow-up communications (Stair & Reynolds, 2018).

- **Suboptimal Matching and Lack of Peer Insight**

  Under the manual system, students select supervisors based on brief information in the TAR UMT Staff Directory. There is no intelligent matching mechanism to align a student's specific research interests (e.g., Machine Learning) with a supervisor's proven expertise, an omission that research shows often leads to subjective and suboptimal academic pairings (Rismanto et al., 2020). Furthermore, the absence of a structured review or feedback repository prevents students from making informed decisions based on the experiences of previous cohorts. Modern academic decision-making heavily relies on peer feedback and student reviews to evaluate supervisory quality, and without this transparency, students are at a significant disadvantage (Zhang & Liu, 2023).

2.1.3 The Automated Approach

In contrast to the manual workflow, contemporary academic institutions are increasingly adopting web-based allocation models that utilise a centralised architecture and dynamic matching logic to address structural flaws and empower the student.

- **Real-Time Transparency and Status Badging**

  A core concept of modern allocation systems is the elimination of information asymmetry. By pulling live data from a centralised database, these platforms can display dynamic status badges (e.g., Green / Red) and exact quota numbers. This real-time visibility prevents students from applying to fully-booked staff, significantly streamlining the discovery process (Silberschatz et al., 2020).

- **Intelligent Discovery and Recommendation**

  Moving beyond static directories, automated models introduce multi-criteria filtering and logic-based recommendation algorithms. By allowing students to input specific research "interest tags", the system programmatically suggests highly compatible supervisors. This algorithmic approach vastly improves the quality of the academic match compared to manual, trial-and-error selection (Maulani et al., 2025).

- **Centralised Request Workflow and Peer Feedback**

  Finally, modern conceptual models replace disjointed emails with centralised request workflows, providing students with a unified dashboard to track their application status (e.g., pending, accepted, rejected) in real-time (Laudon & Laudon, 2020). Additionally, integrating anonymised peer-feedback loops allows students to access structured ratings of previous supervisory experiences, thereby fostering informed, data-driven decision-making regarding academic quality (Zhang & Liu, 2023).

2.1.4 Comparison Summary of Allocation Models

Table 2.1 summarises the transition from the legacy manual workflow to a modern automated architecture, highlighting the structural shifts required to resolve the identified bottlenecks in the student-supervisor matching process.

Table 2.1 Detailed Comparison of Manual vs Automated Allocation Frameworks

Feature	**Manual Method (Excel / Email)**	**Automated Web System (SSAS)**	**Primary Academic / Technical Impact**
Data Integrity and Updates	**Passive:** Relies on manual supervisor updates, prone to "Synchronisation Gaps" and "over-booking".	**Active:** Real-Time database triggers and quota enforcement physically block over-limit entries.	Prevents "over-booking" and human error (Panko, 2016).
Availability Visibility	**Opaque:** Students rely on static lists, actual availability is unknown until receiving a direct response from supervisors.	**Transparent:** Live dashboards display real-time status badges (Green / Red) and exact remaining seats.	Eliminates "Information Asymmetry" (Silberschatz et al., 2020).
Communication Loop	**Fragmented:** Standalone emails with PDF attachments, no centralised audit trail.	**Centralised:** Integrated request portal with automated status tracking (Pending / Accepted / Rejected).	Reduces "Communication Latency" and student anxiety (Stair & Reynolds, 2018).
Supervisor Discovery	**Manual Search:** Scrolling through static staff lists, matching based on "trial-and-error".	**Algorithmic:** Multi-criteria filtering and "Interest-Tag" recommendation logic.	Optimises topic alignment and research quality (Maulani et al., 2025).
Decision Support	**Isolated:** Students choose based on brief bios or unverified word-of-mouth reputation.	**Informed:** Access to structured, anonymised peer-review repositories and supervisor ratings.	Enables "Data-Driven Decision Making" (Zhang & Liu, 2023).
Administrative Oversight	**Blind Spot:** Administrator cannot see the progress of the cohort until manual sheets are submitted.	**Omniscient:** Real-time administrative dashboards show live fill-rates and unassigned student counts.	Enhances managerial visibility and reporting (Laudon & Laudon, 2020).

To provide a visual synthesis of the operational differences, figure 2.1 (Odia & Okolie, 2024) illustrates the modernised, role-based automated workflow, which stands in direct contrast to the sequential manual process previously detailed in Chapter 1 (refer to figure 1.2).

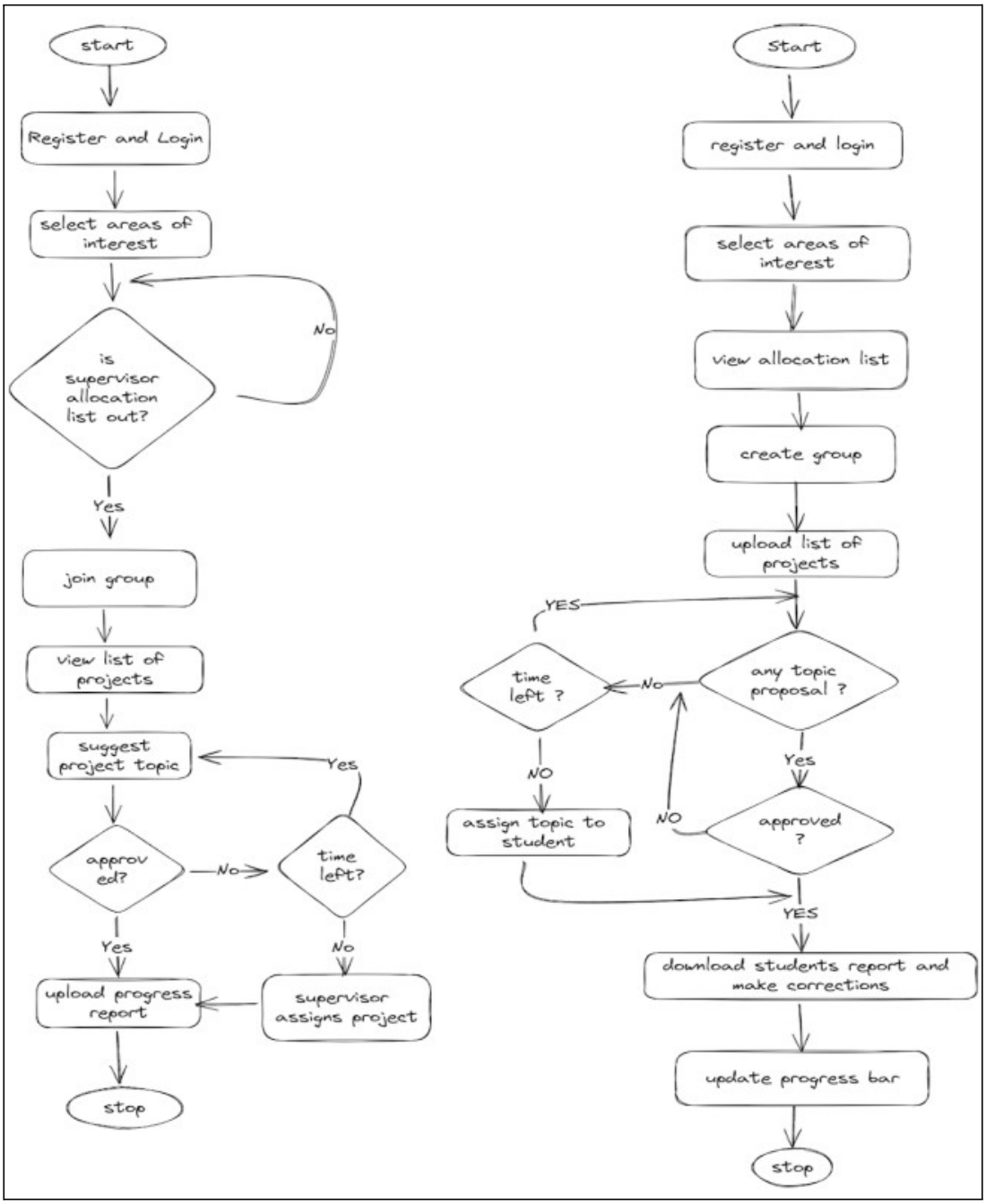

Figure 2.1: Existing Digitalised Supervisor Selection and Allocation System Research by others institutions (Source: Odia & Okolie, 2024)

Analysis of Workflow Distinctions

From Figure 2.1, the legacy manual workflow operates as a fragmented, isolated progression that is heavily reliant on external email communication. This decentralised model suffers from high administrative latency, lacking immediate feedback loops or systemic visibility until the final allocation list is manually compiled by the administrator.

In contrast, Figure 2.2 illustrates the digital SSAS workflow, which replaces this linear bottleneck by splitting the process into specialised, concurrent operational flows (Odia & Okolie, 2024):

- **Student Flow:** Introduces automated wait-states (checks if allocation lists are officially released) and students are allowed to resubmit other proposal topics if their proposals are rejected by supervisors.
- **Supervisor:** Shifts the focus toward management and oversight. The system automates the transition from topic approval to progress monitoring. By replacing manual spreadsheet updates with real-time status indicators and automated report generation, the administrative burden on the faculty is significantly reduced.

By adopting Role-Based Automation, the system ensures that the “Final List” is not a static document compiled at the end of a semester, but a dynamic, living dataset that remains continuously accurate throughout the entire allocation lifecycle.

Conclusion

While the architectural advantages of a centralised, automated approach are evident, it is necessary to evaluate whether existing third-party platforms, such as institutional web directories or standard Learning Management Systems (LMS), can actually fulfil the specific discovery and matching requirements of the student body. As noted by Valacich and Schneider (2018), effective digital information systems must prioritise active decision-support and dynamic user experiences, rather than merely serving as static repositories for administrative records. Therefore, the following section critically reviews these existing platforms to determine if they possess the intelligent filtering and real-time transparency necessary to empower student choice, or if a bespoke software solution is required to facilitate optimal academic pairings.

2.1.5 Summary of Findings

The transition from a manual, file-based workflow to an automated digital ecosystem is not merely a technical upgrade, but a necessary shift to preserve the quality of the undergraduate research experience. As illustrated by comparison in Table 2.1, the legacy reliance on "information silos" like email and static spreadsheets creates an environment of transparency gaps and high administrative latency. These structural flaws directly contribute to student anxiety and suboptimal supervisor pairings, which can compromise the academic integrity of the Final Year Project. While the conceptual benefits of an automated system are clear, it is necessary to evaluate whether existing commercial platforms currently fulfil these specialised academic requirements. Consequently, the following section provides a critical review of prominent Learning Management Systems (LMS) and academic networking tools to identify the remaining research gaps that the proposed solution aims to bridge.

2.2 Review of Similar Existing Systems

This section examines existing technological solutions and platforms currently utilised in academic environments to manage discovery, applications, and student-instructor interactions. Rather than evaluating these systems in isolation, they are categorised by their architectural purpose to establish a baseline for comparison.

2.2.1 Institutional Staff Directories (Static Web Portals)

Traditionally, the baseline method for student-supervisor discovery involves navigating the university's standard staff directory web pages. Architecturally, these portals function as legacy Content Management Systems (CMS) built upon "Web 1.0" design principles, acting as read-only digital networks (Ibrahim, 2021). They typically contain faculty names, departmental affiliations, official email addresses, and brief, flat-text biographical summaries.

The primary utility of these institutional directories lies in their data authority. This is because they are strictly maintained from the top-down by institutional IT or Human Resources departments, they enforce strict authority control, a mechanism used in digital repositories to maintain a single, standardised, and verified bibliographic record for each faculty member, thereby ensuring data consistency and integrity across the university (Almuzara et al., 2012). Common examples include the standard TAR UMT Staff directories.

However, since these systems are fundamentally designed as identity repositories rather than matchmaking tools, their static architecture presents significant operational limitations during an active allocation period. They lack algorithm filtering capabilities, forcing students to manually parse dozens of unstructured profiles. This manual search process not only induces cognitive and information overload (Bawden & Robinson, 2020), but frequently results in suboptimal academic pairings, as students are unable to efficiently align their specific research topics with a supervisor's specialised expertise (Rismanto et al., 2020). Consequently, while they excel at official verification, their lack of real-time capacity tracking and dynamic interaction renders them insufficient as standalone allocation frameworks.

2.2.2 Learning Management Systems (LMS)

Most higher education institutions, including TAR UMT, deploy enterprise-grade Learning Management Systems (LMS) such as Canvas, Moodle, or Google Classroom to manage academic administration. Architecturally, these platforms are designed to support linear pedagogical workflows, with core functionalities focused on course delivery, secure document submission, and rubric-based grading. (Al-Busaidi & Al-Shihi, 2012).

The primary strength of an LMS is its ubiquitous adoption and familiarity among students, providing a secure, centralised ecosystem for managing established academic tasks, such as uploading final FYP reports or broadcasting faculty announcements (Turnbull et al., 2021).

However, an LMS is fundamentally engineered for post-enrolment assignment evaluation, not pre-enrolment discovery or application pipelines. While a student can successfully upload a PDF proposal to an LMS "Assignment" portal, the system's architecture does not support multi-state transactional lifecycles (e.g., Pending, Accepted, Rejected). As highlighted by Oluwayimika (2022) in their analysis of LMS drawbacks, these systems often lack the flexibility required for dynamic, multi-party interactions outside of standard curriculum delivery. Consequently, utilising an LMS for supervisor allocation forces faculty to "shoehorn" a complex, multi-party negotiation process into a rigid, one-way homework submission tool, ultimately resulting in fragmented application tracking and diminished urgency among students.

2.2.3 Social Media and Informal Information Networks

In the absence of a centralised, intelligent discovery platform, students frequently resort to informal, decentralised communication channels, such as senior-junior WhatsApp groups, Telegram, or lifestyle sharing applications like XiaoHongShu (RedNote), to gather intelligence on supervisor reputations and teaching styles. As established by Shafiq and Jan (2017) in their case study of Malaysian private universities, "word-of-mouth" and peer referrals consistently rank among the most critical factors influencing an undergraduate's choice of research supervisor. Students actively cultivate these digital networks to navigate, critique, and circumvent opaque university administrative processes.

Table 4. Combined ranking of individual factors

		Individual factor rankings											
		1st	2nd	3rd	4th	5th	6th	7th	8th	9th	10th	11th	12th
Previous Encounter	#	18	4	4	8	6	14	12	2	4	2	0	0
	%	24.3	5.4	5.4	10.8	8.1	18.9	16.2	2.7	5.4	2.7	0.0	0.0
Education	#	16	16	6	2	12	2	4	4	4	0	4	4
	%	21.6	21.6	8.1	2.7	16.2	2.7	5.4	5.4	5.4	0.0	5.4	5.4
Projects	#	6	14	12	4	0	10	8	8	2	4	4	2
	%	8.1	18.9	16.2	5.4	0.0	13.5	10.8	10.8	2.7	5.4	5.4	2.7
Experience	#	8	12	12	10	12	4	8	2	2	0	4	0
	%	10.8	16.2	16.2	13.5	16.2	5.4	10.8	2.7	2.7	0.0	5.4	0.0
Referral	#	0	0	10	10	12	8	6	10	8	6	2	2
	%	0.0	0.0	13.5	13.5	16.2	10.8	8.1	13.5	10.8	8.1	2.7	2.7
Word of Mouth	#	8	4	6	6	14	10	6	0	0	4	6	10
	%	10.8	5.4	8.1	8.1	18.9	13.5	8.1	0.0	0.0	5.4	8.1	13.5
Research Methodology	#	4	8	6	4	2	10	12	8	4	2	10	4
	%	5.4	10.8	8.1	5.4	2.7	13.5	16.2	10.8	5.4	2.7	13.5	5.4
Profile Picture	#	4	0	2	4	4	4	4	8	14	4	14	12
	%	5.4	0.0	2.7	5.4	5.4	5.4	5.4	10.8	18.9	5.4	18.9	16.2
Age	#	4	10	0	6	4	6	2	10	10	12	4	6
	%	5.4	13.5	0.0	8.1	5.4	8.1	2.7	13.5	13.5	16.2	5.4	8.1
Gender	#	0	2	6	8	6	2	2	2	12	16	14	4
	%	0.0	2.7	8.1	10.8	8.1	2.7	2.7	2.7	16.2	21.6	18.9	5.4
Nationality	#	2	0	6	8	2	2	4	12	10	14	6	8
	%	2.7	0.0	8.1	10.8	2.7	2.7	5.4	16.2	13.5	18.9	8.1	10.8
Religion	#	4	4	4	4	0	2	6	8	4	10	6	22
	%	5.4	5.4	5.4	5.4	0.0	2.7	8.1	10.8	5.4	13.5	8.1	29.7

Figure 2.2: Combined ranking of individual factors influencing supervisor selection (Source: Shafiq & Jan, 2017)

Figure 2.2 Shows that "Experience", "Referral" and "Word-of-Mouth" have the highest vote for the 5th place, while "Word-of-Mouth" gained the most votes among the three.

The primary utility of these informal networks lies in their capacity to deliver rapid, unfiltered peer-to-peer feedback. They provide an accessible space for students to share practical, experiential knowledge regarding a supervisor's specific expectations, strictness,

and communication habits, valuable qualitative data that is entirely absent from official institutional directories.

However, from a software and information system perspective, these networks suffer from critical architectural flaws, the data is entirely unstructured, subjective, and completely decoupled from the university's official backend databases. This is because these external platforms lack internal data validation and user authentication mechanisms (Sommerville, 2016), the information shared is highly susceptible to rapid data decay. For example, a supervisor highly recommended as "available" on a social media thread may have mathematically reached their quota days prior. Additionally, feedback on these channels lacks standardised metrics, and there is no systemic way to verify if an anonymous reviewer was genuinely supervised by the faculty member in question. Consequently, relying on these informal networks frequently results in students acting on outdated or unverified information, leading to wasted applications and systemic allocation inefficiencies.

Figure 2.3: Anonymised example of unstructured supervisor feedback and availability sharing on a social media platform (RedNote)

Transcript (Translated from Chinese):

Comment A: Both PXXX and TXXX are nice.

Comment B: What do you guys think about lXXX?

Comment C: Mr KXXX.

Comment D: MaXXX, I was supervised under her.

Author Reply on Comment D: She's full.

Comment E: Unfortunately, mXXX wasn't in the list.

Comment F: I was supervised under ms bXXX, people said that she's strict, but I think she's good, she'll tell you her marking rubric, just that you need to always meet and show…..

Analysis of Informal Communication (Figure 2.3):

As demonstrated in the translated transcript above, relying on informal social networks creates severe information bottlenecks. Critical logistical data, such as the notification in Comment D that a supervisor has already reached maximum capacity ("She's full"), is buried deep within the nested, unofficial comment threads. Because this information is decoupled from the faculty's official database, other students who do not see this specific reply may continue to send redundant applications to an unavailable lecturer. This real-world example clearly illustrates the problem of unstructured data decay and underscores the absolute necessity for an automated system (SSAS), which will replace these chaotic backchannels with a centralised, real-time availability dashboard.

2.2.4 Comparative Analysis of Existing Frameworks

As established in the preceding sections, the current landscape of supervisor discovery and allocation relies on a fragmented ecosystem of disparate tools. While institutional directories provide data authority, Learning Management Systems (LMS) offer secure document handling, and informal networks supply experiential peer feedback, no single platform possesses the end-to-end architectural capability to manage the complete allocation lifecycle.

Table 2.2 synthesises the technical strengths and architectural limitations of these existing frameworks and contrasts them directly with an ideal Automated SSAS Framework.

Table 2.2: Comprehensive Architectural Analysis of Existing Frameworks vs Ideal Automated Benchmark

Evaluation Criteria	**Institutional Directories (e.g., TAR UMT Staff Portal)**	**Learning Management System (e.g., Canvas, Moodle)**	**Informal Networks (e.g., RedNote, WhatsApp)**	**Ideal Automated SSAS Framework (Theoretical Benchmark)**
Core Architecture	Static Content Management System (Web 1.0).	Centralised pedagogical and assessment delivery platform.	Decentralised, third-party social communication networks.	Dynamic, transactional-capable matchmaking and allocation pipeline.
Data Authority & Integrity	**High:** Centralised top-down authority control.	**High:** Verified student and faculty enrolment.	**Low:** Unauthenticated user-generated content (UGC)	**High:** Integrated with official faculty databases and verified user roles.
Discovery & Filtering	**None:** Flat-text search leading to high cognitive search costs.	**None:** Rigidly tied to predetermined course enrolments.	**Manual:** Dependent on community replies and "Word-of-Mouth".	**Advanced:** Algorithmic multi-criteria filtering utilising research tags.
Real-Time Capacity Tracking	**None:** Read-only displays without inventory tracking.	**None:** Not designed for active supervisor quota management.	**Low / Decaying:** Susceptible to outdated rumours.	**Automated:** Real-Time availability dashboard with active quota enforcement.

Application Lifecycle	**None:** Purely an information identity display.	**Fragmented:** One-way document uploads lacking state tracking.	**None:** External to the formal application process.	**Comprehensive:** Multi-state transactional tracking (Pending, Accepted).
Review & Feedback Mechanism	**None:** Official, sanitised biographical data only.	**None:** Unidirectional (focuses on grading students).	**High but Unstructured:** Subjective, unverified, and prone to decay.	**Structured and Verified:** Anonymous, quantitative rating system validated by administrators.;
Role-Based Access Control (RBAC)	**Low:** Global read-only access for all public visitors.	**Medium:** Strict teacher / Student / Admin silos suited for classrooms.	**None:** Flat hierarchy, anyone can claim any identity.	**High:** Granular permissions for Students, Supervisors, and Admins.

Synthesis of Findings and Gap Analysis

The comparative matrix in Table 2.2 clearly illustrates the current operational gap within standard university infrastructure. Students and faculty are currently forced to bridge the divide between official, static data (Directories) and dynamic, unverified data (Social Media) by manually executing workflows through an incompatible pedagogical tool (LMS).

When evaluated against the requirements of an ideal, automated allocation framework, one that combines strict data authority with dynamic matchmaking algorithms and real-time state tracking, it is evident that existing third-party platforms are architecturally insufficient. This reliance on fragmented, unspecialised tools creates severe data bottlenecks, cognitive overload, and inefficient pairing processes. Consequently, to resolve these systemic inefficiencies, there is a clear, justified requirement for the bespoke development of a dedicated Supervisor-Student Allocation System (SSAS) tailored specifically to the faculty's operational logic.

2.3 Technology Review

To architect an optimal Supervisor Selection and Allocation System (SSAS), a rigorous evaluation of potential deployment paradigms is required. The system must successfully serve two distinct user demographics with conflicting technical needs: a massive, highly distributed student body requiring frictionless discovery tools, and a centralised faculty requiring robust data management interfaces.

Consequently, this technology review is divided into two critical architectural pillars. First, a platform deployment analysis assesses Desktop, Native Mobile, and Web-Based Applications to ensure theoretically ubiquitous, cross-device accessibility. Secondly, a backend database technology analysis evaluates Flat-File Storage against Relational Database Management Systems (RDBMS) to guarantee data integrity and complex relational mapping. Each technology's viability is critically examined in this section through the lens of the project's strict operational constraints, which is a zero-budget development environment, a rapid prototyping and delivery schedule, and the absolute necessity for reliable data synchronisation across the TAR UMT campus.

2.3.1 Platform Architectural Analysis

1. **Desktop Application Architecture**

   A desktop application utilises a “thick-client” architecture, meaning the software is compiled to execute natively on the user's local operating system (OS) rather than over a network browser (Sommerville, 2016).

   - **Architectural Strengths (Performance)**

     The primary advantage of a desktop architecture is computational efficiency. Because the application is installed locally, it has direct access to the device’s underlying hardware resources (CPU, RAM, and local storage). As noted by Stair and Reynolds (2020), this makes desktop applications highly suitable for computation-intensive enterprise environments. For faculty administrators generating heavy, multi-threaded allocation reports, a native desktop interface provides unparalleled stability.

- **Architectural Limitations (The Student Deployment Barrier)**
  Despite its performance benefits for administrators, deploying a desktop-first architecture introduces fatal logistical barriers when scaling to the general student population.
  - **High-Friction Onboarding:** The primary goal of the student discovery phase is to provide frictionless matchmaking. Forcing the entire eligible student cohort to manually download, install, and configure a dedicated software package simply to browse supervisor availability once a semester introduces unnecessary user friction and violates modern usability standards (Laudon & Laudon, 2020).
  - **Heterogeneous Hardware Constraints:** The TAR UMT student body utilises a highly diverse mix of operating systems (Windows, macOS, and Linux). Developing a native desktop application to serve all students would require maintaining entirely separate codebases (e.g., C# for Windows, Swift for macOS) or utilising heavy cross-platform frameworks. This approach exponentially increases the development timeline and directly violates the project's rapid prototyping constraints (Sommerville, 2016).
  - **The Version Fragmentation Crisis:** In a distributed desktop environment, updates cannot be pushed unilaterally. If a critical logical bug is identified during the time-sensitive FYP selection week, developers cannot instantly patch it. Instead, they must distribute a new executable file and rely on thousands of individual students to manually reinstall the software. Relying on client-side manual updates inevitably leads to severe "version fragmentation", causing database synchronisation conflicts when students on outdated versions attempt to claim a supervisor's limited quota simultaneously.

2. Native Mobile Application Architecture

A native mobile application is engineered specifically for a single mobile operating system (OS) using platform-specific Software Development Kits (SDKs), such as Swift for Apple's iOS or Kotlin / Java for Google's Android.

- **Architectural Strengths (Accessibility and Engagement)**

  From a purely user-centric perspective, native mobile applications excel at accessibility. As established by Laudon and Laudon (2020) in their analysis of mobile digital platforms, native applications provide highly optimised, fluid user interfaces tailored for small screens and allow for deep integration with device hardware, such as push notifications to instantly alert a student when a supervisor accepts their proposal. Theoretically, this highly accessible architecture aligns well with the on-the-go nature of the student discovery phase.

- **Architectural Limitations (Engineering Complexity and Deployment Bottlenecks)**

  Despite the apparent UX benefits for the student body, adopting a native mobile architecture introduces insurmountable engineering and logistical barriers for a university-level allocation system operating under strict constraints.

  - **Prohibitive Development Complexity:** To successfully serve the entire TAR UMT student cohort, the system must support both Android and iOS devices. A strictly native approach mandates the development and concurrent maintenance of two entirely separate codebases (e.g., Swift and Kotlin), which exponentially inflates the engineering workload and directly violates the project's rapid delivery schedule (Sommerville, 2016).

    Furthermore, even if the development team utilises modern cross-platform frameworks (such as Flutter or React Native) to unify the codebase, the deployment pipeline remains severely bottlenecked by proprietary hardware dependencies. For instance, compiling, testing, and publishing the iOS version of a cross-platform application still strictly requires the use of Apple hardware (macOS) and an active Apple Developer subscription. This introduces unacceptable development friction and directly violates the project's zero-budget operational mandate (Laudon & Laudon, 2020).

  - **App Store Bottlenecks and Financial Constraints:** Unlike web or desktop deployments, native mobile applications cannot be instantly distributed to users. They must pass through stringent, third-party review protocols (such as the Apple App Store Review Guidelines), which can take weeks and frequently result in arbitrary rejections. Furthermore, hosting applications on these official storefronts requires annual developer licensing fees, strictly violating the project's zero-budget mandate (Laudon & Laudon, 2020).
  - **Suboptimal Information Density:** While mobile apps are excellent for simple tasks, the supervisor selection process requires students to critically evaluate complex academic portfolios, compare research tags, and parse lengthy project descriptions. Forcing this high volume of unstructured text onto a constrained 6-inch mobile

interface risks inducing the same “cognitive overload” seen in the legacy flat-text directories, significantly impairing a student’s ability to make an optimal academic pairing (Bawden & Robinson, 2020).

3. Web-Based Application Architecture

A web-based application operates on a client-server architecture, where the core computational logic and database management reside on a centralised server, while the user interface is rendered dynamically through a standard web browser acting as a "thin client".

- **Architectural Strengths (Frictionless Accessibility and Centralised Maintenance)**

  Aligning perfectly with both the project constraints and the operational needs of the student body, a web-based architecture provides several insurmountable advantages over native deployment:

  - **Platform-Agnostic, Zero-Friction Onboarding:** Web applications completely eliminate the installation barrier. A student can access the SSAS instantly from a mobile browser, while simultaneously, a faculty administrator can access the same system from an office desktop. This ubiquitous, cross-device compatibility guarantees that the system serves the entire heterogeneous TAR UMT demographic without requiring a single software download (Laudon & Laudon, 2020).
  - **Centralised Version Control and Rapid Deployment:** Since that the application logic executes server-side, developers must maintain a single, unified codebase. If a critical algorithm needs adjustment or a logical bug is discovered during the time-sensitive allocation period, developers can deploy a patch directly to the server. Instantly, all users interacting with the system receive the updated version upon their next page refresh, completely eradicating the "version fragmentation" and database synchronisation risks associated with desktop deployments (Sommerville, 2016).
  - **Cost-Effective Infrastructure:** Unlike native mobile development, deploying a web application utilising an open-source stack completely bypasses proprietary hardware dependencies and App Store licensing fees, perfectly satisfying the project's zero-budget mandate (Laudon & Laudon, 2020).

- **Architectural Limitations (Network Dependency and Server Reliance)**

  Despite its significant deployment advantages, a web-based architecture introduces specific operational vulnerabilities that native desktop applications do not face.

  - **Strict Network Dependency:** Because the application functions as a "thin-client", it possesses zero offline capabilities. Users must maintain a continuous, stable internet connection to browse supervisor profiles or submit requests. If the campus network experiences an outage, the entire SSAS becomes temporarily inaccessible (Sommerville, 2016).
  - **Centralised Point of Failure:** While a centralised server allows effortless software updates, it also creates a single point of failure. If the primary hosting server crashes due to traffic spike during the opening hours of the allocation week, no student or faculty member can access the system (Laudon & Laudon, 2020).

- **Mitigating the Limitations**

  While these vulnerabilities are inherent to client-server models, they are highly acceptable trade-offs within the context of the TAR UMT infrastructure. Modern university campuses provide ubiquitous, high-speed Wi-Fi, rendering the lack of offline capabilities a non-issue. Additionally, due to strictly limited supervisor quotas, the system requires real-time, online database locking to prevent two students from claiming the same supervisor simultaneously. Therefore, an offline mode would be logically incompatible with the system's core requirements anyway.

- **Task-Technology Fit for Complex Academic Selection**

  While general market trends often highlight mobile application usage for passive notifications, academic research into Information Systems heavily emphasises the "Task-Technology Fit" (TTF) model for university students. Recent empirical studies by Alyoussef (2021, 2023) regarding e-learning acceptance demonstrate that adoption is dictated by how well the chosen technology "fits" the specific characteristics and complexity of the academic task.

The supervisor selection phase is an active, highly complex cognitive task requiring students to evaluate lengthy research proposals, compare multiple academic portfolios, and make highly consequential decisions. Since that constrained limited size of mobile interfaces struggle to support this volume of unstructured text without inducing cognitive overload (Bawden & Robinson, 2020), a mobile application provides a mathematically poor Task-Technology Fit. Conversely, a web-based architecture provides the necessary visual canvas and high information density required to support this complex comparative analysis, ensuring an optimal TTF and maximising student adoption rates.

Platform Conclusions

When evaluated against the strict operational parameters of a Final Year Project, specifically the rapid delivery schedule, zero-budget mandate, and the necessity for instant, university-wide accessibility, web-based architecture emerges not merely as an option, but as the only logistically and psychologically viable platform for the Supervisor Selection and Allocation System (SSAS).

2.3.2 Database Technology Analysis

For the underlying data architecture of the Supervisor Selection and Allocation System (SSAS), the data persistence layer must be evaluated on its ability to support the complex, high-concurrency demands of the student body during the time-sensitive allocation week. The two primary architectures considered for this implementation are Flat-File Storage and Relational Database Management Systems (RDBMS).

1. **Flat-File Storage Architecture**

   A flat-file database represents the most fundamental paradigm of data persistence. In this architecture, data is stored locally on the server's hard drive as standard, unformatted text documents. Unlike advanced database systems, flat-files possess no internal structural hierarchies, inter-table relationships, or built-in query engines. (Connolly & Begg, 2015).

   - **Data Representation and Formats:** All records within a flat-file are stored sequentially, usually separated by a predefined structural delimiter. Industry-standard formats include Comma-Separated Values (CSV), Tab-Separated Values (TSV), or lightweight data-interchange formats like JavaScript Object Notation (JSON). In the context of the SSAS, a flat-file approach would necessitate storing the entire faculty roster in a single text document, where each sequential line represents a unique supervisor's dataset.
   - **Operational Mechanism (File I/O):** Since that flat-file lacks a dedicated, background Database Management System (DBMS) to process requests, data retrieval and manipulation rely entirely on the hosting server's standard file Input / Output (I/O) operations. To interact with the data, such as a student querying a supervisor's remaining quota, the application must physically open the text file, load the contents into the server's active memory (RAM), and sequentially parse the text line-by-line until the target record is found (Connolly & Begg, 2015).
   - **Data Integrity and Concurrency Control:** Flat-file systems rely on standard operating system (OS) level file locking. As noted by Conolly and Begg (2015), when an application opens a file to write or update data, the OS places a lock on the entire document, strictly preventing any other user or process from writing to that file until the initial transaction is completely closed and saved.

2. Relational Database Management System (RDBMS)

In direct contrast to file-based systems, a Relational Database Management System (RDBMS) physically separates the data definition from the application programs. Data is managed by a centralised, complex background software engine (the DBMS) and stored in a rigorously structured format (Connolly & Begg, 2015).

- **Data Representation (The Relational Method):** Instead of a single sequential text file, an RDBMS normalises data into distinct, interconnected mathematical tables (relations). In the context of the SSAS, supervisors, students, and research tags would each occupy their own independent tables. These tables are permanently linked utilising Primary Keys (unique identifiers) and Foreign Keys, which eliminates data redundancy and guarantees structural integrity.
- **Operational Mechanism (SQL and Indexing):** To interact with the database, the system does not read files line-by-line. Instead, the application communicates with the DBMS and maintains internal data indexing (conceptually similar to the index at the back of a textbook), a student filtering for "AI Supervisors" allows the SQL engine to instantly pinpoint the exact memory location of those records. This guarantees microsecond retrieval times, completely bypassing the latency of sequential parsing (Connolly & Begg, 2015).
- **Data Integrity and Concurrency Control (ACID):** The most critical architectural advantage of an RDBMS is its native concurrency management. Modern RDBMS engines strictly enforce ACID properties (Atomicity, Consistency, Isolation, Durability). As defined by Connolly and Begg (2015), the Isolation property ensures that simultaneous transactions execute independently. If two students attempt to claim a supervisor's final quota slot at the exact same millisecond, the DBMS utilises row-level locking rather than file-level locking. It safely queues the transactions, processing the first and cleanly rejecting the second, thereby mathematically eradicating the race conditions that plague flat-file architectures.

Architectural Benchmarking for Student Matchmaking

To determine the optimal architecture, both systems must be evaluated against the strict operational constraints of the student discovery and selection phases.

Table 2.3: Architectural Comparison of Database Technologies for the SSAS

Evaluation Criteria	**Flat-File Storage**	**RDBMS (MySQL / PostgreSQL)**
Search Efficiency and Time Complexity (Student Discovery Phase)	**Poor:** Relies on sequential scanning (O(N) time complexity). Filtering supervisors by specific research tags requires the application to read every single record in the text file, causing severe latency as the faculty roster grows.	**Excellent:** Utilises B-Tree indexing and SQL (O(log N) time complexity). The DBMS engine can instantly pinpoint the exact memory location of supervisors matching complex, multi-parameter filters (e.g., specific domain + available quota).
Lock Granularity and Concurrency (Student Selection Phase)	**Critical Risk:** Relies on OS-level file locking. If one student is updating a record, the entire database file is locked, freezing out all other students simultaneously attempting to use the system.	**Highly Secure:** Enforces ACID Isolation via row-level locking. If a student claims a slot, the DBMS only locks that specific supervisor's row, allowing hundreds of other students to continue using the system uninterrupted.
Data Integrity and Anomaly Prevention	**Vulnerable:** Highly susceptible to "race conditions". If two students click "Apply" at the exact same millisecond, the lack of an internal concurrency manager leads to a direct loss of data integrity, potentially double-booking a single supervisor slot.	**Robust:** The DBMS natively manages concurrent access. It safely queues simultaneous requests independently, ensuring the partial effects of one student's transaction remain invisible to others until completed, mathematically preventing double-booking.
Operational Footprint and Implementation	**Low Overhead:** Requires zero background engine or installation. Extremely lightweight, consumes minimal server RAM, and the text files are highly portable for human administrators.	**Moderate Overhead:** Requires a dedicated daemon / background service (e.g., the MySQL server) to run continuously, consuming baseline CPU and memory resources even during periods of low student traffic.
Schema Flexibility and	**Rigid:** Changes to the data structure (e.g., adding a new faculty	**Structured but Adaptable:** Relational tables prevent data

Normalisation	requirement) require the developer to write custom scripts to parse and rewrite the entire text document.	redundancy (normalisation) and can be safely altered via standard SQL migration commands without breaking existing application code.

Architectural Justification

While the zero-overhead simplicity of flat-file storage is attractive for simple data logging, the benchmarking in Table 2.3 reveals that it is fundamentally incapable of supporting the dynamic, high-traffic requirements of the TAR UMT student body.

During the student discovery phase, the lack of internal indexing in flat-files necessitates sequential scanning. For a student rapidly applying multiple domain filters to hundreds of supervisor records, this linear search approach results in unacceptable system latency (Connolly & Begg, 2015).

Furthermore, the most critical vulnerabilities of flat-file storage occur during the high-stress selection phase. If multiple students attempt simultaneous access, the lack of a concurrency control subsystem leads directly to a loss of data integrity. A flat-file cannot safely isolate the transactions, resulting in a catastrophic race condition that could erroneously assign multiple students to a single, exhausted quota slot (Connolly & Begg, 2015).

To resolve these critical bottlenecks, an RDBMS is an architectural necessity. By normalising data into distinct tables, an RDBMS guarantees microsecond response times for complex tag-based filtering. Crucially, the RDBMS enforces strict ACID properties. The Isolation property ensures that transactions execute independently. If two students attempt to claim the same supervisor slot simultaneously, the database safely queues the transactions via row-level locking, cleanly processing the first and rejecting the second (Connolly & Begg, 2015).

Database Conclusion:

The operational overhead of running an RDBMS server is an entirely acceptable and necessary trade-off to ensure the ACID-compliant row-level locking and rapid indexed filtering required for equitable student matchmaking. Therefore, an RDBMS emerges as the only mathematically safe and operationally viable data architecture for the SSAS.

2.4 Chapter Summary and Evaluation

The chapter critically reviewed the existing literature, operational paradigms, and underlying technologies relevant to the development of the Supervisor Selection and Allocation System (SSAS). By evaluating the current manual allocation process against modern software engineering principles, the review established a clear empirical justification for developing a comprehensive, automated Discovery and Request platform for the TAR UMT student body.

Through a rigorous comparative analysis of architectural and database technologies, this review has formed the technical bedrock for the proposed system. The evaluation of these technologies yielded the following definitive architectural decisions tailored specifically to the student-facing modules:

- **Platform Architecture (Web-Based Client-Server):** The literature review evaluated native mobile applications against web-based deployments. Guided by the Task-Technology Fit (TTF) model (Alyoussef, 2021, 2023) and research into cognitive overload (Bawden & Robinson, 2020), the evaluation concluded that a mobile architecture fundamentally fails to support the complex cognitive load required by the SSAS. The Discovery & Search and Request & Proposal modules require students to evaluate structured reviews, analyse supervisor profiles, and upload PDF FYP Proposals. Therefore, a platform-agnostic, web-based architecture was selected. This guarantees zero-friction onboarding and provides the necessary high-density visual canvas required for students to effectively utilise the Multi-Criteria Filtering Engine and track their application statuses.
- **Database Architecture (RDBMS):** The data persistence layer was evaluated against the strict concurrency and retrieval demands of the time-sensitive allocation week. The review demonstrated that the sequential parsing and OS-level file locking inherent to flat-file storage would induce severe latency and fail to support the real-time requirements of the SSAS. Consequently, a Relational Database Management System (RDBMS) was selected as the mandatory data architecture. The implementation of an RDBMS guarantees microsecond search efficiency via SQL indexing, directly empowering the Discovery Module's recommendation algorithms. Crucially, the RDBMS enforces ACID Isolation properties and row-level locking (Connolly & Begg, 2015). This provides the mathematical foundation required for the Supervisor Availability & Status Module, allowing the system to safely execute "Submission Blocking" logic and completely eradicate the risk of over-booking when multiple students attempt to claim a final quota slot simultaneously.

Conclusion:

Ultimately, the literature review proves that the success of the SSAS relies not just on digitising a manual form, but on engineering a mathematically safe, highly accessible environment. By integrating a web-based "thin-client" architecture with a robust, ACID-compliant relational database, the proposed system is technically equipped to handle real-time quota tracking, dynamic filtering, and secure digital submissions. These synthesised findings provide the theoretical and architectural foundation required to proceed into Chapter 3, where the specific software development methodology and system requirements (functional and non-functional) for these student modules will be systematically defined.

2.5 Citation and References

Al-Busaidi, Kamla Ali, and Hafedh Al-Shihi. 2012. 'Key Factors to Instructors' Satisfaction of Learning Management Systems in Blended Learning'. *Journal of Computing in Higher Education* 24(1):18–39. https://doi.org/10.1007/s12528-011-9051-x

Almuzara, Leticia Barrionuevo, Ma Luisa Alvite Díez, and Blanca Rodríguez Bravo. 2012. 'A Study of Authority Control in Spanish University Repositories'. *Knowledge Organization* 39(2):95–103. https://doi.org/10.5771/0943-7444-2012-2-95-1

Alyoussef, Ibrahim Youssef. 2021. 'E-Learning Acceptance: The Role of Task–Technology Fit as Sustainability in Higher Education'. *Sustainability* 13(11). https://doi.org/10.3390/su13116450

Alyoussef, Ibrahim Youssef. 2023. 'Acceptance of E-Learning in Higher Education: The Role of Task-Technology Fit with the Information Systems Success Model'. *Heliyon* 9(3):e13751. https://doi.org/10.1016/j.heliyon.2023.e13751

Bawden, David, and Lyn Robinson. 2020. 'Information Overload: An Introduction'. In *Oxford Research Encyclopedia of Politics*. Oxford University Press.

Connolly, Thomas M., and Carolyn E. Begg. 2015. *Database Systems: A Practical Approach to Design, Implementation, and Management*. 6th ed. Boston, MA: Pearson.

Ibrahim, Asaad Khaleel. 2021. 'Evolution of the Web: From Web 1.0 to 4.0'. *Qubahan Academic Journal* 1(3):20–28. https://doi.org/10.48161/qaj.v1n3a75

Laudon, Kenneth C., and Jane Price Laudon. 2020. *Management Information Systems: Managing the Digital Firm*. 16th ed. Harlow: Pearson.

Mårtensson, Moa, and Johanna Söderström. 2026. 'How Does Supervision Shape Student Thesis Outcomes? Expanding the Theory and Measurement of Supervision and Its Impact'. *Journal of Political Science Education* 22(1):1–18. https://doi.org/10.1080/15512169.2024.2446940

Maulani, Nike, Geovanne Farell, Ahmaddul Hadi, Dedy Irfan, Mazen Alzyoud, and Sharshova Regina Nikolaevna. 2025. 'Automated Academic Supervisor Allocation Using the C4.5 Decision Tree Algorithm: A Scalable Web-Based Solution'. *Journal of Hypermedia & Technology-Enhanced Learning* 3(1):37–63. https://doi.org/10.58536/j-hytel.165

Odia, O. O., and Toochukwu C. Okolie. 2024. 'Design and Implementation of a Web-Based Project Allocation System'. *Global Scientific* 12(8):6. https://www.globalscientificjournal.com/researchpaper/DESIGN_AND_IMPLEMENTATION_OF_A_WEB_BASED_PROJECT_ALLOCATION_SYSTEM.pdf

Oluwayimika, Kasumu Rebecca. 2022. 'Learning Management System in Education: Benefits and Drawbacks'. *International Journal of Trendy Research in Engineering and Technology* 07(01):17–23. https://doi.org/10.54473/IJTRET.2022.7103

Panko, Ray. 2016. 'What We Don't Know About Spreadsheet Errors Today: The Facts, Why We Don't Believe Them, and What We Need to Do'. *arXiv preprint arXiv:1602.02601*. https://doi.org/10.48550/arXiv.1602.02601

Prasetyaningrum, Ari, St Ayu Suraya, Astrid Dwi Maulani, and Laila Wati. 2026. 'Students' Anxiety in Undergraduate Thesis Writing'. *English Language in Focus (ELIF)* 8(2):115–24. https://jurnal.umj.ac.id/index.php/ELIF/article/view/29808

Rismanto, Ridwan, Arie Rachmad Syulistyo, and Bebby Pramudya Citra Agusta. 2020. 'Research Supervisor Recommendation System Based on Topic Conformity'. *International Journal of Modern Education and Computer Science* 12(1):26–34. https://doi.org/10.5815/ijmecs.2020.01.04

Shafiq, Ali, and Anbareen Jan. 2017. 'Factors Influencing Gen-Y Undergraduates' Choice of Research Supervisor: A Case Study of a Malaysian Private University'. *Educational Process: International Journal* 6(4):20–34. https://doi.org/10.22521/edupij.2017.64.2

Silberschatz, Abraham, Henry F. Korth, and S. Sudarshan. 2020. *Database System Concepts*. 7th ed. New York, NY: McGraw-Hill.

Sommerville, Ian. 2016. *Software Engineering*. 10th ed. Boston, MA: Pearson.

Stair, Ralph M., and George Walter Reynolds. 2018. *Principles of Information Systems*. 13th ed. Boston, MA: Cengage Learning.

Turnbull, Darren, Ritesh Chugh, and Jo Luck. 2021. 'Learning Management Systems: A Review of the Research Methodology Literature in Australia and China'. *International Journal of Research & Method in Education* 44(2):164–78. https://doi.org/10.1080/1743727X.2020.1737002

Valacich, Joseph S., and Christoph Schneider. 2018. *Information Systems Today: Managing in the Digital World*. 8th ed. New York, NY: Pearson.

Zhang, Yan, and Chang Liu. 2023. 'Online Teaching Quality Evaluation: Entropy TOPSIS and Grouped Regression Model'. *International Journal of Emerging Technologies in Learning (iJET)* 18(16):36–49. https://doi.org/10.3991/ijet.v18i16.41353

Chapter 3

Methodology and Requirements Analysis

3 Methodology and Requirements Analysis

After building the theoretical foundation and architectural decisions established in Chapter 2, this chapter transitions the Supervisor Selection and Allocation System (SSAS) from conceptual research into actionable system engineering. It details the structured framework used to define the project's technical scope, beginning with the selection and justification of an appropriate software development methodology. Furthermore, this chapter outlines the specific requirement gathering techniques applied on the TAR UMT stakeholders and presents a comprehensive requirement analysis visualised through use case diagrams and use case descriptions. Ultimately, these gathered insights are synthesised into rigorous functional and non-functional requirements, establishing a definitive and testable blueprint for the subsequent implementation and testing phases.

3.1 Methodology

The development of the Supervisor Selection and Allocation System (SSAS) follows the Waterfall Model, also known as the Linear Sequential Lifecycle. This model dictates a disciplined, phase-by-phase progression where each phase must be fully completed and documented before the next one begins (Sommerville, 2016).

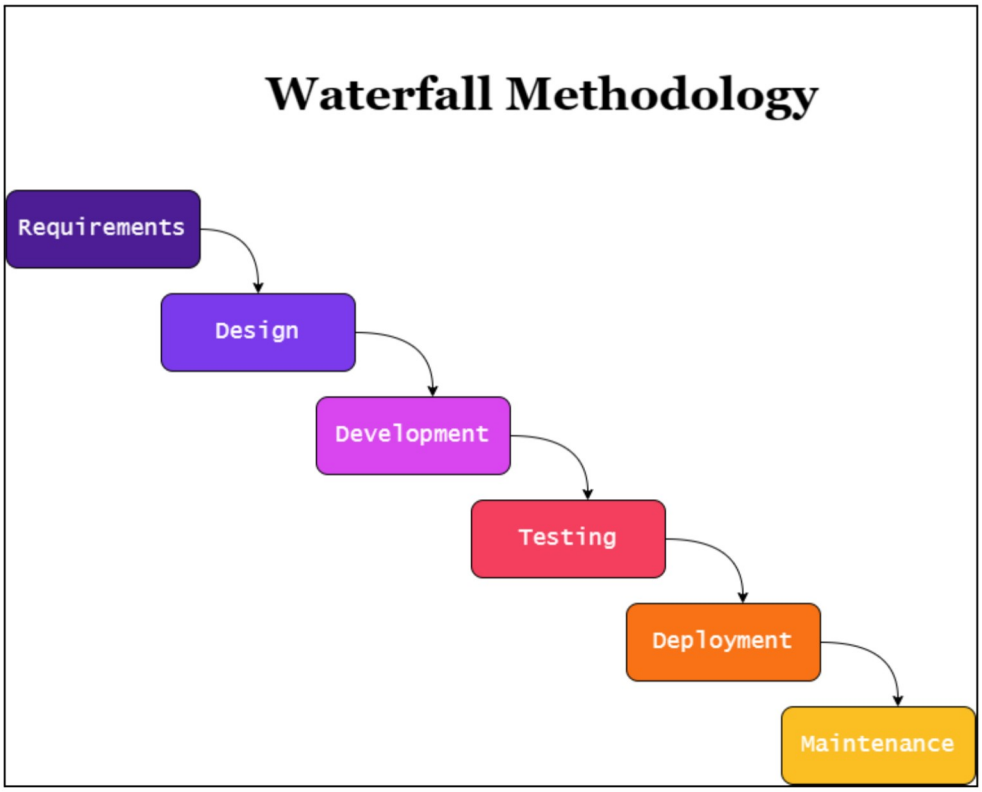

Figure 3.1: The Waterfall Development Model

3.1.1 Overview of the Chosen Methodology

The Software Development Life Cycle (SDLC) model selected for the engineering of the SSAS is the traditional Waterfall Model, frequently referred to as Linear Sequential Lifecycle. Originally adapted from highly structured engineering and construction paradigms, the Waterfall model is characterised by its strict, unidirectional progression through different developmental phases, which includes Requirement Analysis, System Design, Implementation, Testing and Deployment (Sommerville, 2016).

The fundamental principle of the Waterfall methodology is that each phase must be comprehensively documented, formally reviewed, and definitively completed before the subsequent phase can commence. Unlike iterative frameworks such as Agile or Scrum, which encourages cyclical development, continuous changing of requirements, and parallel workflows, the Waterfall model heavily prioritises upfront planning. It assumes that all system requirements can be gathered and mathematically verified before any coding begins, creating a rigid, highly predictable, and easily trackable development pipeline (Stair & Reynolds, 2018).

3.1.2 Justification of the Waterfall Methodology

While modern software engineering often defaults to iterative Agile frameworks, the specific operational constraints, fixed academic deadlines, and rigid business rules of this Final year Project makes the Waterfall model the logistically and architecturally superior choice. The specific advantages and disadvantages as well as their mitigation techniques will be discussed in Table 3.1.

Table 3.1: Advantages and disadvantages for chosen methodology, as well as the strategic mitigations

Advantages	**Requirement Stability and Rule Rigidity:** The administrative parameters governing the SSAS at TAR UMT, such as the strict above 2.0 CGPA and not in 'EF' status eligibility threshold and the fixed supervisor workload quotas, are permanent, policy-based rules that will not change mid-semester. While Agile is designed for shifting requirements (Stair & Reynolds, 2018), the SSAS requires a framework that locks these stable rules into place immediately.
	Prevention of Scope Creep: A primary cause of failure in academic software projects is "scope creep", which is the continuous addition of new features. The Waterfall Methodology forces a definitive "freeze" on requirements after Phase 1 (Sommerville, 2016). This ensures the team remains strictly focused on delivering the core critical modules (such as Discovery & Search Module and the Request & Proposal Module) without being interfered by sudden requests as the deadline approaches.
	Linear Architectural Dependency: The SSAS relies heavily on a complex relational database to manage concurrent user sessions and row-level locking (to prevent double-booking a supervisor). These backend systems possess strict linear dependencies. For example, the Recommendation Algorithm cannot be coded until the database schema is fully finalised. Waterfall's sequential approach inherently protects these dependencies.

	Comprehensive Documentation for Quality Assurance: The traditional sequential approach produces numerous intermediate products at the end of each phase that can be formally reviewed to measure progress and system accuracy (Stair & Reynolds, 2018). Because Waterfall requires this exhaustive documentation, it provides a flawless roadmap for the subsequent testing phase. For example, the test cases for the Submission Blocking logic can be written directly from the Requirement Analysis documents, ensuring a highly structured and traceable Quality Assurance process.
Disadvantages and Strategic Mitigations	**System Inflexibility:** The most significant weakness of the Waterfall model is its rigidity, returning to a previous phase to alter a core requirement is mathematically and temporally expensive once coding has begun (Stair & Reynolds, 2018). ● **Mitigation Strategy:** To neutralise this risk, the development team allocated disproportionally heavy time and resources to the initial Requirement Analysis and Design phases. By mathematically verifying the logic of the allocation engine before any backend PHP or MySQL coding commenced, the need for backward revision was virtually eliminated.
	Delayed Visualisation for Stakeholders: In a traditional Waterfall process, working software is not visible until the late stages of development, which can lead to a disconnect between user expectations and the final product (Sommerville, 2016). ● **Mitigation Strategy:** The team mitigated this by heavily prioritising UI / UX design during Phase 2. By creating comprehensive, high-fidelity visual wireframes of the Real-Time Dashboard and Discovery & Search page early on, the team was able to validate the user flow and navigational logic without needing to write premature, disposable code.
	Late-Stage Testing Bottlenecks: Since that formal testing (Phase 4) is pushed toward the end of the lifecycle, critical bugs can sometimes compound unnoticed until being tested at the end (Stair & Reynolds, 2018). ● **Mitigation Strategy:** The team mitigated this risk through rigorous and exhaustive documentation during the initial Requirement Analysis phase. By defining crystal-clear business rules (such as the exact CGPA eligibility thresholds and strict supervisor quotas) upfront, the pass / fail criteria for the system were inherently locked in. This allowed the team to execute the formal Software Testing Life Cycle (STLC) efficiently in Phase 4 without needing to pause and clarify expected system behaviours.

3.1.3 Application of the SDLC to the SSAS

To ensure the successful of the engineering to the system, the Waterfall methodology was mapped directly to the specific functional modules of the SSAS through the following phases:

Phase 1: Requirements

This initial phase focused on identifying core administrative bottlenecks and defining the project scope. According to Sommerville (2015), the goal is to establish a complete description of the system's intended functions. For the SSAs, this involved analysing manual Excel-based workflows and mapping the requirements for the three primary actors (Students, Supervisors, and Administrators) using Use Case Diagrams. Key constraints, such as real-time quota limits and the need for Anonymous Review toggles, were permanently locked in during this phase.

Phase 2: Design

Once requirements were finalised, the structural and visual blueprints were established. Architectural models were created to map the relational database schema required to support the Supervisor Availability & Status Module. Simultaneously, UI wireframes were created to visualise the end-user experience, ensuring the Timeline & Milestone Module and the Multi-Criteria Filtering Engine provided optimal Information Density without causing cognitive overload.

Phase 3: Development

In this phase, the system was developed based on the blueprints established during the Design phase. Utilising a technology stack of HTML, CSS, JavaScript, PHP, and MySQL within a local XAMPP environment, the theoretical models were translated into machine-readable code (Valacich & Schneider, 2019). Development focused heavily on building the core logic engines, specifically coding the Recommendation Algorithm, configuring the PDF upload parameters for the Request & Proposal Module, and implementing the backed database locking mechanisms.

Phase 4: Testing

The testing phase was governed by a high-level Software Testing Life Cycle (STLC) framework to systematically validate the system's reliability (Sommerville, 2016). Utilising Katalon Studio for Automated Functional Testing alongside manual User Acceptance Testing (UAT), every module was pushed to its limits. A critical focus of this phase was challenging the system's "Negative Logic", rigorously testing the Submission Blocking feature to

mathematically guarantee that over-booking a supervisor's quota was impossible under high-traffic conditions. *(A comprehensive breakdown of the STLC test cases and execution parameters is detailed in Chapter 5)*

Phase 5: Deployment

As the final stage of the project's scope, the isolated functional modules were integrated into a stable working prototype. Because this project serves as an academic proof-of-concept, the lifecycle concludes with a final evaluation against the initial project goals and a presentation of the compiled system, rather than full-scale institutional deployment (Stair & Reynolds 2018).

3.2 Requirement Gathering Techniques

Requirements gathering is a critical phase in the Waterfall development lifecycle, ensuring that the proposed system architecture is tightly aligned with the operational realities and constraints of the end-users. To establish the comprehensive functional and non-functional requirements for the student-centric modules of the SSAS, a mixed-methods approach was employed, integrating direct system observation, asynchronous stakeholder interviews, and quantitative surveys. By gathering data from multiple distinct user sources, specifically students, faculty supervisors, and program coordinators, this approach ensures a more complete system specification. As noted by Sommerville (2016), engaging multiple stakeholder viewpoints is essential to cross-verify operational bottlenecks and identify conflicting system requirements that a single user group might overlook.

3.2.1 Asynchronous Unstructured Interview (Email)

To define the precise institutional policies, deadline structures, and algorithmic constraints that govern the application process, direct asynchronous communication was established with the primary domain expert and key member of the FOCS FYP coordination team.

Justification for Technique:

This technique was chosen because it ensures absolute requirement traceability while respecting administrative schedules. Email naturally creates an archived "paper trail", providing a permanent reference point that could be revisited during the system's backend logic implementation. Furthermore, due to the high-density workload of the FOCS FYP team, this method allowed the domain expert to consult institutional records and provide high-quality, comprehensive responses without the pressure of a real-time meeting. As noted by Meho (2006), asynchronous interviewing via email allows busy domain experts the necessary time to deliver more detailed and accurate information compared to immediate, synchronous interviews.

How it worked:

Due to severe scheduling constraints, this interview was conducted iteratively through email correspondence. Unlike a structured live interview, this method allowed the stakeholder to document the precise workflow of the current manual supervisor allocation process at their own pace, allowing them to prepare their reply at any time they want. It functioned as a recursive dialogue, where initial questions regarding general workflows naturally led to deeper, highly specific inquiries about edge cases and administrative exceptions.

Benefits for the SSAS:

This technique successfully extracted the precise mathematical logic and threshold parameters required to program the student-facing modules. Specifically, the interview yielded the exact duration limits and scheduling constraints necessary to calibrate the automated countdown timers within the Timeline & Milestone Module. Additionally, it clarified the tiered quota limits based on different lecturer roles (e.g., full-time and part-time staff). Gathering these concrete business rules establishes the foundational database logic required to engineer the Submission Blocking mechanism and the dynamic visual indicators in the Supervisor Availability Module, with the specific values to be detailed in the subsequent Requirement Analysis phase.(Refer to Appendix A for the complete interview correspondence).

3.2.2 Observation and Document Analysis of the "As-Is" System

In addition to asynchronous email interviews, a secondary data collection methodology was executed by directly observing the current manual workflows and conducting a document analysis of both institutional artefacts and existing academic literature (Tam & Abdullah, 2025; Shah, 2024; Hussin & Azlan, 2017).

Justification for Technique:

Observation (often referred to as "shadowing" or ethnographic analysis in software engineering) is a highly recommended requirements elicitation technique because it uncovers tacit knowledge, undocumented workarounds, and systemic inefficiencies that users may fail to articulate in formal interviews (Sommerville, 2016). It reveals latent bottlenecks that only become visible when users are actively engaging in the allocation process. Simultaneously, reviewing existing documentation and related literature is a standard, highly effective fact-finding method for understanding the current environment and identifying areas for improvement (Stair & Reynolds, 2018). Analysing existing research on similar university systems provides a roadmap of lessons learned, preventing the reinvention of the wheel and offering evidence-based design insights.

How it worked:

This approach involved "shadowing" the existing manual process, mapping the lifecycle of a student request from the moment an inquiry is sent via email to the point it is manually recorded in the master Excel spreadsheet. The structure of these spreadsheets was examined to understand how the FOCS faculty categorised, updated, and maintained data. Concurrently, document analysis was performed on existing case studies on similar university allocation systems to identify industry best practices and common architectural pitfalls.

Benefits for the SSAS:

This dual technique was instrumental in identifying the "synchronisation gap", a critical vulnerability where the master Excel list becomes rapidly outdated while supervisors manage separate, private email threads. This gap results in severe "Information Asymmetry" and "Communication Latency" for the students. Experiencing the cognitive load of navigating a static, outdated directory served as the direct empirical catalyst for designing the Supervisor Availability & Status Module's real-time indicators. Furthermore, observing the fragmented email trails justified the architectural need for the Request & Proposal Module, which

specifically replaces external email clients with a centralised, state-tracking dashboard (Pending, Accepted, Rejected) to permanently eliminate the synchronisation gap.

3.2.3 Role-Based Structured Questionnaires

While asynchronous unstructured interviews and document analysis provided the foundational business rules for the SSAS, a quantitative approach was required to validate the specific pain points experienced by the student body and faculty. To achieve this, a role-based structured online questionnaire was deployed for the target end-users, including 49 students, 4 supervisors and 1 administrator in TAR UMT (Refer to Appendix B for the survey instrument). This section specifically details the elicitation of data required to engineer the student-centric interfaces and the supervisor availability tracking mechanisms.

Justification for Technique:

Questionnaires are a highly effective empirical data collection method when the objective is to gather statistically significant feedback and generalize findings from a large, distributed population, such as a university student body (Ghazi et al., 2019). Relying solely on qualitative observations can introduce developer bias. Therefore, distributing a structured survey ensures that the proposed system features, such as digital proposal submissions are driven by actual user consensus rather than assumptions.

How it worked:

The survey was designed using conditional logic to isolate the feedback relevant to specific system workflows based on user roles. For the development of the core student and availability modules, the data collection focused on two critical areas:

- **Student Experience Metrics:** Respondents were asked to quantify the cognitive load and stress associated with manually finding a supervisor whose research interests aligned with their own. They were also polled on their preference for utilizing a centralized digital dashboard versus traditional email communication for submitting project proposals.
- **Capacity Management Feedback:** To design the logic for the availability tracking features, faculty respondents were asked to rate the difficulty of decentralized quota tracking and explicitly state their support for strict, system-enforced submission constraints once their capacity is reached.

Benefits for the SSAS:

The quantitative data extracted from this survey served as the direct empirical justification for three primary functional modules. First, the statistical evidence highlighting the difficulty of manual topic-matching validated the necessity of the Discovery & Search Module, specifically the need for an automated recommendation algorithm based on expertise tags. Second, the overwhelming preference for centralized proposal tracking formally justified the

architecture of the Request & Proposal Module. Finally, the faculty feedback regarding quota frustration provided the mandate to engineer the Supervisor Availability & Status Module, confirming that real-time vacancy dashboards and strict “Submission Blocking” mechanics are critical requirements for the final prototype.

3.3 Requirement Analysis

In Section 3.2, the multi-method requirement elicitation strategy used was outlined to understand the operational bottlenecks of TAR UMT's manual supervisor allocation process. By utilizing an asynchronous unstructured interview, direct system observation, "As-Is" system document analysis and role-based structured questionnaires, a robust qualitative business rules and quantitative user preferences was successfully captured. This section, Requirement Analysis, serves as the critical bridge between that initial data collection and the final system architecture. To maintain strict traceability to the data collection phase, the analysis is categorized into three primary subsections: Analysis of Asynchronous Unstructured Interview Findings (Section 3.3.1), Analysis of Observation and Document Analysis Findings (Section 3.3.2), and the Analysis of Role-Based Structured Questionnaires Findings (Section 3.3.3). Finally, to visualise these extracted requirements, this section will culminate in a dedicated architectural mapping of the Use Case diagrams and descriptions (Section 3.3.4) to define the precise actor-to-system interactions required for the SSAS prototype.

3.3.1 Analysis of Asynchronous Unstructured Interview Findings

The asynchronous email correspondence with the FOCS FYP coordination team yielded critical, quantitative business rules necessary for establishing the backend logic of the SSAS. The data extracted from this interview was analysed and translated into strict system constraints, directly informing the architecture of the supervisor availability tracking and timeline management modules.

1. **Tiered Quota Constraint for Supervisor Availability Tracking**

   The most critical finding from the interview was the explicit definition of supervisor workload limits, which vary drastically based on the staff member's administrative role. The interview established the following tiered quota system:

   - **Standard Full-Time Lecturers:** Maximum of 15 students per semester, and a maximum of 30 students per academic year.
   - **Lectures with Administrative Roles (Dean, Deputy Dean, AD, PL):** Maximum of 15 per academic year to prevent overburdening.
   - **Part-Time (PT) Lecturers:** Limited to a maximum of 10 students per semester.

   Design Justification: The data collected defines the logic for the Supervisor Availability & Status Module. The system database must include "Role" Tags for each lecturer to dynamically set their maximum capacity threshold. Once the system detects that a supervisor has reached their specific mathematical limit (e.g., a Full-Time lecturer hitting 15/15 in a semester), the Submission Blocking mechanism must automatically trigger, physically disabling the "Apply" button and turning their Dynamic Status Badge to red ("Full") to prevent over-booking.

2. Semester-Dependent Scheduling & Deadlines

The interview clarified the rigid, time-sensitive nature of the FYP selection process. After the initial briefing from the course coordinator, students will be able to secure a supervisor in a strict timeframe, which change depending on the current academic semester:

- **Long Semester (LS):** Students have a maximum of 2 weeks to finalize a supervisor.
- **Short Semester (SS):** The timeframe is accelerated, giving students a maximum of 1 week (including weekend).
- Following the student phase, lecturers are given exactly 1 additional week to finalise their supervisee lists. The final lists will be distributed to students and supervisors in 2 weeks. The Course Coordinator will then assign any lecturer available for supervision to students who are not able to secure a supervisor.

Design Justification: These temporal constraints are the foundational parameters for the Timeline & Milestone Module. The system must feature a Phase Tracker to manage the differing access rights during these periods. During the initial 7 or 14-day window, the system is in the "Submission Phase", where the Deadline Countdown is active and students can utilize the Request & Proposal Module to submit PDFs. Once the countdown expires, the Phase Tracker must automatically lock student submissions and shift the system state to the "Review Phase". During this final 1-week period, supervisors utilize their dashboard to review pending PDF proposals, execute Accept / Reject actions, and officially fill their quotas before the final list is automatically generated.

3. Baseline Eligibility Pre-Conditions

The domain expert noted that students with a CGPA below 2.0 or an "EF" (Existing Students (Appeal Successful – Failed Out)) status are not permitted to undertake their FYP in the following semester. Furthermore, students must strictly be in Year 2, Semester 3 to participate.

Design Justification: While primarily an administrative filtering rule, this impacts the initial access to the student-centric modules. The system must verify these academic preconditions before a student is granted access to the Discovery & Search Module

or permitted to utilise the Request & Proposal Module to send digital PDF submissions.

3.3.2 Analysis of Observation and Document Analysis Findings

The systematic observation of TAR UMT's existing "As-Is" manual process (relying on static Excel spreadsheets and fragmented email chains), combined with a review of existing institutional case studies, provided the structural blueprint for resolving current operational inefficiencies. The analysis of these sources directly informed the architecture of the student-facing interfaces and capacity-tracking mechanisms.

Resolution of the "Synchronisation Gap" (Observation Analysis)

Direct observation of the legacy workflow revealed a severe synchronisation delay. Supervisors would accept students via email, but the master Excel directory remained unaltered for days. This created a "Phantom vacancy" issue, where students unknowingly wasted time applying to lecturers whose quotas were already full.

- *Design Justification:* To resolve this, the system must implement the Supervisor Availability & Status Module. It requires Dynamic Status Badges that automatically shift from "Available" (Green) to "Full" (Red) when a supervisor accepts a proposal. Furthermore, to eliminate human error, a strict Submission Blocking constraint must be engineered to physically disable the "Apply" button for any full supervisor.

Centralised of Proposal Submissions (Observation Analysis)

Observing the manual communication process highlighted a critical "Loss of State" for students. Proposals were sent via personal email clients, leaving students without a clear timeline of when their PDF was viewed, often resulting in severe anxiety and redundant follow-up emails.

- *Design Justification:* The analysis necessitates the creation of the Request & Proposal Module. The system must strip communication out of external email clients and provide a centralised digital dashboard. Students must be able to upload PDF proposals directly to the platform and view a real-time Status Tracker (Pending, Accepted, Rejected), providing immediate transparency and reducing cognitive load.

Fragmented Expertise Discovery (Observation Analysis)

Mapping the current student user journey revealed a highly fragmented discovery process. To determine a potential supervisor's research area, students cannot view expertise within the master Excel list, instead, they are forced to leave the workflow and navigate an external portal (The official TAR UMT staff directory). This heavy context-switching drastically increases the time required to compile a list of suitable candidates.

- *Design Justification:* To resolve this fragmentation, the architecture requires a consolidated Discovery & Search Module. By allowing supervisors to maintain "Interest Tags" directly within the SSAS, students can perform multi-criteria filtering (e.g., searching for "IoT" and "AI" simultaneously) without ever leaving the platform, drastically reducing cognitive load and time-on task.

Absence of Temporal Visibility (Observation Analysis)

Observation of the manual process indicated a complete lack of system urgency. Because the deadlines (e.g., the 2-week selection window) are only communicated verbally during the initial briefing, students are forced to manually track their own time constraints, which frequently leads to missed deadlines and last-minute panic. Furthermore, the schedule for lecturer finalisation and the official list distribution is completely opaque to the student body, forcing students into a state of indefinite waiting without a definitive publication date.

- *Design Justification:* This observation justifies the absolute necessity of the Timeline & Milestone Module. The system UI must feature persistent, dynamic Deadline Countdown timers on the student dashboard. By visually displaying the remaining time (e.g., "3 Days Left"), the system inherently enforces temporal visibility and urgency. Additionally, the Phase Tracker must clearly broadcast the current system state (e.g., shifting to "Review Phase"), providing students with complete transparency regarding when to expect their final results.

Algorithmic Benchmarking (Document Analysis)

In addition to physical observations, existing literature on academic allocation systems was analysed to establish evidence-based design standards for the SSAS:

- **Tag-Based Discovery Algorithms:** Based on the architectural recommendations of Shah (2024), relying on simple scrolling directories is highly inefficient for academic pairing. Shah's study emphasises the necessity of aligning specific student research interests with corresponding supervisor expertise. This directly validates the engineering of the Discovery & Search Module, specifically the requirement for an Recommendation Algorithm powered by self-selected "Interest Tags" (e.g., IoT, Machine Learning).
- **Granular Real-Time Vacancy Data:** Research conducted by Tam and Abdullah (2025) highlights that providing real-time information regarding available slots is a critical factor in helping students save a significant amount of time during the application process. This empirical finding directly justifies the UI design of the Supervisor Availability Module. To maximise this time-saving benefit, the system dashboard must display highly granular, fractional quota data (e.g., "3/8 slots taken") rather than relying on delayed manual updates, allowing students to instantly assess capacity before initiating contact.
- **Mitigating "First-Come, First-Served" Bias:** Hussin and Azlan (2017) identified a major flaw in manual allocation systems: students often panic and rush to secure any available supervisor, leading to poor academic compatibility. To counteract this stress-induced bias, the SSAS must implement the Student Profile and Student Review Module. By integrating a Structured Review System where seniors can leave 5-star ratings and specific feedback on lecturers, juniors are empowered with qualitative data to make informed, academically sound decisions rather than rushing to the first available slot.

3.3.3 Analysis of Role-Based Structured Questionnaires Findings

To validate the qualitative findings derived from the observations and literature review, a role-based structured questionnaire was deployed to the target population at TAR UMT. (Refer to Appendix C for the comprehensive survey data summary). Because this specific subset of the system architecture focuses on the end-user interfaces and capacity management, this analysis strictly isolates and evaluates the quantitative data provided by the student and supervisor cohorts.

Student Experience Metrics (n=49)

The student dataset provided overwhelming empirical evidence validating the necessity of the proposed student-centric modules. The analysis of their responses highlighted three primary areas of friction:

- **Demand for Centralised Discovery:** When surveyed about the difficulty of finding compatible supervisors, 77.6% of students indicate that it is difficult to find a supervisor whose research interests align with their FYP topics using the current Excel list. This statistical majority provides the mandate for the Discovery & Search Module, proving that users require an in-system, tag-based filtering mechanism rather than relying on external staff directories.
- **Preference for In-Platform Submissions:** The data revealed a severe operational bottleneck in the legacy communication method. A staggering 83.7% of students reported emailing a lecturer to request supervision, only to find out later that the quota was already full. Consequently, 65.3% of respondents explicitly preferred uploading their PDF proposals to a centralised dashboard rather than sending them via email. This directly validates the architecture of the Request & Proposal Module, cementing the requirement for real-time status trackers (Pending / Accepted / Rejected) to eliminate the anxiety of manual follow-ups.
- **The Need for Peer-Driven Data:** When asked if an anonymous, 5-star peer-review system would be helpful before choosing a supervisor, 95.9% responded positively. This quantifiable demand for transparency directly justifies the inclusion of the Student Review Module, establishing that qualitative peer feedback is a highly desired user requirement.

Capacity Management & Supervisor Feedback (n=4)

Although a smaller sample size, the feedback from the faculty supervisors provided critical validation for the backend constraints governing the system. Their responses focused heavily on workload management and administrative burden.

- **Validation of Submission Blocking:** When asked about the manual tracking of their student quotas, 3 out of 4 supervisors rated the process as highly tedious (scoring 4 or 5 out of 5). Crucially, 3 out of 4 members of the faculty cohort strongly supported a feature that physically prevents students from sending requests once their quota is 100% full. This near-unanimous consensus is the definitive justification for the strict constraint logic programmed into the Supervisor Availability & Status Module.
- **Requirement for Centralised Dashboards:** The survey revealed that supervisors spend a moderate to large amount of time answering repetitive emails. 3 out of 4 supervisors confirmed that a centralised dashboard to instantly view current applicants and remaining capacity would be highly useful, with the remaining respondent expressing a neutral stance. This feedback directly validates the UI design for the supervisor-facing interfaces, ensuring they have immediate oversight of their capacity during the active selection window.

Qualitative Feedback Synthesis

To triangulate the quantitative metrics, an open-ended feedback section allowed respondents to articulate specific operational frustrations with the "As-Is" manual system. The thematic analysis of this qualitative data heavily corroborated the statistical findings, revealing three critical areas of systemic failures:

- **The Psychological Toll of Opaque Communication:** Students repeatedly cited the emotional and temporal cost of the legacy email system, describing the "anxiety to reach out to supervisors and waiting for their reply" without knowing if they are "full or still available". The prevailing sentiment that "response time is really long and students get easily frustrated" directly justifies the Request & Search Module, which aims to replace this "lack of transparency" with a definitive, real-time status tracker.
- **Inefficiency of Fragmented Discovery:** The qualitative data highlighted the severe limitations of static spreadsheets, with users noting that "supervisor lists were always not up to date" and that it is highly "time-consuming and troublesome" to manually search for lecturer's specialisation. One detailed response explicitly requested a platform that provides a "supervisor profile that clearly shows their expertise, research interests, and available slots". This is the exact architectural blueprint for the Discovery & Search Module.
- **Administrative Burden and Data Integrity:** Beyond student frustrations, the feedback highlighted the "tedious work" caused by quota mismanagement, specifically noting the administrative friction that occurs when manual allocations are "rejected by the management due to overload". This qualitative evidence provides the ultimate validation for the strict, automated capacity constraints engineered into the Supervisor Availability & Status Module.

3.3.4 Summary of Requirement Analysis Findings

Table 3.2 synthesises the qualitative and quantitative data elicited through stakeholder interviews, system observations, document analysis, and structured questionnaires. It maps the identified operational inefficiencies directly to the architectural solutions and target modules of the proposed SSAS platform.

Table 3.2: Summary of Elicited Findings and Architectural System Justifications

Data Elicitation Source	**Key Findings & Identified Inefficiencies**	**Architectural Solutions & Target Module**
Asynchronous Interview (Course Coordinator)	**Tiered Quota Limits:** Staff have differing capacity rules (e.g., Full-Time: 15/sem, Admin: 15/yr, Part-Time: 10/sem). **Scheduling Constraints:** Deadlines vary (Long Sem: 14 days, Short Sem: 7 days), followed by a 1-week lecturer finalisation period. **Eligibility:** Strict academic prerequisites (Year 2 Sem 3, minimum CGPA > 2.0, No "EF" status).	**Supervisor Availability & Status Module:** Requires Role-Based Capacity Tags and an automated Submission Blocking mechanism. **Timeline & Milestone Module:** Requires a Phase Tracker shifting from "Submission Phase" to "Review Phase". **System Access:** Requires baseline academic verification before granting access to student-centric modules.

Direct Observation (Legacy Process)	**Synchronisation Gap:** Excel sheets lag behind actual email acceptances, creating "phantom vacancies". **Loss of State:** Students have no visibility over proposal status via email. **Fragmented Discovery:** Students must leave the workflow to cross-reference external staff directories. **No Temporal Urgency:** Systemic deadlines are not visually tracked.	**Supervisor Availability & Status Module:** Dynamic Status Badges (Green / Red) that update instantly. **Request & Proposal Module:** Centralised dashboard with PDF uploads and real-time trackers. **Discovery & Search Module:** In-platform "Interest Tags" for multi-criteria filtering. **Timeline & Milestone Module:** Persistent Deadline Countdown timers on the dashboard.
Document Analysis (Academic Literature)	**Tag-Based Matching (Shah, 2024):** Scrolling alphabetical lists are inefficient for academic pairing. **Real-Time Vacancies (Tam & Abdullah, 2025):** Live slot data is the primary metric for saving student application time. **Bias Mitigation (Hussin & Azlan, 2017):** Panic-induced "first-come, first-served" selections lead to poor compatibility.	**Discovery & Search Module:** Recommendation algorithm powered by self-selected "Interest Tags". **Supervisor Availability & Status Module:** Highly granular, fractional quota data displays (e.g., "3/8 slots"). **Student Review Module:** 5-star peer review system to encourage informed academic choices.
Structured Questionnaires (Student Cohort, n=49)	**Discovery Demand:** 77.6% struggle to align FYP topics using the current Excel lists. **Submission Bottleneck:** 83.7% wasted time applying to full quotas, 65.3% explicitly prefer a dashboard over email. **Peer Data:** 95.9% positively demanded an anonymous review system.	**Discovery & Search Module:** Statistical mandate for tag-based filtering. **Request & Proposal Module:** Justifies digital PDF uploads and Pending / Accepted tracking logic. **Student Review Module:** Validates the qualitative rating system requirement.

Structured Questionnaires (Supervisor Cohort, n=4)	**Manual Tracking:** 3 out of 4 find manual quota tracking highly tedious. **Overload Protection:** 3 out of 4 strongly support system-level submission blocking. **Visibility:** 3 out of 4 require centralised oversight of applicant capacities.	**Supervisor Availability & Status Module:** Definitive validation for strict constraint logic and physical application blocking.
Structured Questionnaires (Open Ended Question)	**Psychological Toll:** High anxiety over opaque email waiting times and lack of transparency. **Inefficiency:** Out-of-date lists and difficult specialisation searches. **Admin Burden:** Tedious quota mismanagement and overload rejections.	Triangulates and finalises the absolute necessity of the Request & Proposal, Discovery & Search and Supervisor Availability & Status Module.

3.3.5 Use Case Diagram and Use Case Description

Master Overall Use Case Diagram

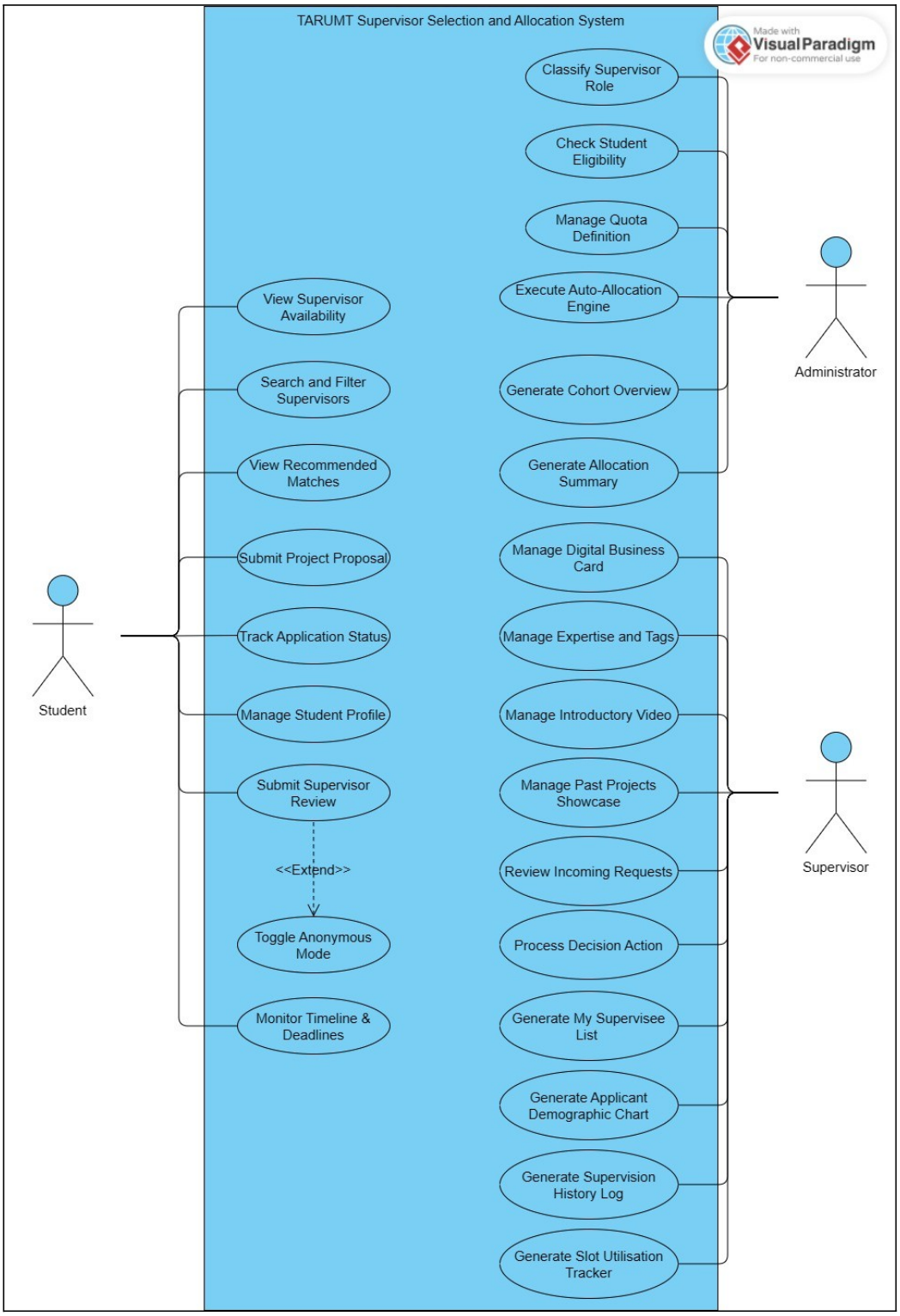

Figure 3.2 Overall Use Case Diagram for SSAS System

Individual Use Case Diagram

Module 1: Supervisor Availability & Status Module

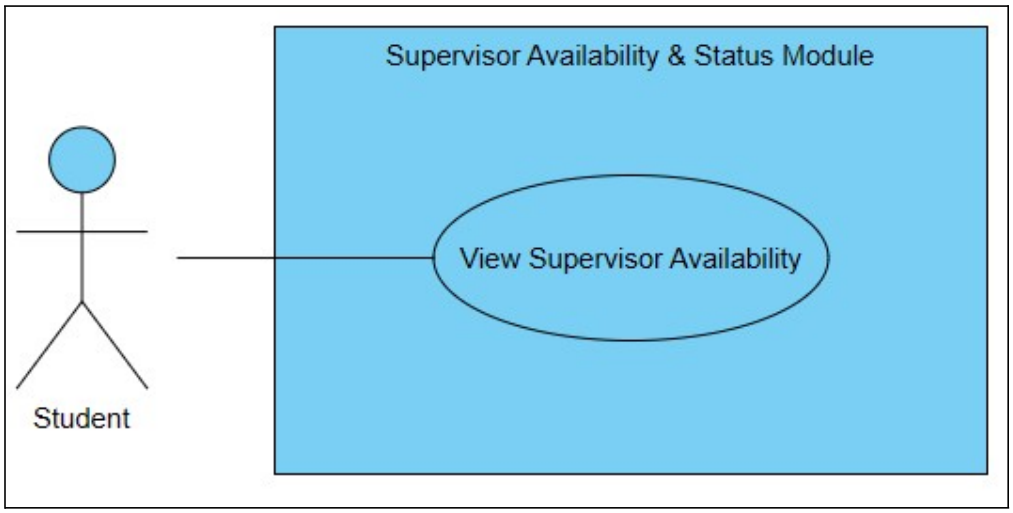

Figure 3.3 Use Case Diagram for Supervisor Availability & Status Module

Table 3.3 Use Case Description for UC100_VIEW_SUPERVISOR_AVAILABILITY

USE CASE NAME	**UC100_VIEW_SUPERVISOR_AVAILABILITY**
Use Case Description	This use case describes the interaction between the student and the system when viewing a specific supervisor's profile to check their real-time quota, view their availability status badge, and determine if they are eligible to submit a subject proposal.
Actor	Student.
Pre-condition	The user must be logged into the system.
Post-condition	The system successfully retrieves and displays the real-time capacity of the supervisor, applies the correct visual status badge, and conditionally enforces the submission blocking logic based on quota, temporal phases, and student application limit.

BASIC FLOW	
USER	**SYSTEM**
1. This use case begins when the user navigates to the Discovery list and taps on a specific supervisor's profile card [UC100_1].	
	2. The system retrieves the supervisor's active supervisee count, individual quota override and fallback administrative tier limit from the database [UC100_2]. 3. The system calculates the real-time quota using the retrieved data [C1: Quota Calculation] [UC100_3]. 4. The system evaluates the calculated quota to determine the availability state [C2: Status State Logic] [UC100_4]. 5. The system verifies the global system phase and the student's active application count [C3: Application Validation Gates] [UC100_5]. 6. The system displays the supervisor's profile page, rendering the calculated quota as a fractional integer string (e.g., "3/8") [UC100_6]. 7. The system renders a green visual badge containing the text "Available" [A1: Supervisor is Full] [UC100_7]. 8. The system renders an enabled and interactive "Apply" button [A2: Application Gates Closed] [UC100_8].
9. The user views the available status and quota data [UC100_9]. 10. The user taps the "Apply" button to proceed to proposal submission [A3: User Navigates Back] [UC100_10].	
	11. The system successfully navigates the use of the Proposal Submission interface [UC100_11].
12. Use case ends.	
ALTERNATE FLOW	
A1: Supervisor is Full	
Condition: During Basic Flow Step 4, the system determines that the active supervisee count is equal to the predefined maximum slots. A1.1 The system renders a red visual badge containing the text "Full" instead of the green badge. A1.2 The system physically disables, greys out, and removes the hyperlink functionality	

of the “Apply button”. A1.3 The system updates the button label to indicate the supervisor is no longer accepting proposals. A1.4 The user views the full status and clicks the “Discovery” link in the persistent sidebar navigation. A1.5 The system returns the user to the Discovery & Search page. A1.6 Use case ends.
A2: Application Gates Closed (System or Student Limit)
Condition: During Basic Flow Step 5, the system determines the timeline is outside the submission phase, OR the student has reached their pending application limit. A2.1 The system physically disables, greys out, and removes the hyperlink functionality of the “Apply” button. A2.2 The system updates the button label to indicate the exact restriction (e.g., “Application Closed”). A2.3 Use case returns to Basic Flow Step 9.
A3: User Navigates Back
Condition: During Basic Flow Step 9, the user decides not to apply and wishes to return to the search list. A3.1 The user clicks the “Discovery” link in the persistent sidebar navigation. A3.2 The system returns the user to the Discovery & Search page. A3.3 Use case ends.
CONSTRAINT [C]
C1: Quota Calculation & Formatting ● Backend Logic: The system prioritizes the individual supervisor quota override. If none exists, it falls back to the administrative tier limit. It performs a direct numerical comparison between the active supervisee count and the determined capacity limit. If the limit is calculated as zero, the system safely evaluates the supervisor as at maximum capacity. ● Frontend UI: System renders the quota data as a concatenated string format: [Active Supervisee Count] + “/” + [Max Slots] . C2: Status State Logic ● If Active Supervisee Count < Max Slots = Set state to “Available” ● If Active Supervisee Count == Max Slots = Set state to “Full” C3: Application Validation Gates ● The “Apply” button is protected by a multi-tier server-side validation check. It requires the system phase to be evaluated as ‘Open’ AND the student’s active pending requests to be below the maximum threshold, superseding the supervisor’s availability status.

Module 2: Discovery & Search Module

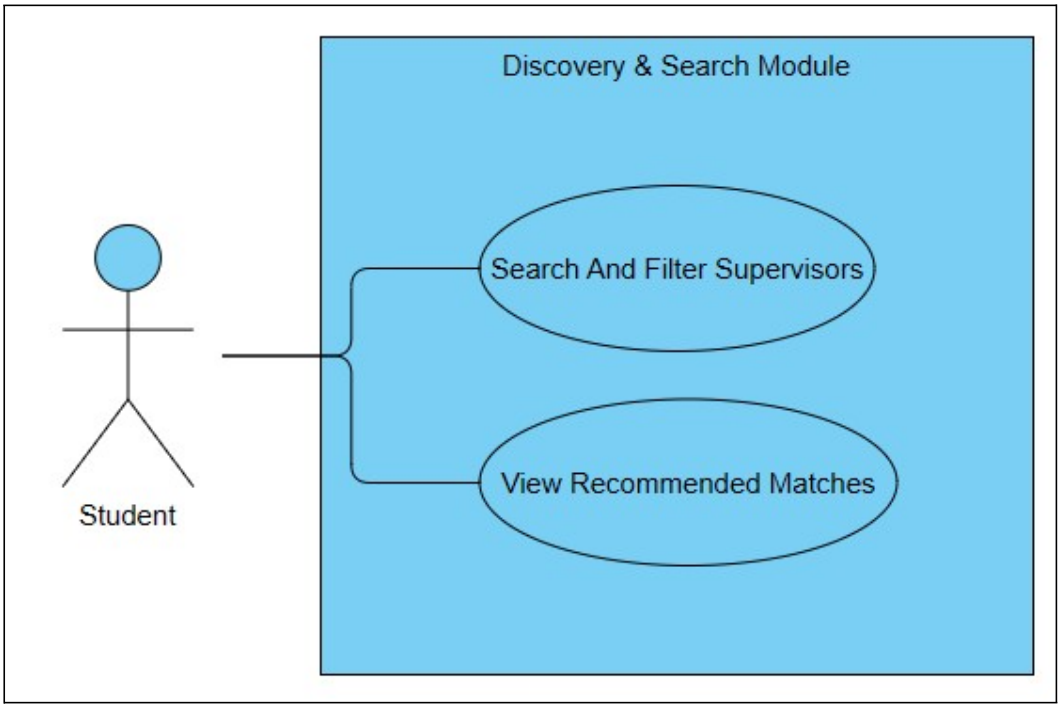

Figure 3.4 Use Case Diagram for Discovery & Search Module

Table 3.4 Use Case Description for UC200_SEARCH_AND_FILTER_SUPERVISORS

USE CASE NAME	**UC200_SEARCH_AND_FILTER_SUPERVISORS**
Use Case Description	This use case describes the interaction between the student and the system when utilising the multi-criteria filtering engine to narrow down the master list of faculty members based on specific academic requirements and availability.
Actor	Student.
Pre-condition	The user must be logged into the system.
Post-condition	The system successfully queries the database and renders an updated, filtered list of supervisor profile cards that strictly match the user's selected parameters.

BASIC FLOW	
USER	**SYSTEM**
1. This use case begins when the user navigates to the Discovery & Search page [UC200_1].	
	2. The system retrieves the master list of all active supervisors (unfiltered) and renders the default view [UC200_2].
3. The user inputs a specific name into the search bar, and/or selects filter parameters from the dropdown menus (Programme, Research Interest) and the Availability Status tab widget [UC200_3]. 4. The user taps the “Apply Filters” button [A1: User Clears Filters] [UC200_4].	
	5. The system executes sequential in-memory data processing, filtering the master list of supervisor records against the selected paramters [C1: Multi-Criteria Query Logic] [UC200_5]. 6. The system successfully retrieves the matching supervisor records [A2: No Matches Found] [UC200_6]. 7. The system refreshes the UI and renders the filtered list of supervisor profile cards [UC200_7].
8. The user views the filtered list [UC200_8].	
9. Use case ends.	
ALTERNATE FLOW	
A1: User Clears Filters	
Condition: During Basic Flow Step 4, the user taps the “Reset Filters” button instead of “Apply Filters”. A1.1 The system triggers a full page reload via a standard hyperlink. A1.2 The system clears all form states, re-fetches the unfiltered master list, and renders the default view. A1.3 Use case ends.	
A2: No Matches Found	

Condition: During Basic Flow Step 6, the database query returns zero matching records. A2.1 The system clears the current list of profile cards. A2.2 The system displays a text notification [M1: Msg No Results]. A2.3 The system returns to Step 3.
MESSAGE [M]
M1: Msg No Results = {No supervisors match your selected criteria. Please adjust your filters and try again.}
CONSTRAINT [C]
C1: Multi-Criteria Query Logic The system sequentially filters the pre-fetched active supervisor list in PHP memory. It must enforce strict Boolean AND logic across different filter categories (e.g., Programme == “Software Engineering” AND Status == “Available”) to yield the final result set.

Table 3.5 Use Case Description for UC201_VIEW_RECOMMENDED_MATCHES

USE CASE NAME	**UC201_VIEW_RECOMMENDED MATCHES**
Use Case Description	This use case describes the automated process where the system utilises an intelligent recommendation algorithm to suggest highly compatible supervisors based on the student's half selected research interests.
Actor	Student.
Pre-condition	The user must be logged into the system and must have successfully configured at least one (1) "Research Interest" tag in their Student Profile.
Post-condition	The system successfully computes compatibility scores and displays a dedicated UI section featuring the Top 3 recommended supervisors.

BASIC FLOW	
USER	**SYSTEM**
1. This use case begins when the user navigates to their main dashboard or discovery & search page[UC201_1].	
	2. The system checks the authenticated student's database record for saved Research Interest tags [A1: No Tags Configured] [UC201_2]. 3. The system extracts the array of student tags [UC201_3]. 4. The system executes the Recommendation Algorithm against a pre-filtered pool of currently available supervisors [C1: Algorithm Logic] [UC201_4]. 5. The system sorts the resulting array by Highest Match Count, followed by an alphabetical sort of the supervisor's name [UC201_5]. 6. The system limits the final output array to a maximum of three (3) supervisor records [A2: Zero Matches Found] [UC201_6]. 7. The system renders a highlighted "Top Matches For You" horizontal carousel displaying the matched profiles [UC201_7].
8. The user views the recommended matches [UC201_8].	

9. Use case ends.	
ALTERNATE FLOW	
A1: No Tags Configured	
Condition: During Basic Flow Step 4, the system detects that the student's Research Interest array is completely empty/null. A1.1 The system aborts the Recommendation Algorithm. A1.2 The system replaces the "Top Matches For You" carousel with a plain text paragraph prompt [M1: Msg Setup Profile]. A1.3 Use case ends	
A2: Zero Matches Found	
Condition: During Basic Flow Step 6, the system determines the algorithm yielded an empty array (no available supervisors matched the tags). A2.1 The system renders a plain text notification indicating no matches were found [M2: Msg No Matches]. A2.2 Use case ends	
MESSAGE [M]	
M1: Msg Setup Profile = {Update your Research Interests in your Profile to unlock personalised supervisor recommendations.} M2: Msg No Matches = {No recommended supervisors currently match your saved interests with Available slot status. Browse Discovery to review all supervisors.}	
CONSTRAINT [C]	
C1: Algorithm Logic • The system applies a strict pre-filter, exclusively extracting supervisors whose quota status evaluates to "Available" before scoring begins. • The matching algorithm cross-references the Student's Interest Array against each Supervisor's Expertise Array. 1 point is awarded per exact tag ID match. Supervisors with 0 matches are excluded from the output pool entirely.	

Module 3: Request & Proposal Module

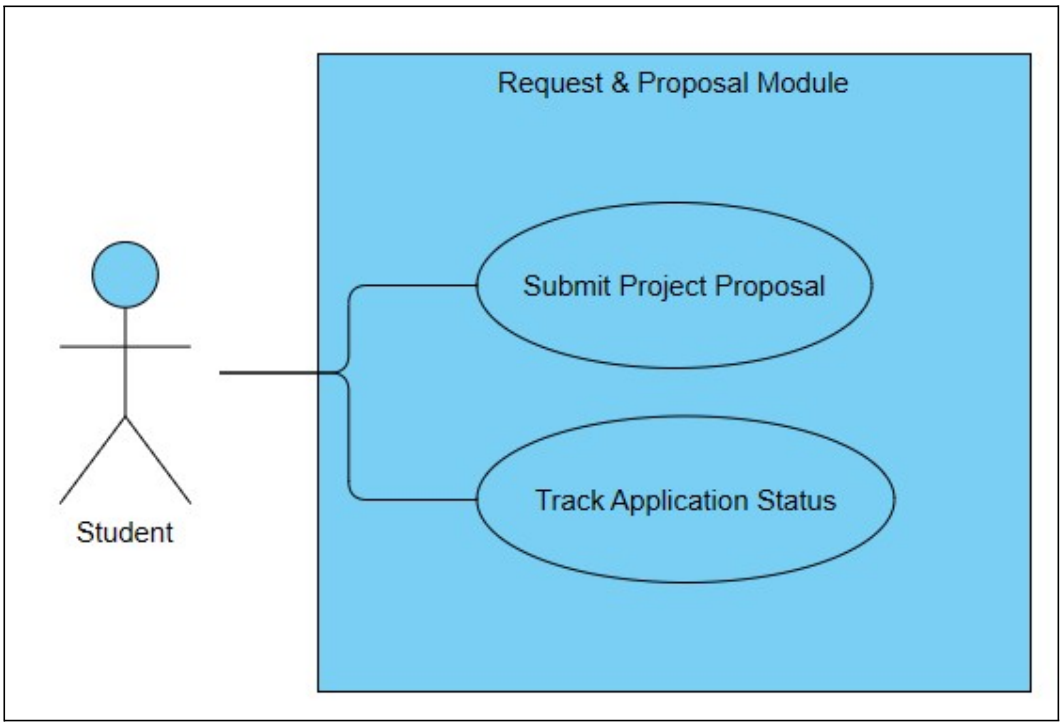

Figure 3.5 Use Case Diagram for Request & Proposal Module

Table 3.6 Use Case Description for UC300_SUBMIT_PROJECT_PROPOSAL

USE CASE NAME	**UC300_SUBMIT_PROJECT_PROPOSAL**
Use Case Description	This use case describes the interaction between the student and the system when uploading a digital project proposal document to formally request FYP supervision, utilising a concurrent application architecture with a maximum submission limit.
Actor	Student.
Pre-condition	The user must be logged into the system and explicitly verified as FYP-Eligible. The user must not currently hold an “Accepted” allocation status. The user must have fewer than three (3) active “Pending” or “Proposal Requested” proposals. The targeted supervisor must have available capacity. The system’s globally active phase must be the ‘Submission Phase’
Post-condition	The system successfully saves the proposal document to the server, updates the student’s database allocation status to “Pending”, affixes an expiration timestamp, and renders the proposal visible on the targeted supervisor’s dashboard.

BASIC FLOW	
USER	**SYSTEM**
1. This use case begins when the user navigates to the Proposal Submission interface for a specific supervisor [UC300_1].	
	2. The system renders the submission form, including mandatory Project Title text input and a file upload field restricted to PDF formats [UC300_2].
3. The user inputs their proposal Project Title and selects a PDF document from their local device [UC300_3]. 4. The user taps the “Submit Proposal” button [UC300_4].	
	5. The system validates the file size and server-side MIME type [C1: File Constraint] [A1: Invalid File Type or Size] [UC300_5]. 6. The system securely commits the file to the allocated server storage [UC300_6]. 7. The system executes a database update, logging the proposal as “Pending”, capturing the current server time as the Application Date, and generating a 72-hour expiration timestamp [C2: Concurrency & Timeout Logic] [UC300_7]. 8. The system displays a success notification [M1: Msg Submission Success] [UC300_8]. 9. The system automatically redirects the user to the Status Tracking dashboard [UC300_9].
10. Use case ends.	
ALTERNATE FLOW	
A1: Invalid File Type or Size	
Condition: During Basic Flow Step 5, the user attempts to upload a file exceeding 5.0 MB, or a non-PDF file extension (e.g., .docx, .png). A1.1 The frontend client immediately intercepts the action and blocks the file attachment. A1.2 The system alters the visual state of the upload dropzone (e.g., turning the border to red) to indicate an invalid file type. A1.3 The user remains on the submission form to select a valid file. A1.4 Flow resumes at Basic Flow Step 3.	
MESSAGE [M]	

M1: Msg Submission Success = {Your proposal has been successfully submitted to the supervisor. They have 72 hours to respond.}
CONSTRAINT [C]
C1: File Constraints The system must mathematically enforce a maximum file payload size of 5.0 Megabytes (MB) and explicitly restrict accepted MIME types to application/pdf. C2: Concurrency & Timeout Logic ● Concurrency Limit: The system shall enforce a strict mathematical maximum of three (3) concurrent “Pending” proposals per student account, calculated by combining both “Pending” and “Proposal Requested” status. ● Auto-Expiry (TTL): Every successfully submitted proposal shall be affixed with a 72-hour Time-to-Live (TTL) database timestamp. If the targeted supervisor does not execute an Accept / Reject action within exactly 72 hours, the system shall automatically update that specific proposal’s status to “Rejected (Timeout)”

Table 3.7 Use Case Description for UC301_TRACK_APPLICATION_STATUS

USE CASE NAME	**UC301_TRACK_APPLICATION_STATUS**
Use Case Description	This use case describes the interaction between the student and the system when viewing their personalised dashboard to monitor the real-time progression of their submitted project proposals, including countdown timers for pending requests and final allocation decisions.
Actor	Student.
Pre-condition	The user must be logged into the system.
Post-condition	The system successfully queries the database and renders a dynamic UI dashboard displaying the current status (Pending, Accepted, Rejected, Rejected Timeout, or Withdrawn) of all historical and active proposal submissions associated with the student's account.

BASIC FLOW	
USER	**SYSTEM**
1. This use case begins when the user navigates to the application status page [UC301_1].	
	2. The system queries the database for all proposal records linked to the authenticated student's unique ID [UC301_2]. 3. The system calculates aggregate count tiles (Total, Pending, Accepted, Closed) and formats the retrieved records into a table [A1: No Records Found] [UC301_3]. 4. For any proposal with a "Pending" status, the system embeds an expiration epoch timestamp into the page, allowing client-side JavaScript to locally tick down the remaining time-to-live (TTL) without continuous server polling [UC301_4]. 5. The system renders the UI dashboard, displaying the aggregated tiles, targeted supervisor, submission date, current status badge, and the TTL countdown timer [UC301_5].

6. The user views the statuses of their applications [UC301_6]. 7. Use case ends.	
ALTERNATE FLOW	
A1: No Records Found	
Condition: During Basic Flow Step 3, the database query returns zero proposal records for the student. A1.1 The system renders a visual “Empty State” graphic on the dashboard. A1.2 The system displays a text prompt encouraging the user to explore available supervisors [M1: Msg Empty Dashboard]. A1.3 The system renders a hyperlink / button redirecting the user to the Discovery & Search Module. A1.4 Use case ends.	
MESSAGE [M]	
M1: Msg Empty Dashboard = {You have not submitted any project proposals yet. Browse supervisors with open slots to start your application.}	
CONSTRAINT [C]	
C1: Status Badge Formatting The system apply strict UI color-coding to the status strings to enhance scannability: ● “Pending” = Yellow/Warning ● “Accepted” = Green/Success ● “Rejected” and “Rejected Timeout” = Red/Danger ● “Withdrawn (System Auto-Resolved)” = Grey/Muted	

Module 4: Student Profile Module

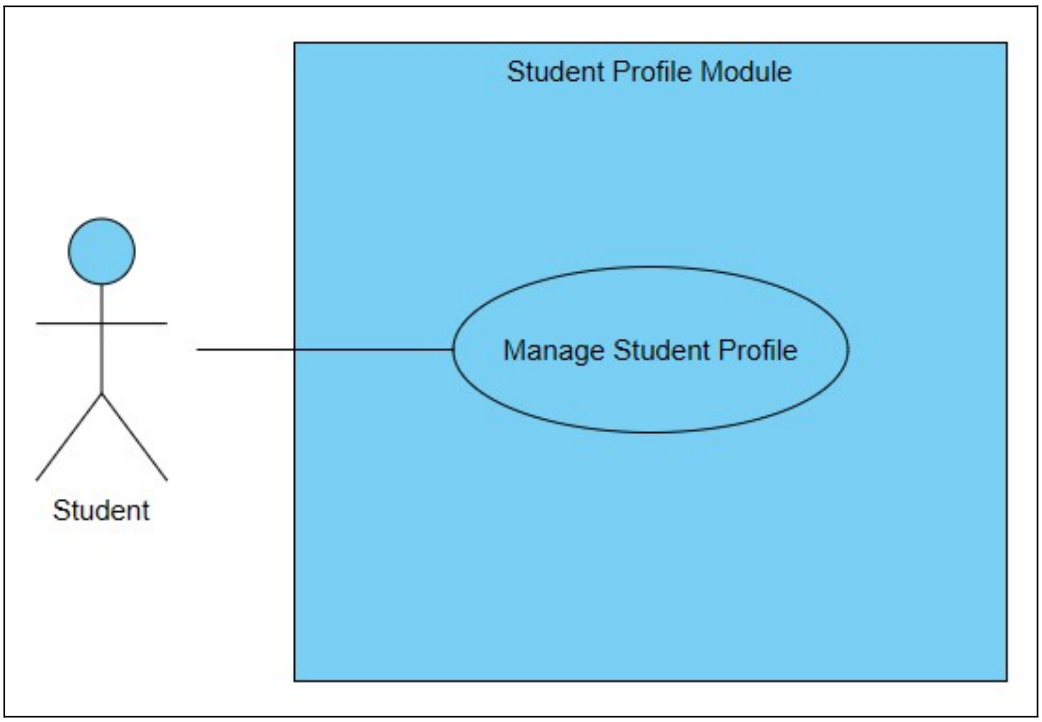

Figure 3.6 Use Case Diagram for Student Profile Module

Table 3.8 Use Case Description for UC400_MANAGE_STUDENT_PROFILE

USE CASE NAME	**UC400_MANAGE_STUDENT_PROFILE**
Use Case Description	This use case describes the interaction between the student and the system when configuring their academic and professional profile, specifically updating their contact details, personal bio, digital portfolio links, profile avatar, and selecting research interest tags to calibrate the platform's recommendation engine.
Actor	Student.
Pre-condition	The user must be logged into the system.
Post-condition	The system successfully validates the inputted profile data, updates the student's relational database record, and subsequently unlocks or recalibrates the automated Supervisor and Recommendation Algorithm.

BASIC FLOW	
USER	**SYSTEM**
1. This use case begins when the user navigates to the Profile Management interface [UC400_1].	
	2. The system retrieves the student's existing profile data (Name, Student ID, Programme, University Email, Intake Batch, Academic Status, and CGPA) and populates the uneditable fields [UC400_2].
3. The user inputs or updates their Contact Number and Personal Bio [C1: Bio Constraints] [UC400_3]. 4. The user uploads a profile picture file via the designated interface [C3: Avatar File Constraints] [A1: Invalid Avatar Upload] [UC400_4]. 5. The user selects or deselects "Research Interest" tags from the predefined UI grid [A2: Tag Limit Exceeded] [UC400_5]. 6. The user inputs their external URLs (LinkedIn, GitHub, Portfolio) [UC400_6]. 7. The user taps the "Save Changes" button [UC400_7].	
	8. The system validates URL inputs [C3: URL Validation] [A3: Invalid Input Format] [UC400_8]. 9. The system executes a database commit, updating the student's full profile record [UC400_9]. 10. The system displays a success notification [M1: Msg Profile Saved] [UC400_10].
11. Use case ends.	
ALTERNATE FLOW	
A1: Invalid Avatar Upload	
Condition: During Basic Flow Step 4, the system detects that the uploaded file exceeds the maximum file size or is not an accepted MIME type. A1.1 The system aborts the file upload process to protect server storage. A1.2 The system clears the file input field and renders a contextual error message [M4: Err Invalid File]. A1.3 The user remains on the form to select a valid file. A1.4 Flow resumes at Basic Flow Step 4.	

A2: Tag Limit Exceeded
Condition: During Basic Flow Step 5, the user attempts to select a 6th research interest tag when the array is already at maximum capacity. A2.1 The UI allows the toggle click, but the system immediately triggers a rollback, reverting the tag to an “unselected” state. A2.2 The system triggers a block native browser dialog [M2: Err Max Tags]. A2.3 The user dismisses the alert and must deselect an existing tag to add a new one. A2.4 Flow resumes at Basic Flow Step 5.
A3: Invalid Input Format
Condition: During Basic Flow Step 8, the system detects that the inputted URLs are improperly formatted. A3.1 The browser natively intercepts and prevents the form submission. A3.2 The browser renders a native tooltip pointing directly to the offending input field displaying a standard URL error [M3: Err Invalid Data]. A3.3 The user remains on the form to correct their inputs. A3.4 Flow resumes at Basic Flow Step 8.
MESSAGE [M]
M1: Msg Profile Saved = {Your profile has been successfully updated.} M2: Err Max Tags = {You can select a minimum of 1 and a maximum of 5 research interests . Please remove one to add another.} M3: Err Invalid Data = {Please enter a URL} M4: Err Invalid File = {Upload failed. Please ensure your profile picture is in .JPG or .PNG format and does not exceed 5MB.}
CONSTRAINT [C]
C1: Bio Constraints The system must enforce a maximum character limit of 500 characters on the “Personal Bio” text area to prevent database bloat. C2: URL Validation The system must execute frontend Regular Expression (Regex) validation on the LinkedIn, GitHub, and Portfolio fields to ensure the inputted string is a properly formatted hyperlinks. C3: Avatar File Constraints The system must restrict profile picture uploads to a maximum file size of 2.0 Megabytes (MB) and strictly accepts only .jpg, .jpeg or .png MIME types to ensure rendering compatibility and prevent malicious file executions.

Module 5: Student Review Module

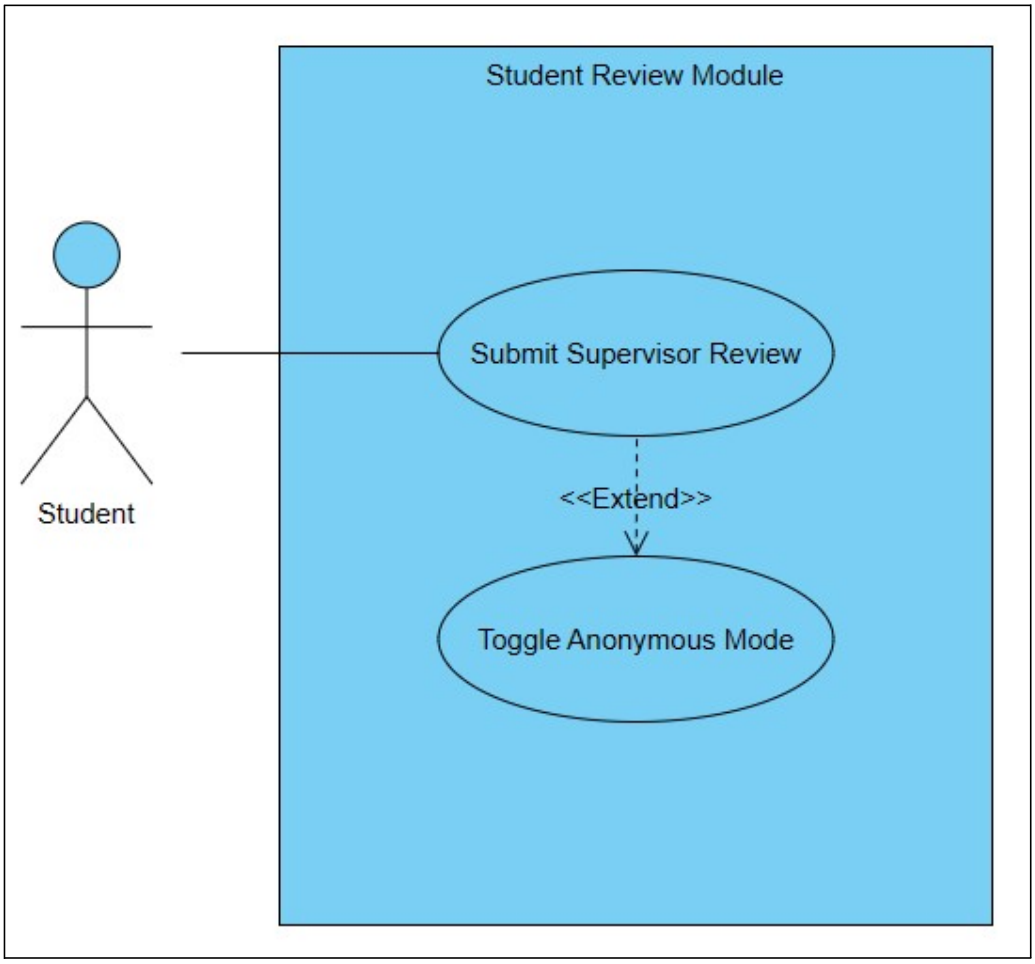

Figure 3.7 Use Case Diagram for Student Review Module

Table 3.9 Use Case Description for UC500_SUBMIT_SUPERVISOR_REVIEW

USE CASE NAME	**UC500_SUBMIT_SUPERVISOR_REVIEW**
Use Case Description	This use case describes the interaction between the student and the system when evaluating their allocated supervisor's performance at the end of the academic cycle, utilising a rating scale and optional text feedback with data-masking privacy controls.
Actor	Student.
Pre-condition	The user must be logged into the system. The user must currently hold an active allocation status with a valid supervisor. The administrative Timeline phase must be set to the "Review Period".
Post-condition	The system successfully saves the evaluation to the database, prevents the student from submitting duplicate reviews, and recalculates the targeted supervisor's aggregate rating score without exposing the student's identity on the public frontend.

BASIC FLOW	
USER	**SYSTEM**
1. This use case begins when the user navigates to the "Submit Review" interface [UC500_1].	
	2. The system queries the database to verify the student's active allocation record and checks for pre-existing reviews tied to that specific allocationID [A1: Duplicate Review Attempt] [UC500_2]. 3. The system renders the evaluation form, including a 5-star rating component, a text area, and an "Anonymous Submission" toggle [UC500_3].

4. The user interacts with the rating component to select a numerical score from 1 to 5 [UC500_4]. 5. The user input their evaluation into the text feedback area [C1: Text Limit] [UC500_5]. 6. The user activates the "Anonymous Submission" toggle [UC500_6]. 7. The user taps the "Submit Review" button [A2: Missing Mandatory Rating] [UC500_7].	
	8. The system executes a secure database commit, linking the review payload to the Supervisor ID [C2: Anonymity Data Masking] [UC500_8]. 9. The system triggers a backend math function to recalculate and update the supervisor's global aggregate rating [UC500_9]. 10. The system displays a success notification and executes a full page redirect to a "Completed" UI state [M1: Msg Review Success] [UC500_10].
11. Use case ends.	
ALTERNATE FLOW	
A1: Duplicate Review Attempt	
Condition: During Basic Flow Step 2, the database query detects that a review payload already exists for this specific allocationID. A1.1 The system aborts rendering the submission form. A1.2 The system renders a "Completed" UI state, displaying the student's previously submitted review. A1.3 The system displays an informational text prompt [M2: Msg Already Reviewed]. A1.4 Use case ends.	
A2: Missing Mandatory Rating	
Condition: During Basic Flow Step 7, the server-side validation detects a null or zero value in the mandatory numerical rating component. A2.1 The system intercepts and halts the database commit. A2.2 The system highlights the rating component in red and displays a contextual error with an internal prefix [M3: Err Missing Rating]. A2.3 The user remains on the form to select a rating. A2.4 Flow resumes at Basic Flow Step 4.	
MESSAGE [M]	
M1: Msg Review Success = {Your review has been successfully submitted. Thank you for	

your feedback.} M2: Msg Already Reviewed = {You have already submitted an evaluation for your supervisor this semester.} M3: Err Missing Rating = {Submission failed. You must select a star rating (1-5) to submit a review. Text feedback is optional.}
CONSTRAINT [C]
C1: Text Limit The system must enforce a maximum character limit of 1000 characters on the feedback text area to prevent database payload bloat. C2: Anonymity Data Masking If the “Anonymous Submission” boolean is set to TRUE, the system must permanently mask the Student_Name and Student_ID columns when retrieving this specific review record for display on the Supervisor’s frontend UI dashboard.

Table 3.10 Use Case Description for UC501_TOGGLE_ANONYMOUS_MODE

USE CASE NAME	**UC501_TOGGLE_ANONYMOUS_MODE**
Use Case Description	This is an extension use case that triggers optionally during the supervisor review process, allowing the student to mathematically mask their primary identifier columns prior to database submission.
Base Use Case	UC500_SUBMIT_SUPERVISOR_REVIEW.
Pre-condition	The user must be currently executing the Basic Flow of UC500 and must be interacting with the evaluation form.
Post-condition	The system sets the isAnonymous database boolean flag to TRUE, instructing the backend to mask the student's identity during future data retrieval.

EXTENSION FLOW	
USER	**SYSTEM**
1. This use case is triggered at Step 6 of UC500 when the user activates the "Anonymous Submission" toggle [UC501_1].	
	2. The system registers the input locally on the client-side without executing any immediate session state updates or server requests [UC501_2]. 3. The system relies on a static, always visible privacy notice explaining the anonymity rules [UC500_3].
4. Flow immediately returns to Step 7 of UC500.	

Module 6: Timeline & Milestone Module

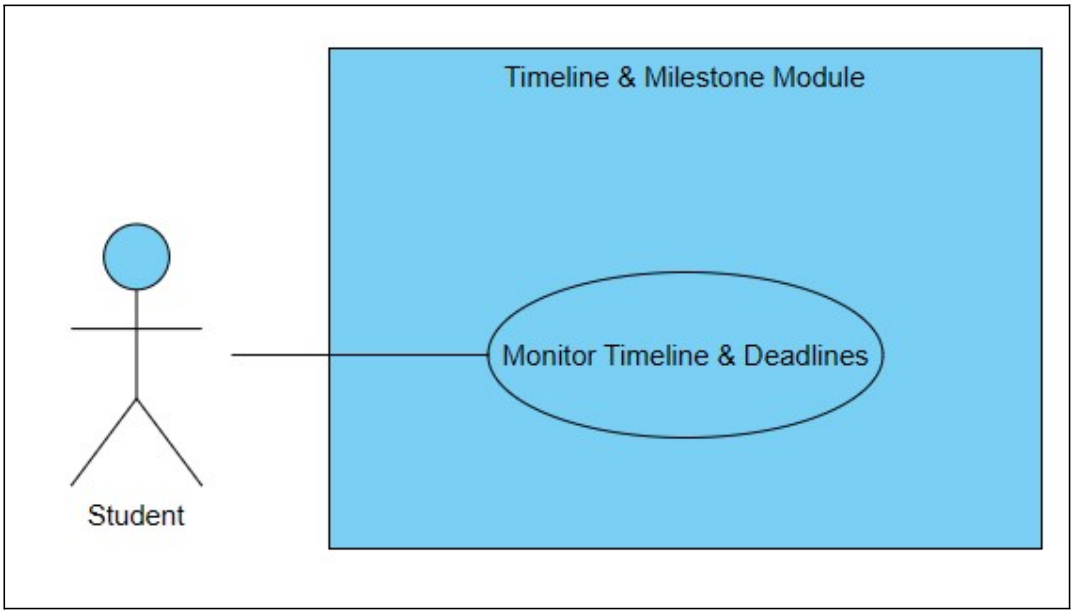

Figure 3.8 Use Case Diagram for Student Review Module

Table 3.11 Use Case Description for UC600_MONITOR_TIMELINE_AND_DEADLINES

USE CASE NAME	**UC600_MONITOR_TIMELINE_AND_DEADLINES**
Use Case Description	This use case describes the interaction between the student and the system when viewing the globally active supervisor selection phase and monitoring the live, server-synchronised countdown timer for critical academic deadlines..
Actor	Student.
Pre-condition	The user must be logged into the system.
Post-condition	The system successfully retrieves the global academic schedule, mathematically determines the current active phase, and persistently renders a live countdown timer on the student dashboard until the phase expires.

BASIC FLOW	
USER	**SYSTEM**
1. This use case begins when the user navigates to their primary Student Dashboard or the Timeline & Milestones page [UC600_1].	
	2. The system queries the database to retrieve the predefined start and end timestamps for all administrative phases [UC600_2]. 3. The system compares the centralised server time against the retrieved timestamps to determine the current "Active Phase" [UC600_3]. 4. The system renders a persistent visual banner explicitly broadcasting the title of the active phase (e.g., "Active: Submission Phase") [UC600_4]. 5. The system mathematically calculates the remaining time allowance by subtracting the current server time from the active phase's expiration timestamp to establish an initial server-offset [UC600_5]. 6. The system renders the remaining time in a dynamic visual countdown timer on the interface [C1: Timer Formatting] [UC600_6]. 7. The system ticks the timer down every second utilizing the local device clock. On the Timeline page specifically, the system continuously re-synchronises the timer via server polling at exactly 60,000ms (1-minute) intervals [A1: Countdown Expiration] [UC600_7].
8. Use case ends (The user remains on the dashboard, monitoring the time).	
ALTERNATE FLOW	
A1: Countdown Expiration	
Condition: During Basic Flow Step 7, the system detects that the countdown timer has reached exactly zero (00:00:00).. A1.1 The system automatically executes a state transition to the subsequent operational phase. A1.2 The system invokes a global server-side lock on the associated module write path (e.g., revoking student submission rights for the Request & Proposal Module, which is re-verified on every backend request). A1.3 The system updates the persistent visual banner to reflect the new phase. A1.4 Flow resumes at Basic Flow Step 5.	

CONSTRAINT [C]
C1: Timer Formatting & Time Sync The system must mathematically evaluate the current time strictly utilising the centralised backend server timestamp (e.g., UTC) to establish the initial time offset and enforce backend module locks.

3.4 Functional and Non-Functional Requirements

Building upon the empirical evidence and architectural justifications established in Section 3.3, this section transitions from problem analysis to formal system specification. The qualitative and quantitative findings are synthesised herein to define the precise technical parameters required to engineer the Supervisor Selection and Allocation System (SSAS). To ensure a robust and testable architecture, these system parameters are explicitly categorised into two distinct subsections. Section 3.4.1 details the Functional Requirements (FR), which dictate the explicit operational behaviours, user interactions, and data-processing capabilities mandated for the individual modules. Subsequently, Section 3.4.2 outlines the Non-Functional Requirements (NFR), establishing the stringent quality attributes, such as system performance, security constraints, and usability standards, that govern the overall execution of the platform.

3.4.1 Functional Requirements (FR)

1. **Supervisor Availability & Status Dashboard**

1.0 Real-Time Dashboard

- **1.1 FR1** The system shall retrieve the supervisor's active supervisee count and their predefined administrative role limit from the database upon page load.
- **1.2 FR2** The system shall mathematically calculate the real-time quota by comparing the active supervisee count against the role limit.
- **1.3 FR3** The system shall display the calculated quota as a fractional string (e.g., "3/8 slots taken") on the supervisor's public profile interface
- **1.4 FR4** The system shall automatically recalculate and update the fractional string across all user interfaces immediately upon the successful database commitment of a newly accepted student proposal.

2.0 Dynamic Status Badges

- **2.1 FR1** The system shall assign a status of "Available" to any supervisor whose active supervisee count is strictly less than their predefined role limit.
- **2.2 FR2** The system shall assign a status of "Full" to any supervisor whose active supervisee count is equal to their predefined role limit.
- **2.3 FR3** The system shall render a green visual badge containing the text "Available" on the UI for supervisor in the "Available" state.
- **2.4 FR4** The system shall render a red visual badge containing the text "Full" on the UI for supervisors in the "Full" state.

3.0 Submission Blocking

- **3.1 FR1** The system shall evaluate the assigned status state of a selected supervisor prior to initialising the proposal submission interface for a student.
- **3.2 FR2** The system shall physically disable, grey out, and remove the hyperlink functionality of the "Apply" button if the system detects the targeted supervisor is in the "Full" status state.

- **3.3. FR3** The system shall block any direct backend POST requests for proposal submissions targeted at a "Full" supervisor to prevent concurrent over-booking, returning an explicit error prompt to the user.

2. Discovery & Search Module

1.0 Multi-Criteria Filtering Engine

- **1.1 FR1** The system shall provide interactive graphical interface elements (e.g., dropdown menus and checkboxes) to allow students to input filtering parameters for Academic Programme, Research Interest, and Availability Status.
- **1.2 FR2** The system shall execute a multi-conditional database query to filter the master supervisor list immediately upon the selection or deselection of any filtering parameter.
- **1.3 FR3** The system shall process the selected filtering parameters and refresh the discovery interface to accurately display the updated list of matching supervisors.
- **1.4 FR4** The system shall display a "No Result Found" textual prompt if the applied filtering criteria yield zero matching supervisor records.

2.0 Recommendation Algorithm

- **2.1 FR1** The system shall retrieve the authenticated student's saved "Interest Tags" array from the user profile initialisation of the discovery dashboard.
- **2.2 FR2** The system shall algorithmically compare the student's "Interest Tags" array against the registered "Expertise Tags" array of every active supervisor in the database.
- **2.3 FR3** The system shall calculate a numerical matching score for each supervisor based on the exact count of overlapping tags.
- **2.4 FR4** The system shall sort the evaluated supervisors in descending order based on their calculated matching score.
- **2.5 FR5** The system shall extract the top three highest-scoring supervisors and render them in a dedicated "Top Matches For You" visual container on the student interface.

- **2.6 FR6** The system shall mathematically exclude any supervisor who currently holds a "Full" status state from being processed or displayed by the Recommendation Algorithm.

3. Request & Proposal Module

1.0 Digital Submission

- **1.1 FR1** The system shall provide a dedicated submission interface on the supervisor profile page requiring a mandatory Project Title text input (maximum 255 characters) and a document upload strictly configured to accept .pdf file extensions.
- **1.2 FR2** The system shall enforce a strict client-side file size constraint, rejecting any uploaded document exceeding 5.0 Megabytes (MB) before initiating server transmission.
- **1.3 FR3** The system shall perform secondary backend validation to ensure the uploaded file header matches a valid PDF MIME type (application/pdf) to prevent malicious file execution.
- **1.4 FR4** The system shall securely store the validated PDF document in the server directory and create a new relational database record linking the file path, Project Title, Application Date (Current Server Timestamp), student ID, and the target supervisor ID.
- **1.5 FR5** The system shall render a definitive "Submission Successful" UI confirmation dialogue immediately upon the secure database commit of the proposal document.

2.0 Status Tracking

- **2.1 FR1** The system shall automatically assign a default status state of "Pending" to all newly committed proposal submissions and concurrently generate a 72-hour Time-to-Live (TTL) database expiration timestamp.
- **2.2 FR2** The system shall programmatically enforce a concurrency limit, restricting a student account to a maximum of three (3) active "Pending" proposals simultaneously, and automatically disable new submission functionalities across the platform once this threshold is reached.
- **2.3 FR3** The system shall execute an automated background process to monitor TTL timestamps and autonomously mutate the application status to "Rejected (Timeout)" upon expiration of the 72-hour window.
- **2.4 FR4** The system shall execute an automatic conflict-resolution script, upon students receiving an "Accepted" status, the system shall locate any of their remaining "Pending" proposals and automatically mutate them to "Withdrawn (System Auto-Resolved)".
- **2.5 FR5** The system shall render a persistent Status Tracker interface on the student's personal dashboard, visually displaying the current database state (Unassigned, Pending, Accepted, Rejected, Rejected-Timeout, Withdrawn, Auto-Allocated) alongside the Supervisor's "Reason for Rejection" comments (if applicable) and a live graphical countdown timer for any active pending requests.
- **2.6 FR6** The system shall automatically block a student from initiating any new digital submissions to other supervisors while they currently have a proposal in the "Accepted" state, ensuring a strict 1-to-1 final allocation.
- **2.7 FR7** The system shall automatically revert the student's global allocation status to "Unassigned" immediately following a "Rejected" or

"Rejected-Timeout" proposal state, ensuring the database accurately reflects their availability for subsequent allocation processes.

4. Student Profile Module

1.0 Personal Details

- **1.1 FR1** The system shall provide a secure user interface allowing authenticated students to view and update their mutable profile data (Contact Number, Personal Bio, and External URLs), all of which shall be designated as optional fields, relying on the immutable University Email (Google Chat) as the primary communication channel.
- **1.2 FR2** The system shall strictly lock and disable the input fields for primary immutable identification attributes (Student Name, Student ID, Programme, and University Email, Intake Batch and Academic Status, and CGPA) to prevent unauthorised data mutation.
- **1.3 FR3** The system shall securely retrieve the student's authoritative CGPA from the database and render it as a read-only typography element, formatting the float value to exactly four decimal places (e.g., 3.9000).
- **1.4 FR4** The system shall enforce a strict maximum character limit of 500 characters on the "Personal Bio" text area to prevent database payload bloat.
- **1.5 FR5** The system shall execute backend Regular Expression (Regex) validation via the Facade on the external URL fields to ensure the inputted string is a properly formatted hyperlink (e.g., must begin with a valid URL scheme).
- **1.6 FR6** The system shall securely execute a partial database update command upon the user triggering "Save Changes" action, ensuring that only modified fields are overwritten and unmodified fields retain their existing database state.

- **1.7 FR7** The system shall allow the student to upload a profile avatar image, strictly enforcing frontend validation to accept only MIME types (image/jpeg, image/png) and reject files exceeding a 2.0 MB payload limit.
- **1.8 FR8** The system shall securely store the uploaded physical image file on the server and commit only the resulting file path URL to the student's database record during the save operation.

2.0 Interest Tagging

- **2.1 FR1** The system shall provide an interactive interface containing a predefined, multi-selectable list of "Research Interest" tags.
- **2.2. FR2** The system shall enforce validation constraints allowing the student to select a minimum of one (1) and a maximum of five (5) distinct interest tags.
- **2.3 FR3** The system shall execute a backend database commit to securely save the array of selected tags to the authenticated student's user record.
- **2.4 FR4** The system shall securely expose the committed array of student interest tags via a database query specifically to feed the Recommendation Algorithm located within the Discovery Module.

5. Student Review Module

1.0 Structured Review System

- **1.1 FR1** The system shall provide an evaluation interface allowing students to input a quantitative rating (1 to 5 stars) and qualitative text feedback for their officially assigned supervisor.
- **1.2 FR2** The system shall enforce a strict backend validation check, ensuring a student can only submit a review if they possess a valid supervisor allocation record and the system is currently within the designated review period.
- **1.3 FR3** The system shall automatically aggregate all submitted quantitative ratings and mathematically calculate the average star score for each supervisor.
- **1.4 FR4** The system shall render the calculated average score and the list of qualitative text reviews exclusively on the supervisor's private student review page and the administrator's supervisor review audit pages.

2.0 Anonymous Review Toggle

- **2.1 FR1** The system shall display an interactive Boolean toggle switch labeled "Submit Anonymously" within the review submission interface.
- **2.2 FR2** The system shall parse the state of the toggle upon form submission, if evaluated as true, the system shall render the public author string as "Anonymous Student" on the frontend UI.

- **2.3 FR3** The system shall permanently bind the authenticated user's true Student ID to the submitted review record within the backend database, regardless of the anonymous toggle state, to ensure administrative traceability.

6. Timeline & Milestone Module

1.0 Phase Tracker

- **1.1 FR1** The system shall retrieve the predefined start and end timestamps for all administrative phases (e.g., "Submission Phase", "Review Phase") from the database upon system initialisation.
- **1.2 FR2** The system shall continuously compare the current centralised server time against the retrieved database timestamps to mathematically evaluate and determine the active operational phase.
- **1.3 FR3** The system shall render a persistent visual widget on the student dashboard explicitly broadcasting the title of the currently active phase.
- **1.4 FR4** The system shall dynamically evaluate and execute state transitions to subsequent operational phases on-demand whenever a database request is processed, using live temporal boundary comparisons against system timestamp.

2.0 Deadline Countdown

- **2.1 FR1** The system shall mathematically calculate the remaining time allowance by subtracting the current server time from the active phase's expiration timestamp.

- **2.2 FR2** The system shall render the calculated remaining time in a visual countdown timer formatted dynamically with days, hours, minutes, and/or seconds on the student interface.
- **2.3 FR3** The system shall automatically update the visual countdown timer UI in real-time while periodically synchronizing phase status with the server.
- **2.4 FR4** The system shall automatically invoke a global system lock on the Request & Proposal module, instantly revoking student submission rights the exact moment the countdown timer reaches zero (00:00:00) for the "Submission Phase".

3.4.2 Non-Functional Requirements (NFR)

Non-Functional Requirements (NFR) define the quality attributes, system architecture constraints, and performance baselines of the SSAS. These criteria ensure the system operates reliably under stress and provides a secure, efficient user experience.

1. **Performance Requirements**
   - **NFR 1:** The system shall support a minimum of 500 concurrent student users during the peak hours of the "Submission Phase" without experiencing system degradation, HTTP timeouts, or dropped database connections.
   - **NFR 2:** The system shall execute and propagate the real-time capacity calculation and Dynamic Status Badge update (e.g., changing from "Available" to "Full") across all active client sessions with 500 milliseconds of a successful database commit to prevent over-allocation upon proposal acceptance..
   - **NFR 3:** The system shall process, validate, and securely commit a maximum-capacity (4.9 MB) PDF proposal document to the server storage within 3 seconds under normal network conditions.

- **NFR 4:** The system shall execute the multi-criteria database queries within the Discovery & Search Module (including the Recommendation Algorithm matching) and fully re-render the filtered supervisor list on the UI within 1.5 seconds.
- **NFR 5:** The system shall authenticate user credentials against the database and fully load the respective Role-Based Dashboard within 2 seconds.
- **NFR 6:** The system shall execute the automated global system lock and revoke submission rights within 1 second of the Timeline Countdown Timer reaching zero.

2. **Safety and Security Requirements**
   - **NFR 1:** The system shall transmit all client-server data over a secure HTTPS connection.
   - **NFR 2:** The system shall encrypt all user passwords within the database to prevent plaintext data exposure.
   - **NFR 3:** The system shall enforce Role-Based Access Control (RBAC), explicitly preventing Student accounts from accessing Supervisor or Administrative interfaces.
   - **NFR 4:** The system shall automatically terminate the user session after fifteen (15) minutes of continuous inactivity to protect unattended university laboratory computers.
   - **NFR 5:** The system shall restrict the visibility of a student's sensitive academic data (specifically: Contact Number and CGPA) exclusively to the account owner and the specific Supervisor to whom the student has actively submitted a proposal.
3. **Usability**

- **NFR 1:** The system shall automatically scale and restructure all Graphical User Interfaces (GUI) to render fully operational without requiring horizontal scrolling across viewport widths ranging from 320 pixels (mobile devices) to 1920 pixels (desktop monitors).
- **NFR 2:** The system shall utilize standard sans-serif typography with a minimum body font size of 11 pixels to ensure cross-device readability without requiring user zoom functionalities.
- **NFR 3:** The system shall render a persistent, primary navigation menu on all authenticated dashboards, allowing users to transition between the 'Discovery', 'Profile' and 'Status' modules without utilising the native browser navigation (back / forward) buttons.
- **NFR 4:** The system shall render explicit, non-destructive textual error messages immediately upon any invalid form submission, identifying the specific field of failure and detailing the exact corrective action required by the user.
- **NFR 5:** The system shall display contextual tooltip definitions for all interactive graphical icons (specifically: the 'Available' / 'Full' badges and the 'Accept' / 'Reject' action buttons) upon a user mouse-hover or mobile long-press event.

4. **Reliability**
   - **NFR 1:** The system shall enforce transactional integrity (ACID compliance) during the proposal submission process, if a database commit fails mid-transaction due to network loss, the system shall completely roll back the entry to prevent orphaned or corrupted data.
   - **NFR 2:** The system shall gracefully handle unexpected backend execution errors by rendering a standardised, user-friendly "Service Temporarily Unavailable" UI Page, explicitly preventing the exposure of raw server stack traces (e.g., the "White Screen of Death") to the end-user.
   - **NFR 3:** The system shall temporarily cache partially completed text inputs within the user's local browser storage (SessionStorage) to ensure data is not lost if the user accidentally refreshes the page prior to form submission.
5. **Capacity**
   - **NFR 1:** The system shall allocate a minimum of 150 Gigabytes (GB) of dedicated server file storage specifically to accommodate the maximum potential volume of student PDF proposal uploads for a single academic year

(calculated as 15,000 maximum students x 5.0 MB per file, inclusive of a 100% safety bugger).

- **NFR 2:** The system database shall be structurally configured to efficiently store, index and query a minimum of 100,000 concurrent relational records (comprising User Profiles, Proposal Statuses, and Structural Reviews) without experiencing query delays exceeding 1.0 second.
- **NFR 3:** The system shall restrict the maximum upload size of any single digital project proposal strictly to 5.0 Megabytes (MB) to mathematically enforce the 150 GB overall server storage capacity constraint.
- **NFR 4:** The system shall provide an administrative data-retention function to safely archive and purge all stored PDF proposals at the exact conclusion of the academic year, resetting the active file storage capacity back to 0% for the subsequent student cohort.

3.5 Chapter Summary and Evaluation

This chapter successfully transitioned the Supervisor Selection and Allocation System (SSAS) from an abstract concept into a definitive technical blueprint using the Waterfall methodology in Section 3.1. By locking in TAR UMT's strict administrative policies, such as workload quotas and academic deadlines, the project effectively mitigated the risk of late-stage scope creep.

In section 3.3, a mixed-methods requirement elicitation strategy involving stakeholder interviews, system observations, and user questionnaires, critical flaws in the legacy manual system were identified. These operational bottlenecks were structurally resolved and translated into precise Use Case Models in Section 3.3 and Functional Requirements (FRs) in Section 3.4, establishing the logic for core features like automated Submission Blocking, the Recommendation Algorithm, and real-time Status Trackers. Additionally, Non-Functional Requirements (NFRs) were defined in Section 3.4 to guarantee system performance, security, and ACID compliance under peak academic load.

With the requirements phase now formally frozen and mathematically verified, the project advances to Chapter 4: System Design. This next phase will translate these documented requirements into a structural architecture, focusing on the Database Entity-Relationship Diagram (ERD), system behaviour models, and User Interface (UI) wireframes.

Chapter 4

System Design

4 System Design

This chapter details the comprehensive system design of the Smart Supervisor Allocation System (SSAS), serving as the technical blueprint for the implementation phase. It bridges the gap between the theoretical requirements established in Chapter 3 and the physical construction of the software. The chapter translates functional constraints and user workflows into rigorous technical specifications across seven core domains. It begins by mapping system behaviour via Process Design, followed by a robust Data Design encompassing Data Flow Diagrams (DFD) and relational Entity-Relationship (ER) models, Subsequent sections detail the User Interface (UI) and the Security architectures. The chapter culminates in an in-depth Structural Design, System Architecture Design, and the explicit mathematical Algorithm Design required to power the system's temporal security and intelligent recommendation engines.

4.1 Process Design (Activity Diagram)

Following the formal specification of system requirements in Chapter 3, this section transitions into the dynamic behavioural design of the Supervisor Selection and Allocation System (SSAS). Activity Diagrams are utilised to graphically represent the step-by-step procedural flow of control for each core module. By translating the previously defined Use Case Descriptions into visual flowcharts, these diagrams map the exact execution sequences, conditional logic (Alternate Flows), and backend data validation processes. Furthermore, the use of partitions (swimlanes) explicitly delineates the operational boundaries between the Student's frontend interactions and the System's automated backend executions, providing a clear and traceable blueprint for the subsequent implementation phase..

4.1.1 Supervisor Availability & Status Module

UC100_VIEW_SUPERVISOR_AVAILABILITY

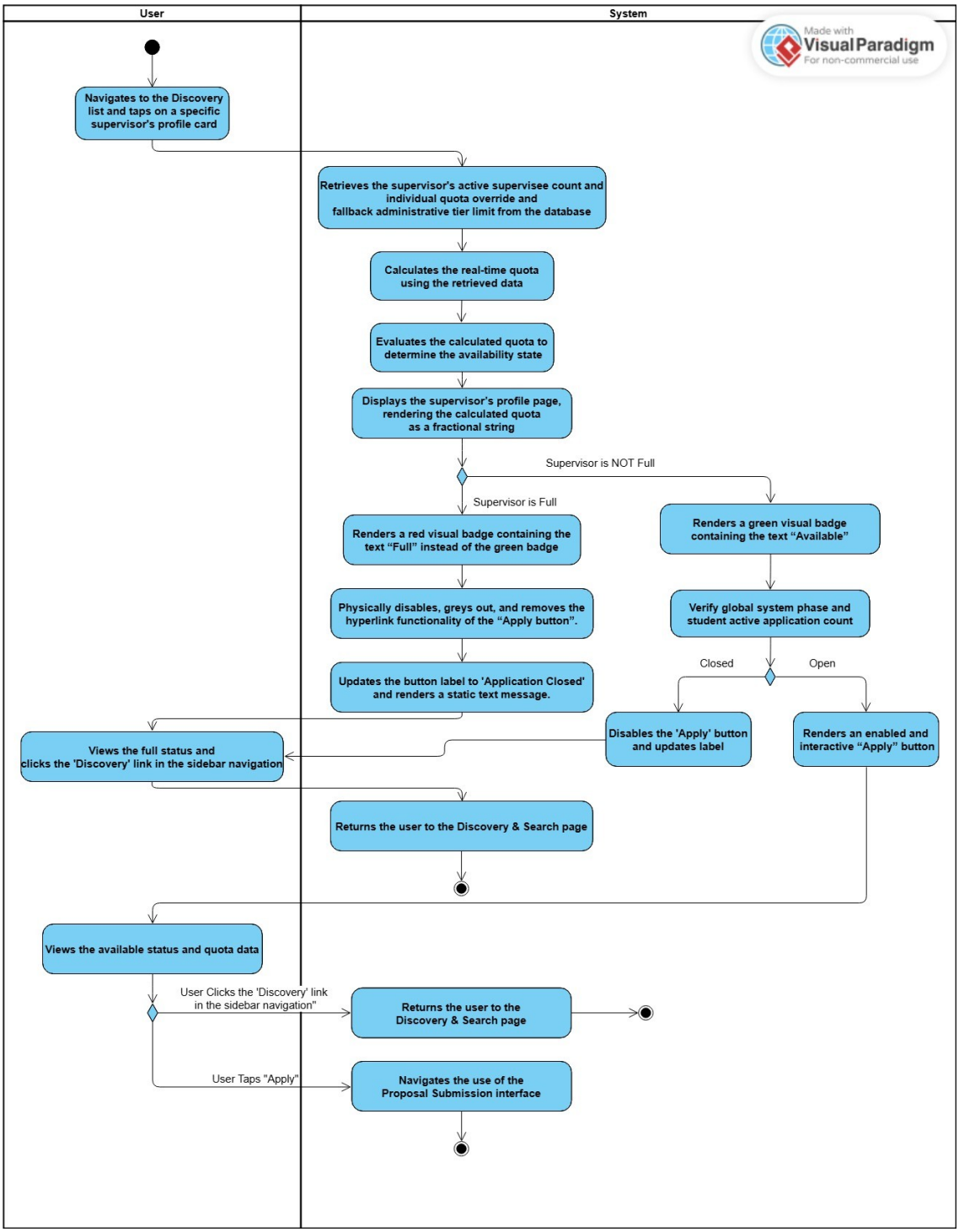

Figure 4.1: Activity Diagram for UC100_VIEW_SUPERVISOR_AVAILABILITY

4.1.2 Discovery & Search Module

UC200_SEARCH_AND_FILTER_SUPERVISORS

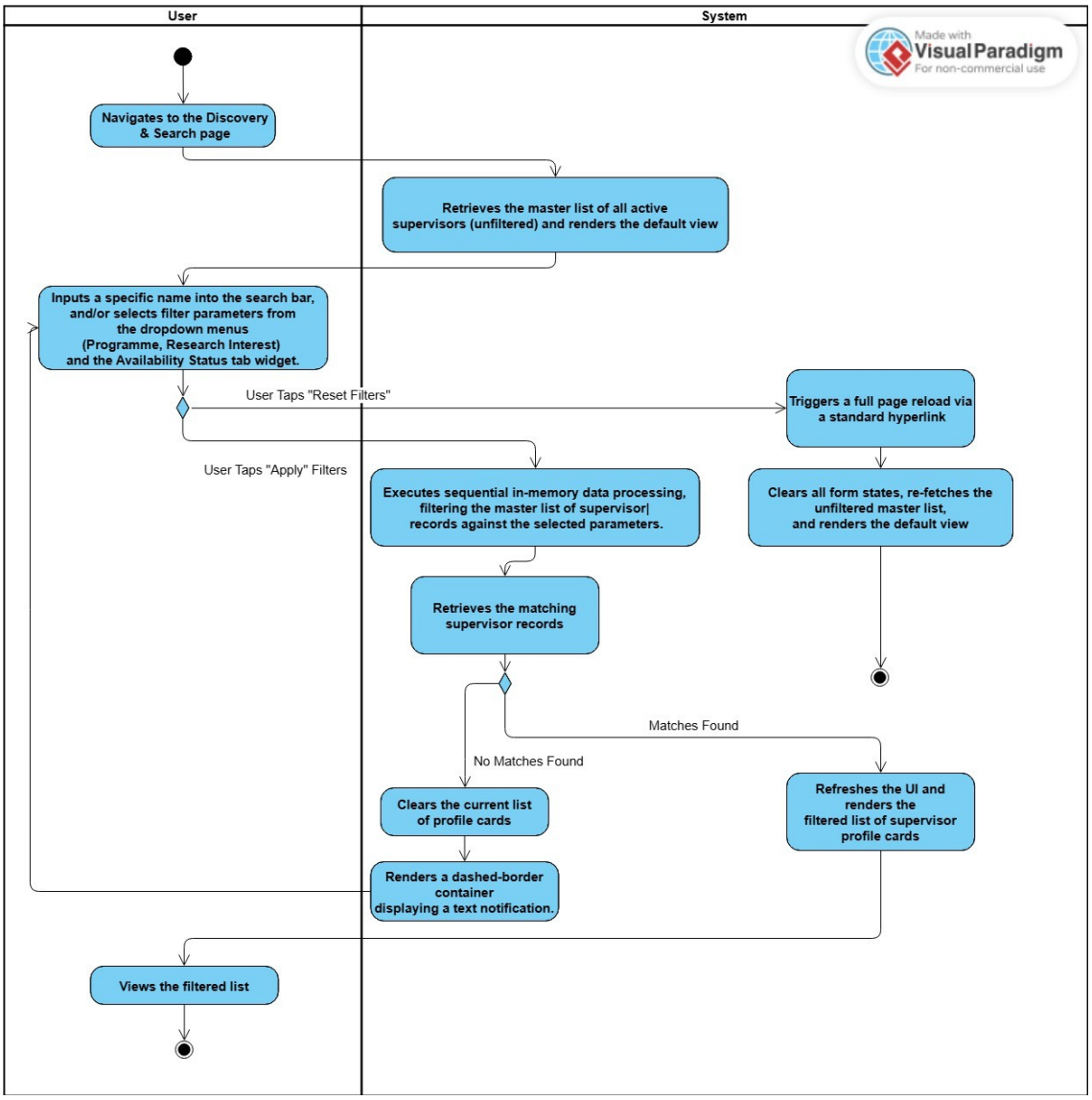

Figure 4.2: Activity Diagram for UC200_SEARCH_AND_FILTER_SUPERVISORS

UC201_VIEW_RECOMMENDED_MATCHES

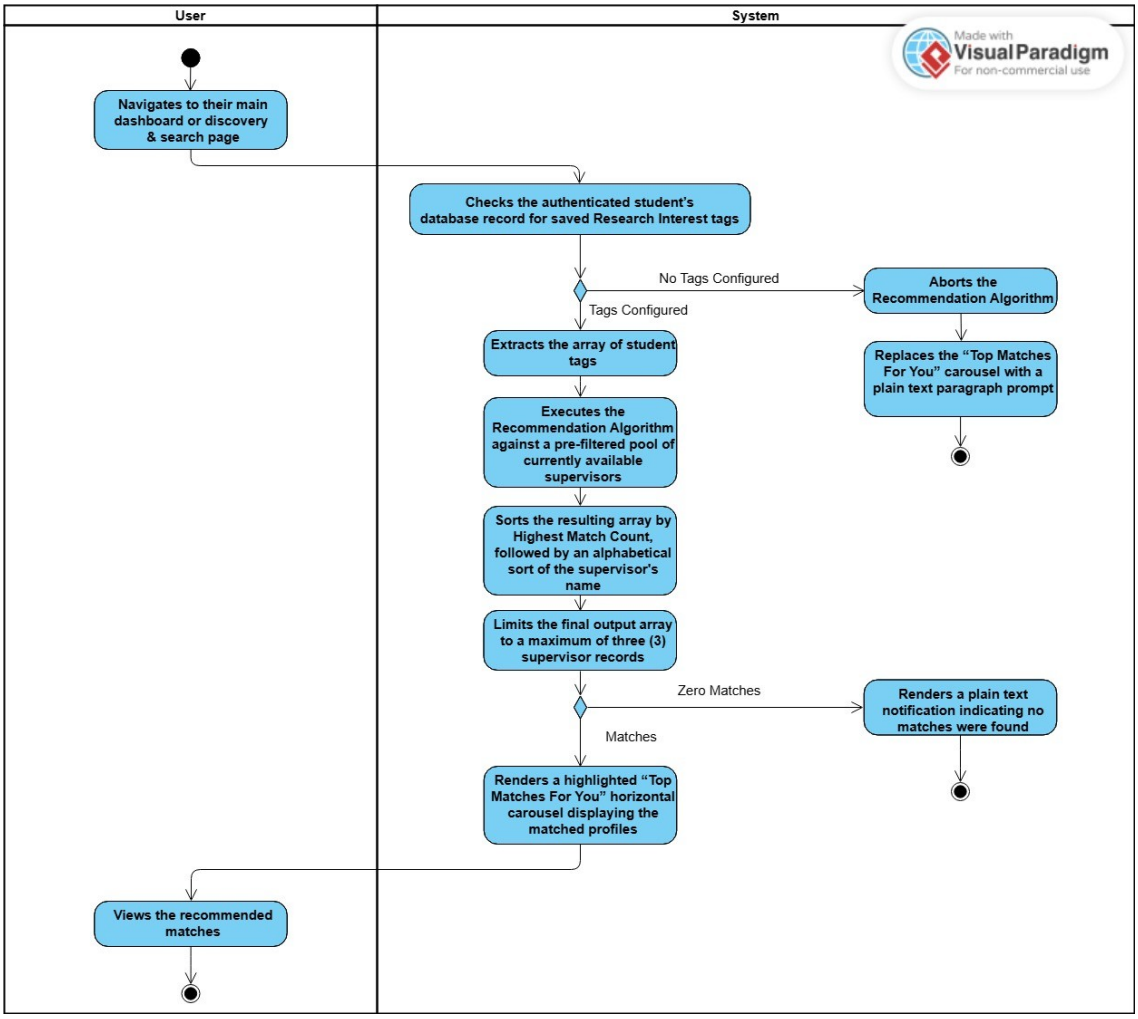

Figure 4.3: Activity Diagram for UC201_VIEW_RECOMMENDED_MATCHES

4.1.3 Request & Proposal Module

UC300_SUBMIT_PROJECT_PROPOSAL

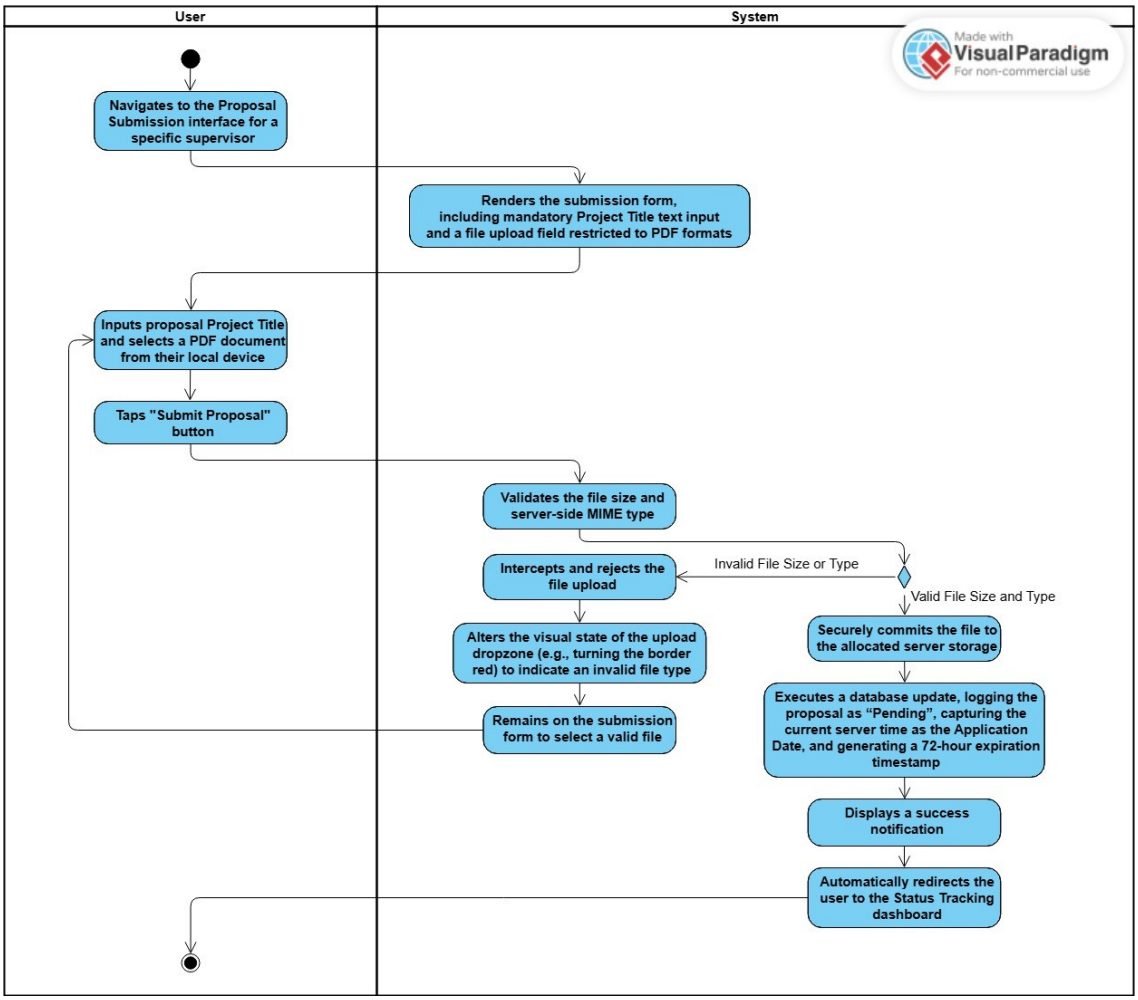

Figure 4.3: Activity Diagram for UC300_SUBMIT_PROJECT_PROPOSAL

UC301_TRACK_APPLICATION_STATUS

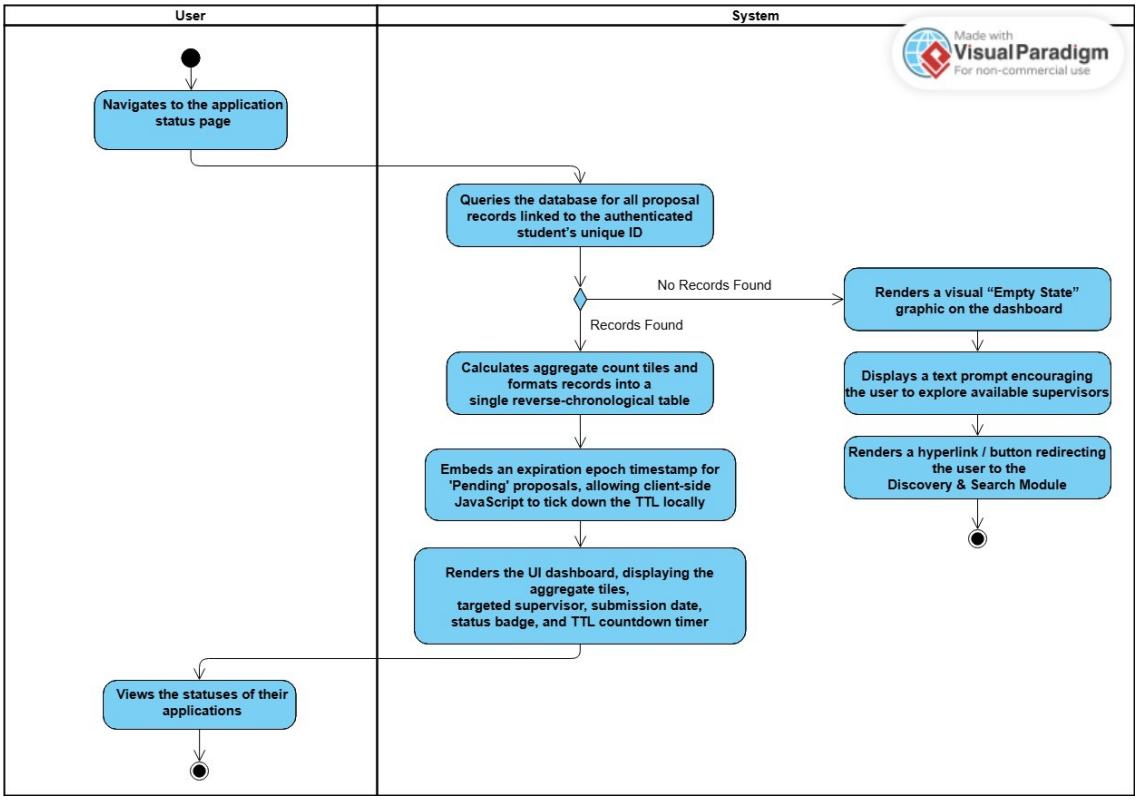

Figure 4.4: Activity Diagram for UC301_TRACK_APPLICATION_STATUS

Pseudocode:

```
START

   1. INPUT searchName (String), reqProgramme (String), reqInterests (Array), and reqAvailability (String)
   2. SET finalResults to empty array []
   3. RETRIEVE allSupervisors

      FROM USER

      WHERE systemRole EQUALS “Supervisor”

   4. FOR EACH supervisor IN allSupervisors DO


            // Filter 1: Name Search (Text Input)

            IF searchName is NOT EMPTY AND supervisor.fullName DOES NOT CONTAIN searchName THEN

                  CONTINUE to next supervisor

            ENDIF


            // Filter 2: Programme Dropdown

            IF reqProgramme is NOT EMPTY AND supervisor.programme DOES NOT EQUAL reqProgramme THEN

                  CONTINUE to next supervisor

            ENDIF


            // Filter 3: Research Interests Dropdown
```